Practical no. 1(C)

Aim: Write the program for the following:
c. Write C# code to arrange the name of cities in sorted order. Accept name of 10 cities from the user.

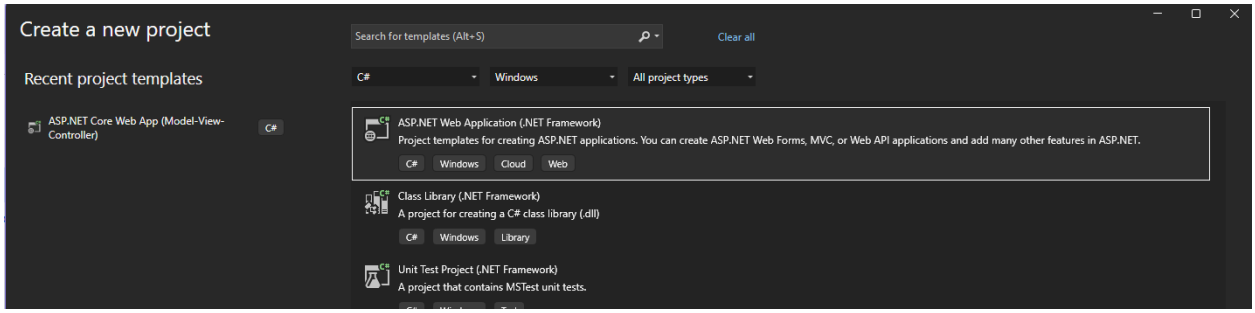
Software Required:

Visual Studio 2026

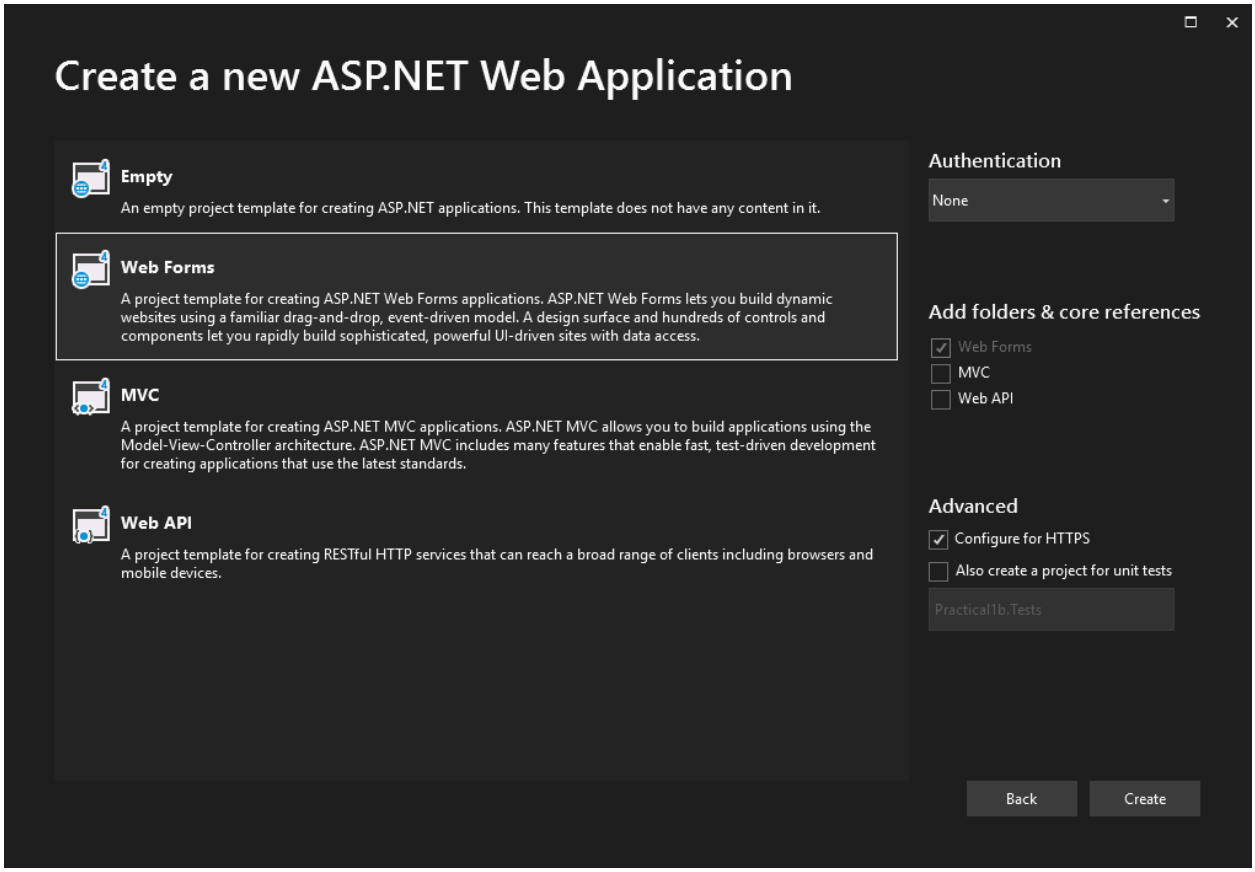
Procedure:

Step 1: Create New Project

1. Open Visual Studio.
2. Click **Create a new project**.
3. Select **ASP.NET Web Application (.NET Framework)**.



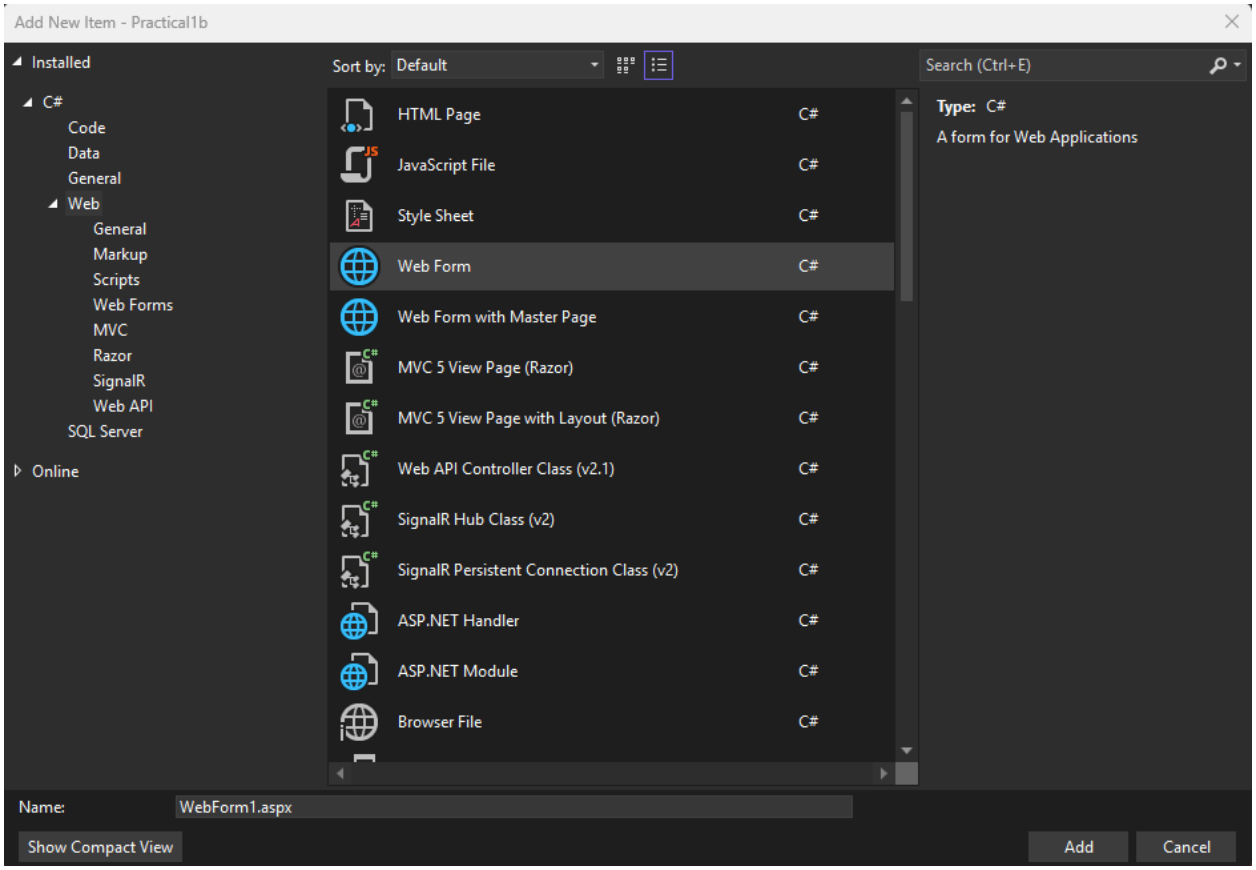
4. Click **Next**.
5. Enter Project Name: **Practical1C**
6. Click **Create**.
7. Select **Web Forms**.



8. Click **Create**.

Step 2: Add a Web Form

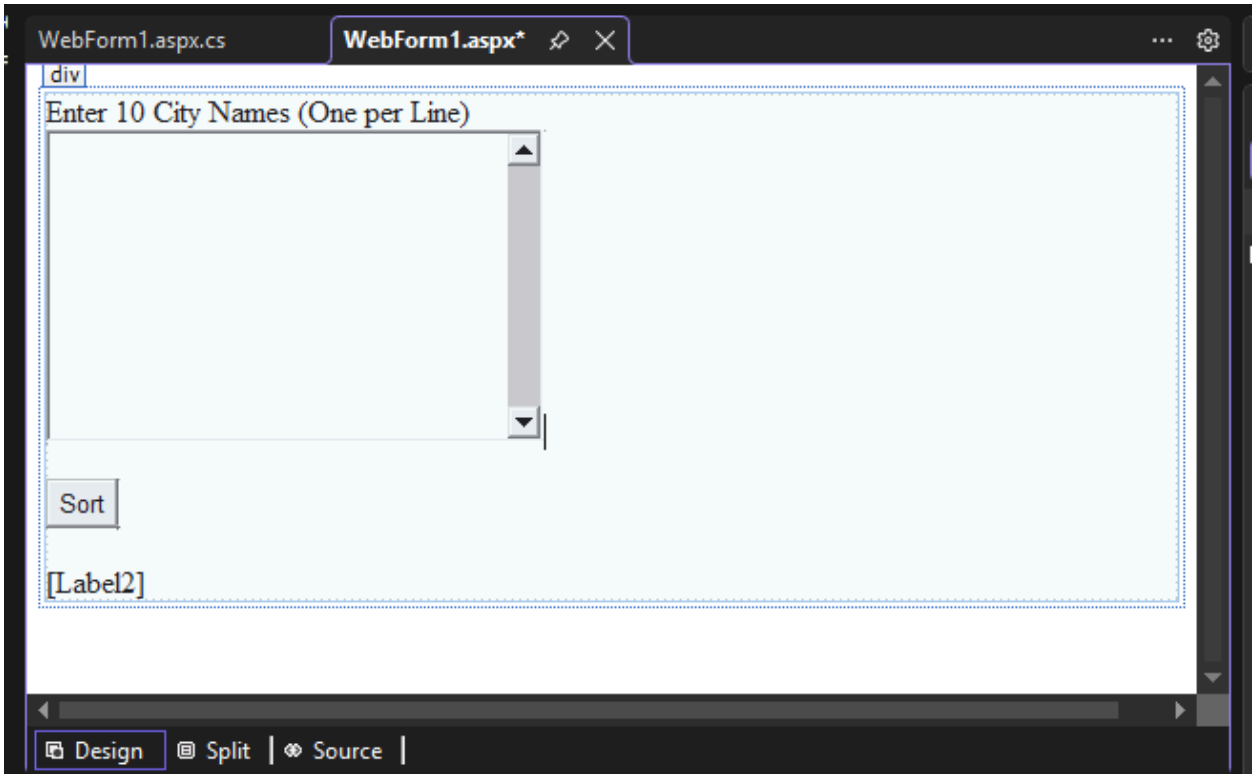
1. In Solution Explorer, right-click the project **Practical1C**.
2. Select Add → New Item.
3. Select Web Form.
4. Click **Add**.



Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. Label
2. TextBox
3. Button
4. Label



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;

namespace CitySort
{
   public partial class WebForm1 : System.Web.UI.Page
   {
      protected void Page_Load(object sender, EventArgs e)
      {

      }

      protected void Button1_Click(object sender, EventArgs e)
      {
         string[] cities;

         cities = TextBox1.Text.Split(
```

```
            new string[] { Environment.NewLine },
            StringSplitOptions.RemoveEmptyEntries);

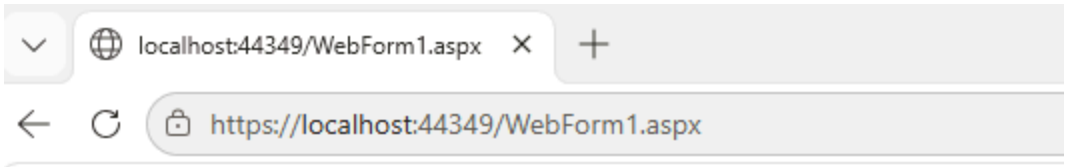
        Array.Sort(cities);

        Label2.Text = "<b>Sorted City Names:</b><br/>";

        foreach (string city in cities)
        {
            Label2.Text += city + "<br/>";
        }
    }
  }
}
```

Step 5: Run the Application

1. Save all files.
2. Click Build → Build Solution.
3. Press **F5**.
4. Browser window will open.
5. Enter the 10 city names in the multiline TextBox .
6. Click **Sort**.



Enter 10 City Names (One per Line)

```
Mumbai
Delhi
Pune
Nagpur
Chennai
Kolkata
Ahmedabad
Jaipur
Surat
Hyderabad
```

Sort

Sorted City Names:

Ahmedabad
Chennai
Delhi
Hyderabad
Jaipur
Kolkata
Mumbai
Nagpur
Pune
Surat

Practical no. 1(a)

Aim: Write the program for the following:
a. Create an MVC application to perform basic Arithmetic Operations.

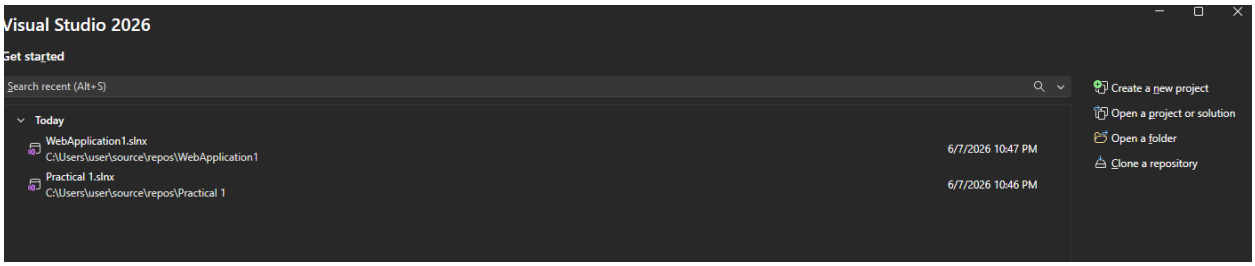
Software Required:

Visual Studio 2026

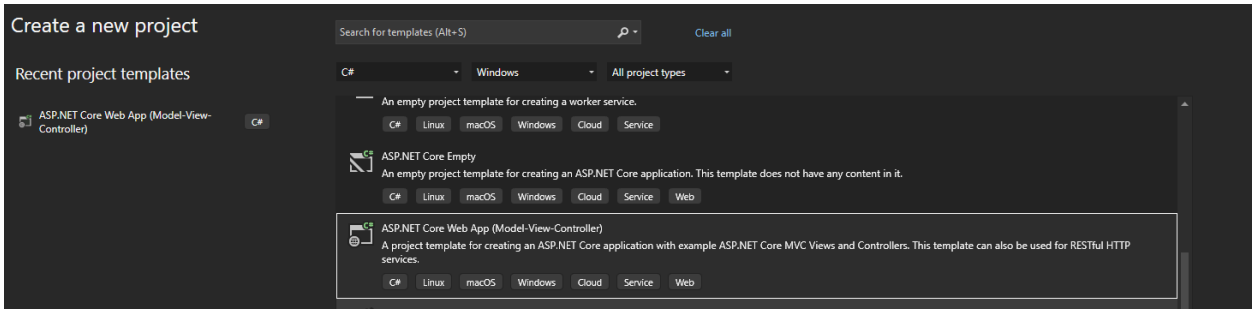
Procedure:

Step 1: Create New MVC Project

1. Open Visual Studio 2022.
2. Click Create a New Project.



3. Select ASP.NET Core Web App (Model-View-Controller).



4. Click Next.
5. Enter Project Name: ArithmeticMVC
6. Click Next.
7. Select .NET 8.0.
8. Click Create.

Step 2: Create Model

1. Click View.

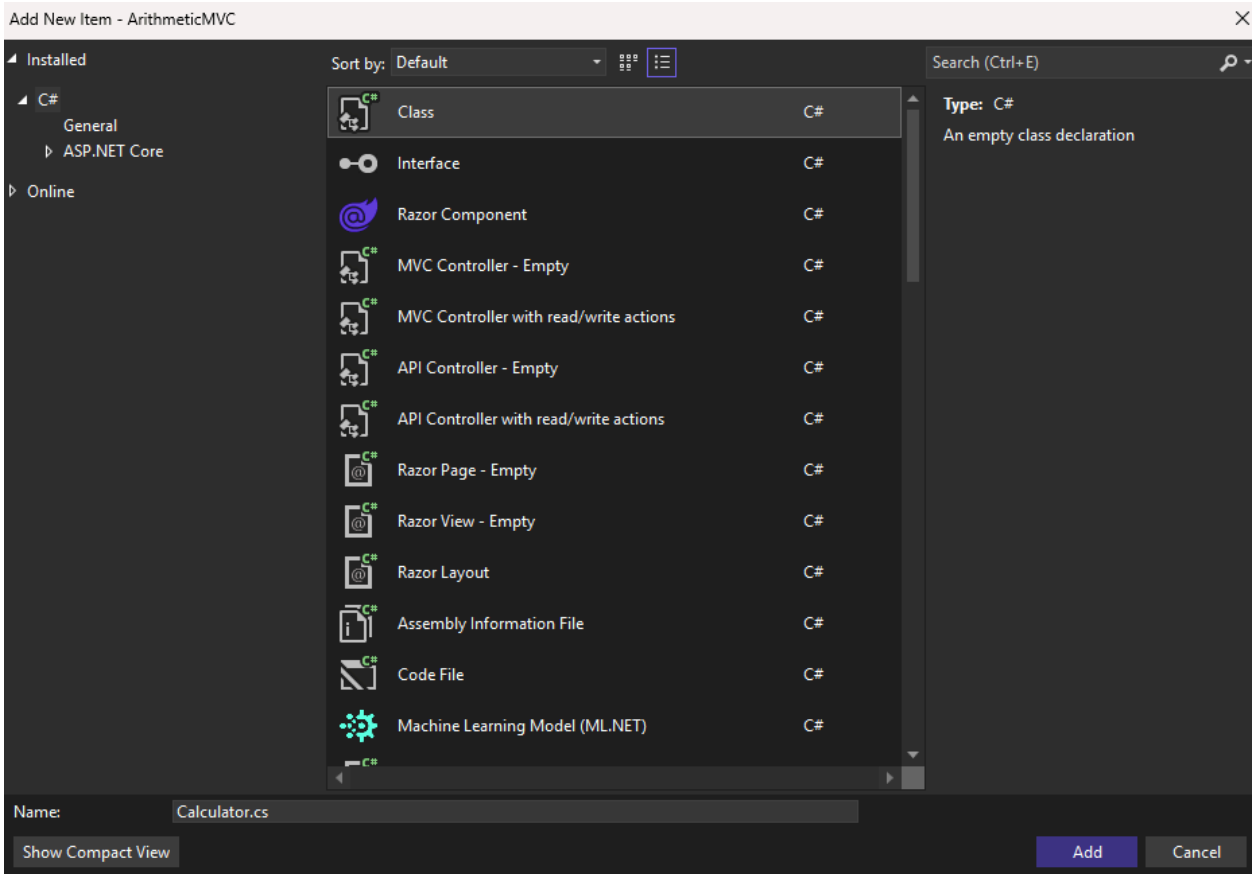
2. Click Solution Explorer..

Create a class named Calculator inside the Models folder.

1. Right Click Models.
2. Click Add.
3. Click New item.
4. Select Class.
5. Give Name Calculator.cs.
6. Click Add.

Calculator.cs

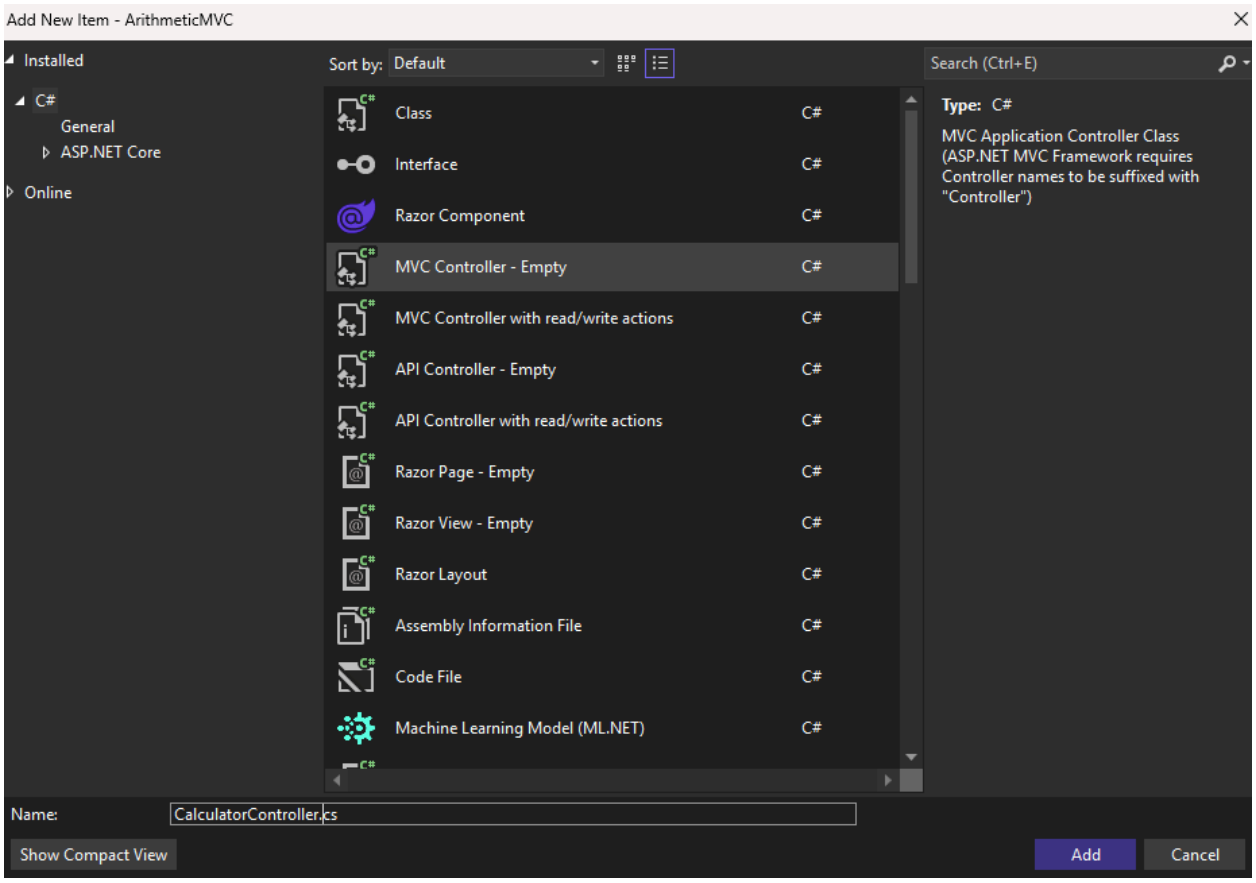
```
namespace ArithmeticMVC.Models
{
   public class Calculator
   {
      public double Number1 { get; set; }
      public double Number2 { get; set; }
      public string Operation { get; set; }
      public double Result { get; set; }
   }
}
```



Step 3: Create Controller

Create a CalculatorController inside the Controllers folder.

1. Right Click Controllers.
2. Click Add.
3. Click New item.
4. Select MVC Controller - Empty.
5. Give Name CalculatorController.
6. Click Add.



CalculatorController.cs

```
using Microsoft.AspNetCore.Mvc;
using ArithmeticMVC.Models;

namespace ArithmeticMVC.Controllers
{
   public class CalculatorController : Controller
   {
      public IActionResult Index()
      {
         return View();
      }

      [HttpPost]
      public IActionResult Index(Calculator calc)
      {
         switch (calc.Operation)
         {
            case "+":
```

```
                calc.Result = calc.Number1 + calc.Number2;
                break;

            case "-":
                calc.Result = calc.Number1 - calc.Number2;
                break;

            case "*":
                calc.Result = calc.Number1 * calc.Number2;
                break;

            case "/":
                if (calc.Number2 != 0)
                    calc.Result = calc.Number1 / calc.Number2;
                else
                    ViewBag.Message = "Division by zero is not allowed.";
                break;
        }

        return View(calc);
    }
  }
}
```

Step 4: Create View

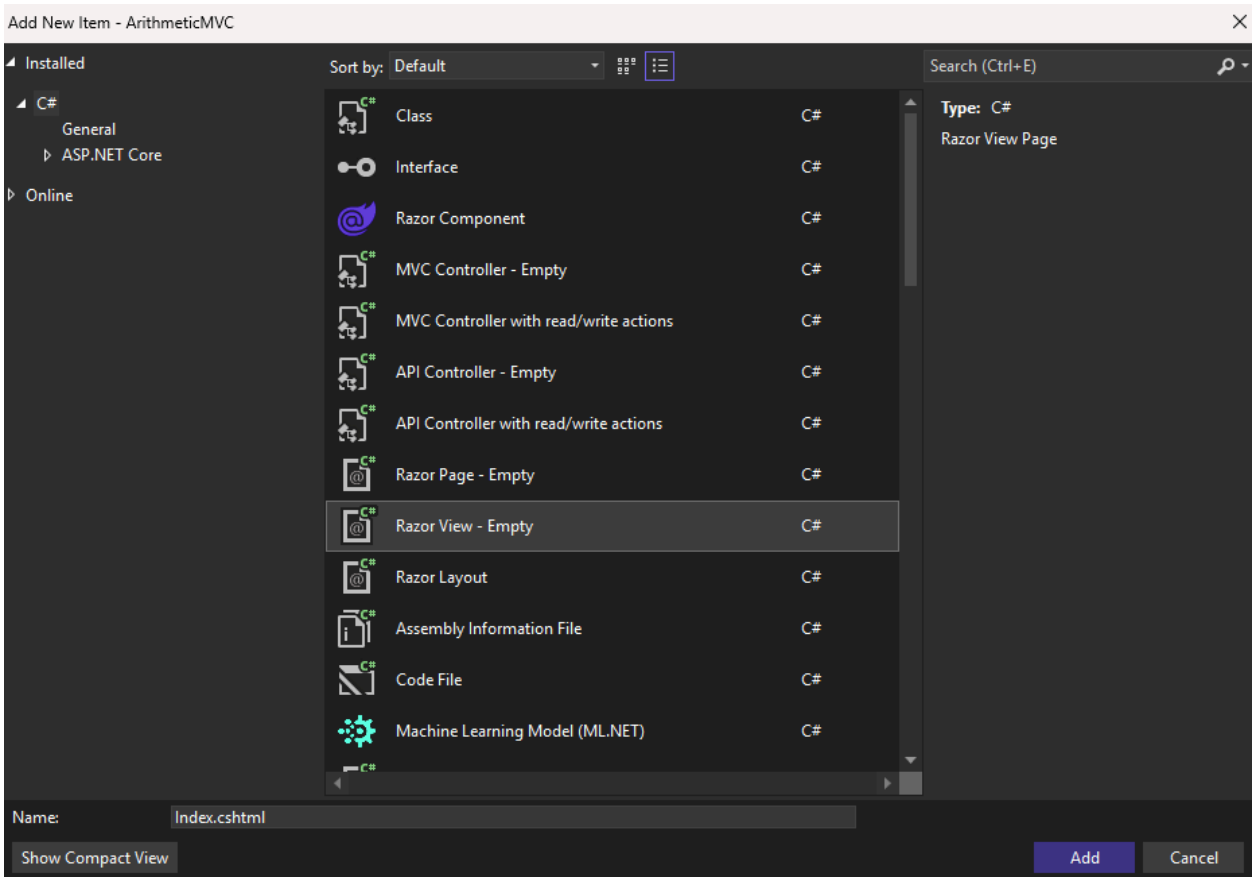
Create the following folder structure:

```
Views
 └── Calculator
     └── Index.cshtml
```

Steps to Create View

1. Right-click Views folder.
2. Select Add → New Folder.
3. Name the folder Calculator.
4. Right-click Calculator folder.
5. Select Add → New Item.
6. Select Razor View.
7. Name it Index.cshtml.

8. Click Add.



Step 5: Add View Code

Index.cshtml

```
@model ArithmeticMVC.Models.Calculator

@{
    ViewData["Title"] = "Calculator";
}

<h2>Arithmetic Calculator</h2>

<form method="post">

    <div>
        <label>First Number:</label>
        <input asp-for="Number1" />
    </div>
```

```
    <br />

    <div>
        <label>Second Number:</label>
        <input asp-for="Number2" />
    </div>

    <br />

    <div>
        <label>Operation:</label>

        <select asp-for="Operation">
            <option value="+">Addition</option>
            <option value="-">Subtraction</option>
            <option value="*">Multiplication</option>
            <option value="/">Division</option>
        </select>
    </div>

    <br />

    <button type="submit">Calculate</button>

</form>

@if (Model != null)
{
    <h3>Result = @Model.Result</h3>
}

@if (ViewBag.Message != null)
{
    <h3 style="color:red">@ViewBag.Message</h3>
}
```

Step 6: Open Program.cs file

Solution Explorer → Program.cs → Change MapControllerRoute.

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production scenarios,
see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

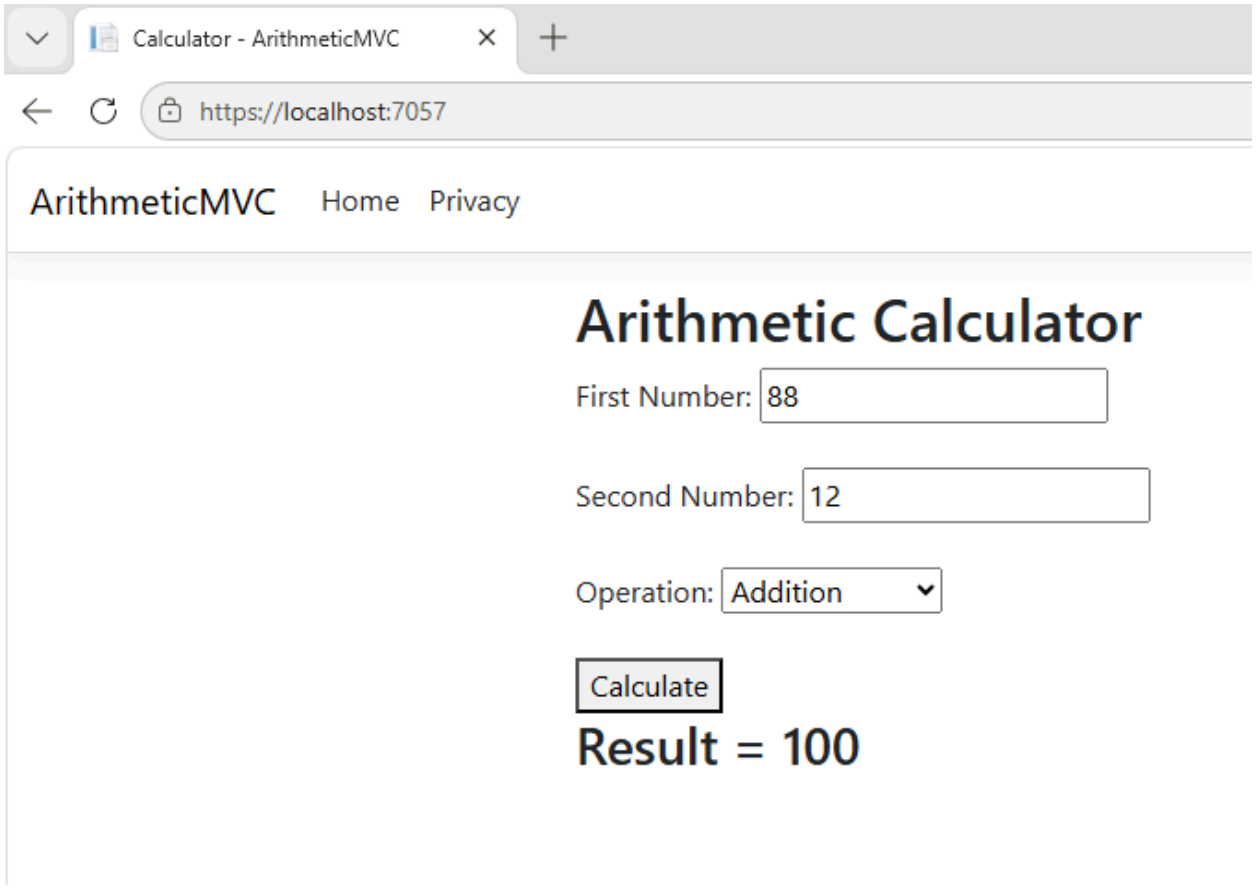
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=calculator}/{action=Index}/{id?}")
    .WithStaticAssets();


app.Run();
```

Step 7: Run Application

1. Click Build → Build Solution.
2. Press F5.
3. The browser opens automatically.
4. Enter two numbers.
5. Select operation.
6. Click Calculate.

Output:


Calculator - ArithmeticMVC
https://localhost:7057
ArithmeticMVC
Home
Privacy
Arithmetic Calculator
First Number: 88
Second Number: 12
Operation: Addition
Calculate
Result = 100

Practical no. 2

Aim: Write the program with different features of C#
a. Function Overloading
b. Inheritance (all types)
c. Constructor overloading
d. Interfaces
e. Using Delegates and events
f. Exception handling

Software Required:

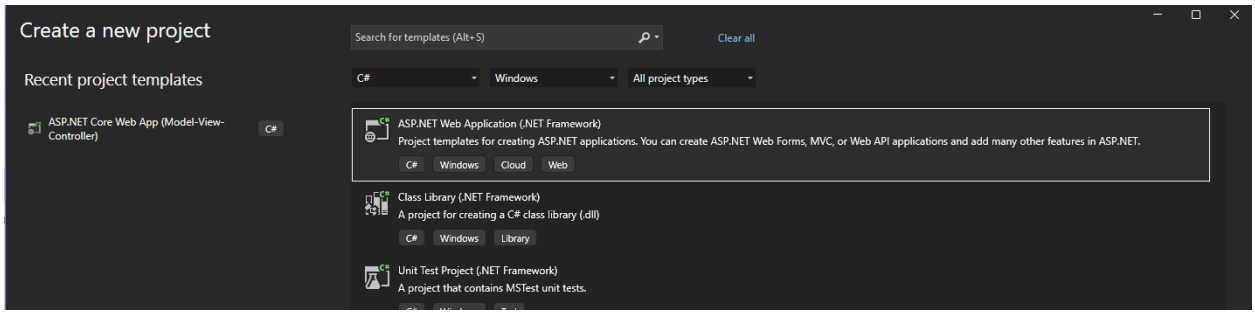
Visual Studio 2026

Procedure:

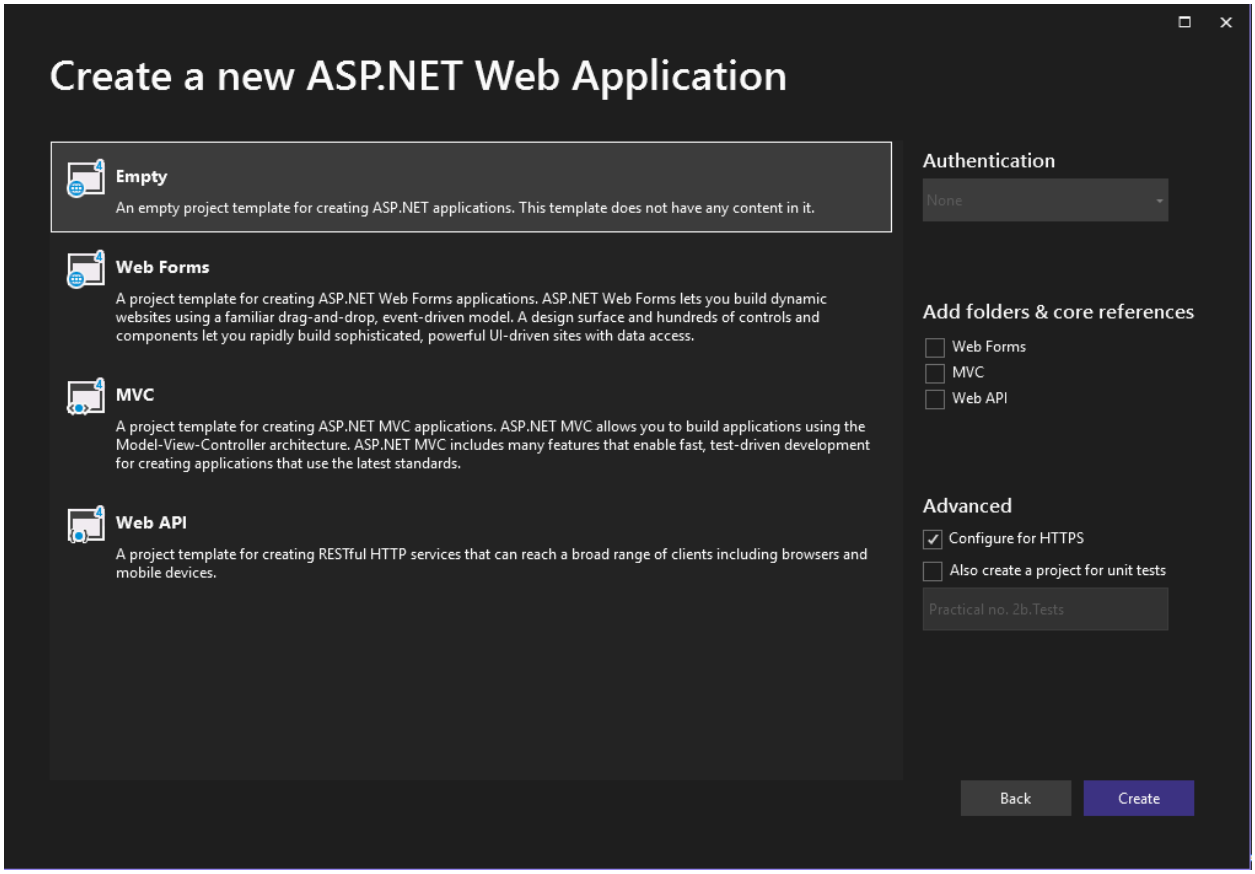
a. Function Overloading

Step 1: Create New Project

1. Open Visual Studio.
2. Click **Create a new project**.
3. Select **ASP.NET Web Application (.NET Framework)**.



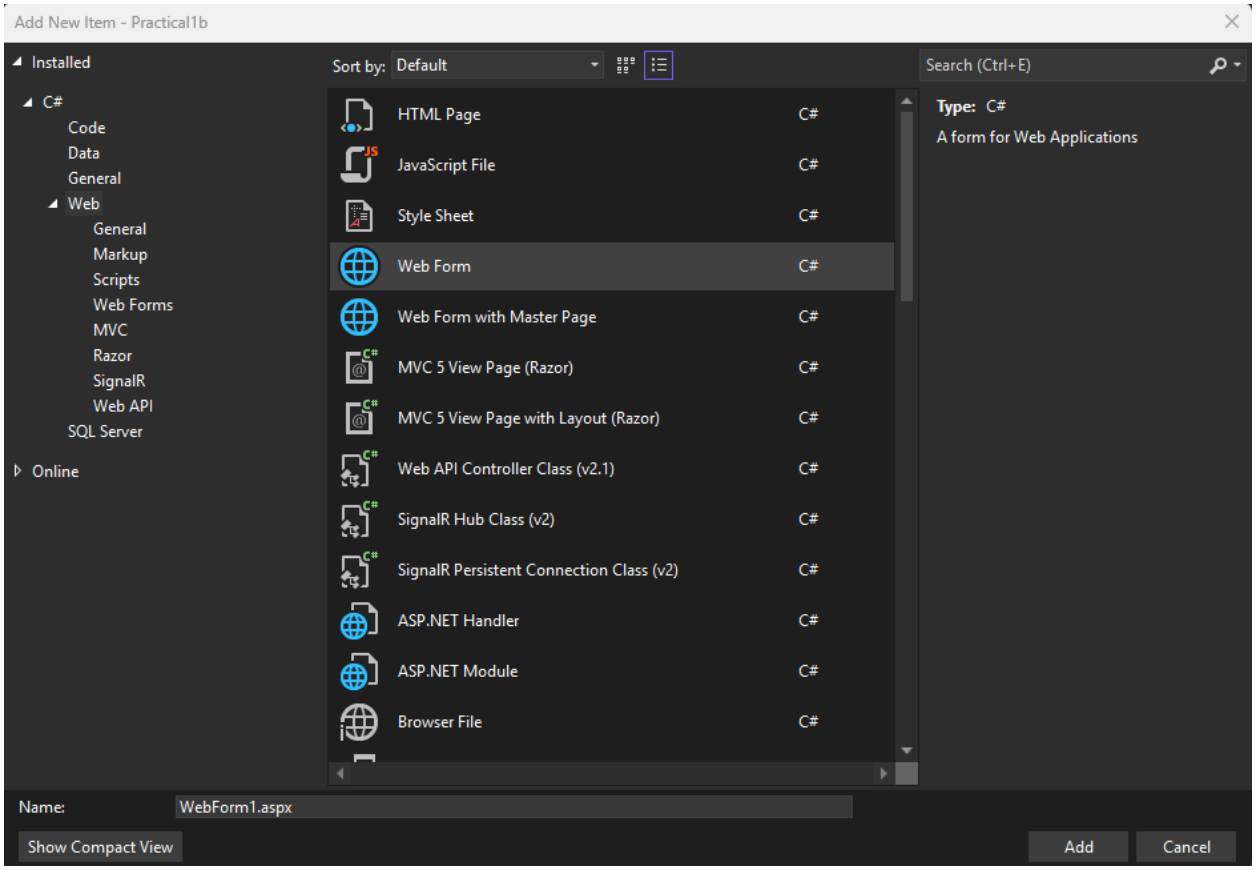
4. Click **Next**.
5. Enter Project Name: **Practical2**
6. Click **Create**.
7. Select **Empty**.



8. Click **Create**.

Step 2: Add a Web Form

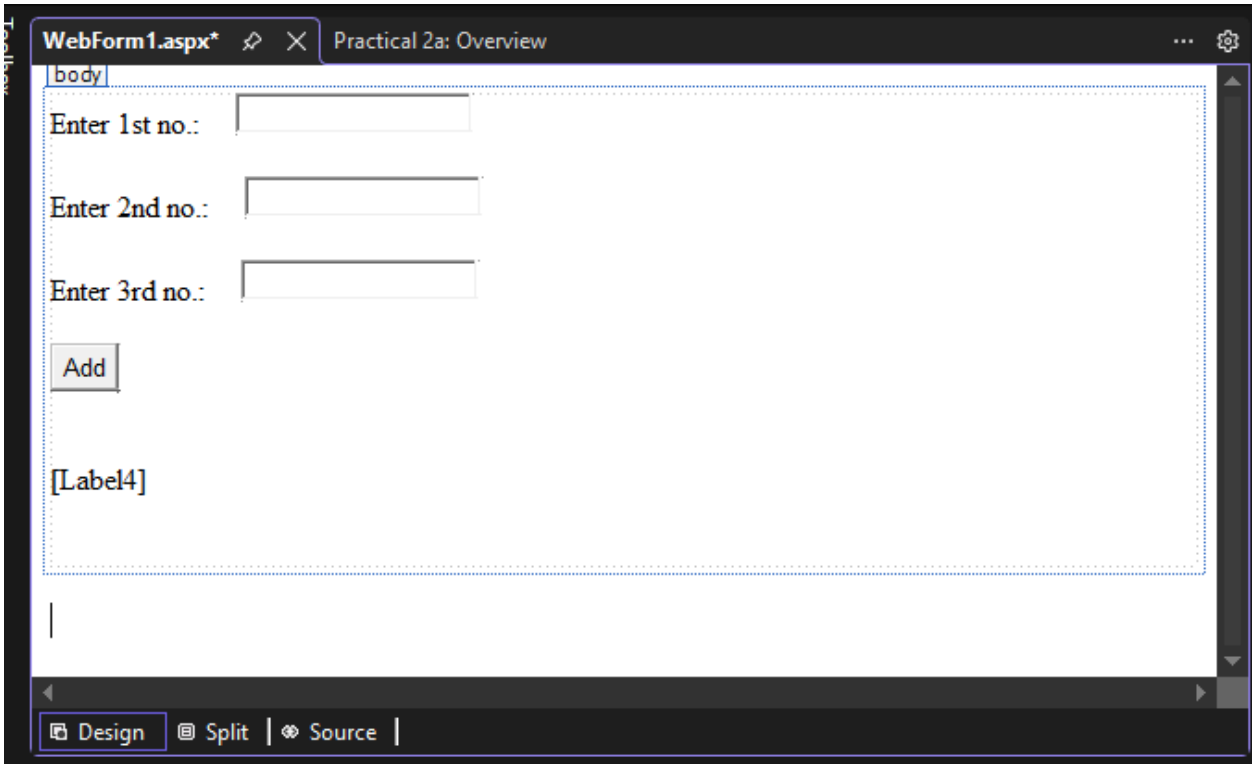
1. In Solution Explorer, right-click the project **Practical2**
2. Select Add → New Item.
3. Select Web Form.
4. Click **Add**.



Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. TextBox
2. Button
3. Label



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical_2a
{
    public partial class WebForm1 : System.Web.UI.Page
    {
        public int Add(int a, int b)
        {
            return a + b;
        }

        public int Add(int a, int b, int c)
        {
```

```
        return a + b + c;
    }
    protected void Page_Load(object sender, EventArgs e)
    {

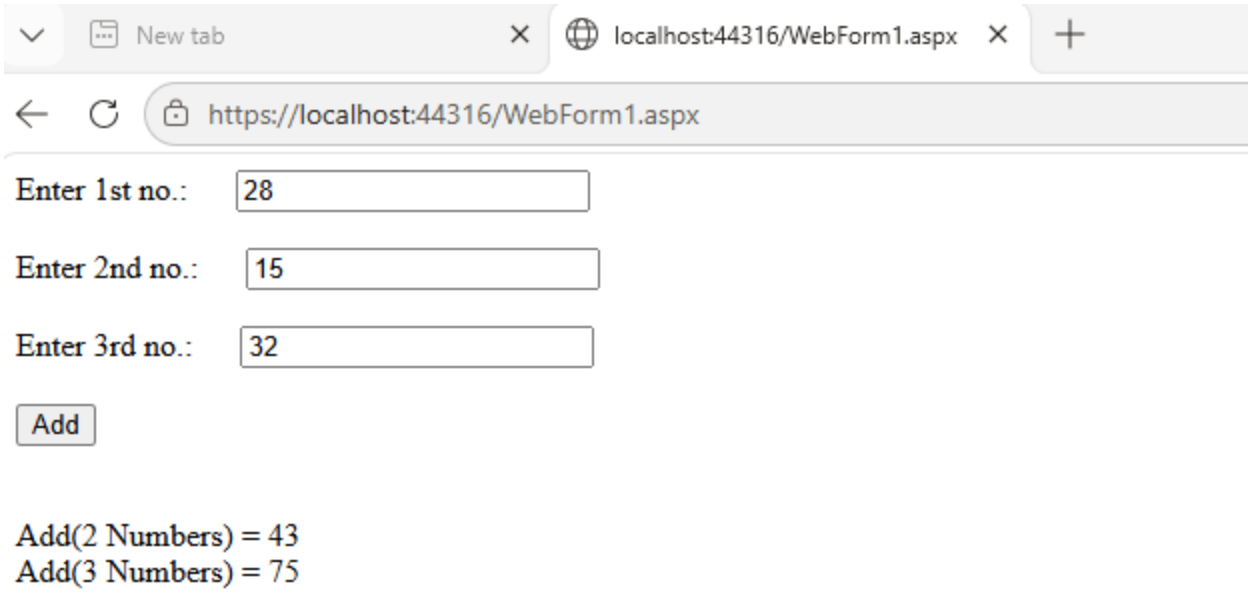
    }

    protected void Button1_Click(object sender, EventArgs e)
    {
        int a = Convert.ToInt32(TextBox1.Text);
        int b = Convert.ToInt32(TextBox2.Text);
        int c = Convert.ToInt32(TextBox3.Text);

        Label4.Text =
        "Add(2 Numbers) = " + Add(a, b) +
        "<br/>Add(3 Numbers) = " + Add(a, b, c);
    }
  }
}
```

Step 5: Run the Application

1. Save all files.
2. Click Build →Build Solution.
3. Press F5.
4. Browser window will open.
5. Enter three numbers in TextBox .
6. Click **Add**.



b. Inheritance (all types)

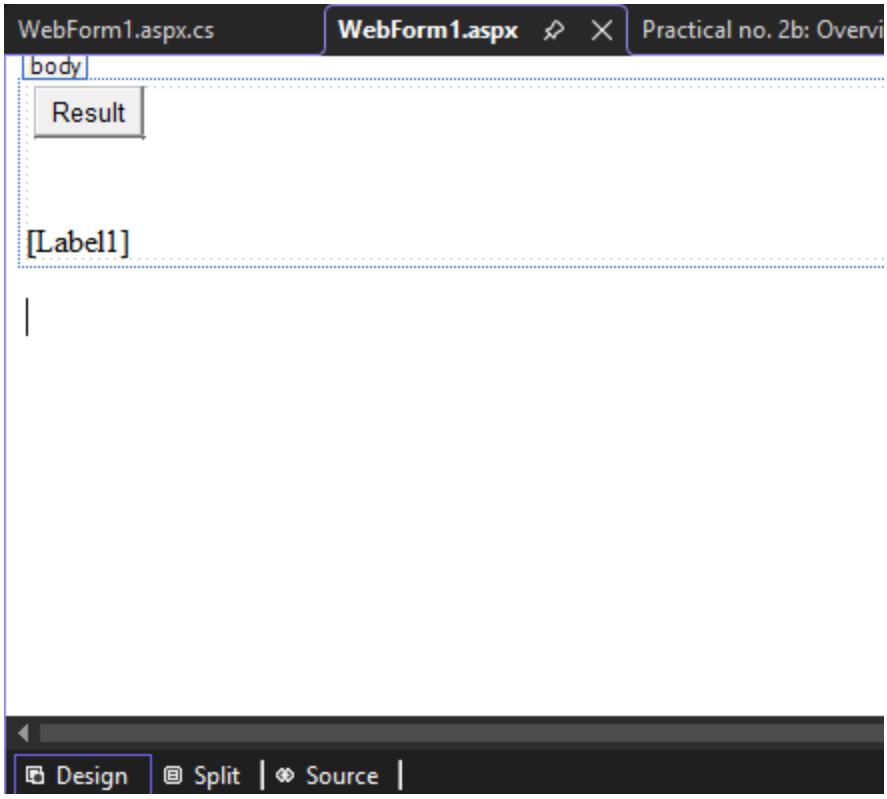
Procedure:

Step 1 and Step 2 are the same as the previous practical.

Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. Button
2. Label



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical_no._2b
{
   public partial class WebForm1 : System.Web.UI.Page
   {

      // SINGLE INHERITANCE

      class Person
      {
```

```
    public string ShowName()
    {
        return "Person Class";
    }
}

class Student : Person
{
    public string Study()
    {
        return "Student is Studying";
    }
}
```

// MULTILEVEL INHERITANCE

```
class GrandFather
{
    public string Grand()
    {
        return "GrandFather";
    }
}

class Father : GrandFather
{
    public string Dad()
    {
        return "Father";
    }
}

class Son : Father
{
    public string Child()
    {
        return "Son";
    }
```

```
}
```

// HIERARCHICAL INHERITANCE

```
class Animal
{
   public string Eat()
   {
      return "Animal Eats Food";
   }
}

class Dog : Animal
{
   public string Bark()
   {
      return "Dog Barks";
   }
}

class Cat : Animal
{
   public string Meow()
   {
      return "Cat Meows";
   }
}
```

//MULTIPLE INHERITANCE

```
interface ISports
{
   string Play();
}

class Teacher
{
   public string Teach()
```

```
    {
        return "Teacher Teaches";
    }
}

class SportsTeacher : Teacher, ISports
{
    public string Play()
    {
        return "Sports Teacher Plays Cricket";
    }
}



protected void Page_Load(object sender, EventArgs e)
{

}

protected void Button1_Click(object sender, EventArgs e)
{
    Label1.Text = "";

    // Single Inheritance

    Student s = new Student();

    Label1.Text += "<b>SINGLE INHERITANCE</b><br/>";
    Label1.Text += s.ShowName() + "<br/>";
    Label1.Text += s.Study() + "<br/><br/>";

    // Multilevel Inheritance

    Son son = new Son();

    Label1.Text += "<b>MULTILEVEL INHERITANCE</b><br/>";
    Label1.Text += son.Grand() + "<br/>";
    Label1.Text += son.Dad() + "<br/>";
    Label1.Text += son.Child() + "<br/><br/>";
```

```
        // Hierarchical Inheritance

        Dog d = new Dog();
        Cat c = new Cat();

        Label1.Text += "<b>HIERARCHICAL INHERITANCE</b><br/>";
        Label1.Text += d.Eat() + "<br/>";
        Label1.Text += d.Bark() + "<br/>";
        Label1.Text += c.Eat() + "<br/>";
        Label1.Text += c.Meow() + "<br/><br/>";

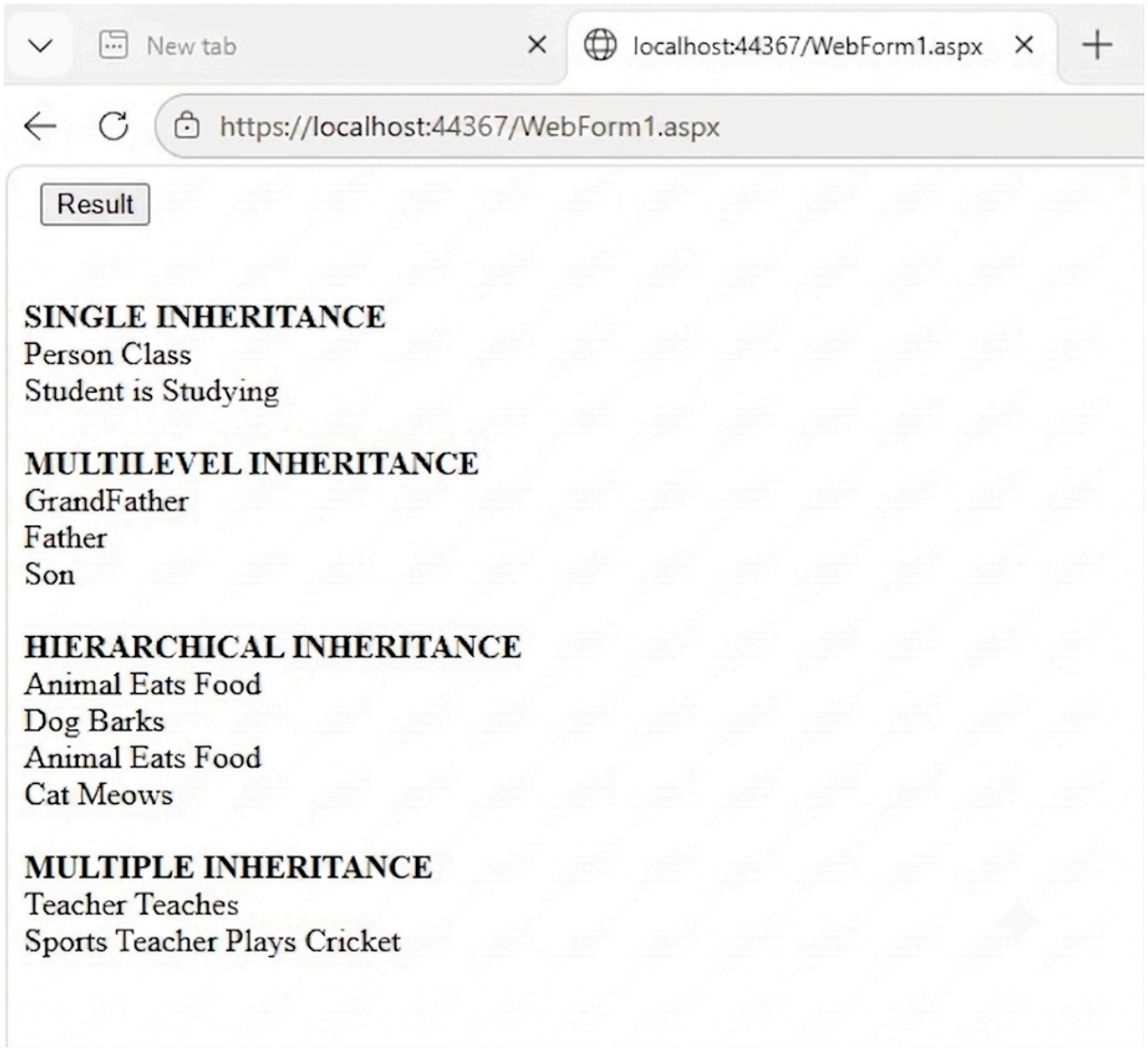
        // Multiple Inheritance

        SportsTeacher st = new SportsTeacher();

        Label1.Text += "<b>MULTIPLE INHERITANCE</b><br/>";
        Label1.Text += st.Teach() + "<br/>";
        Label1.Text += st.Play();
    }
  }
}
```

Step 5: Run the Application

1. Save all files.
2. Click Build →Build Solution.
3. Press **F5**.
4. Browser window will open.
5. Enter three numbers in TextBox .
6. Click **Result**.



c. Constructor overloading

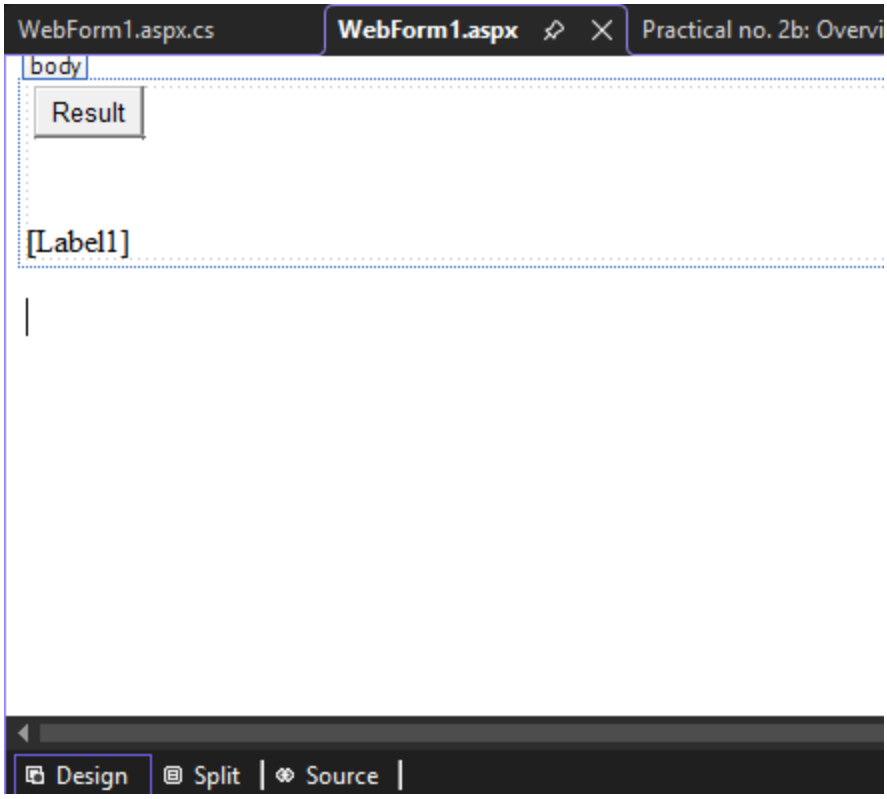
Procedure:

Step 1 and Step 2 are the same as the previous practical.

Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. Button
2. Label



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical_no._2b
{
   public partial class WebForm1 : System.Web.UI.Page
   {
      class Student
      {
         public string msg;

         public Student()
         {
            msg = "Default Constructor";
```

```
        }

        public Student(string name)
        {
            msg = "Name = " + name;
        }

        public Student(string name, int age)
        {
            msg = "Name = " + name +
                  ", Age = " + age;
        }
    }

    protected void Page_Load(object sender, EventArgs e)
    {

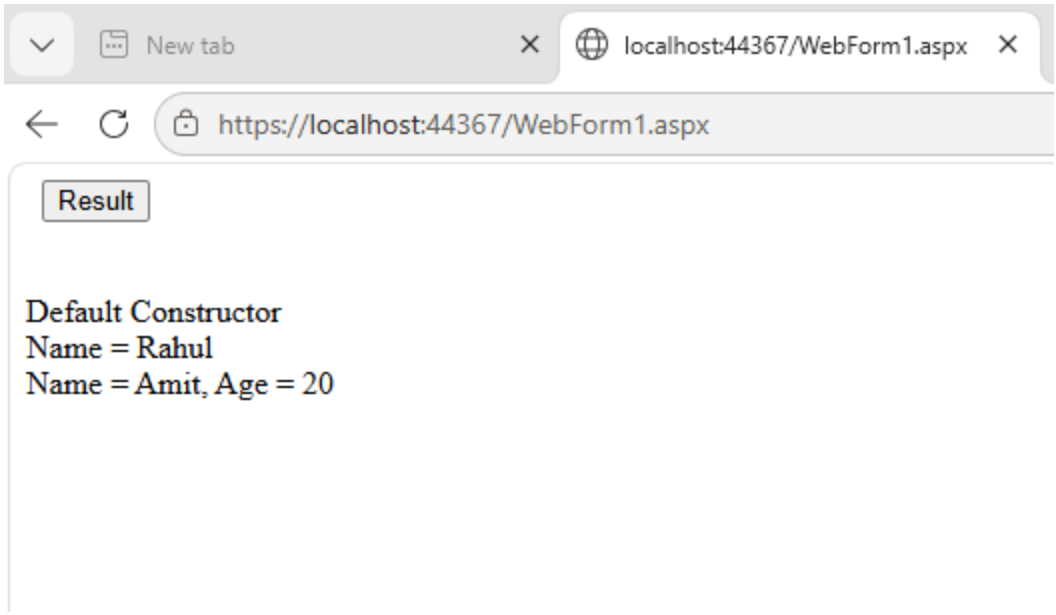
    }

    protected void Button1_Click(object sender, EventArgs e)
    {
        Student s1 = new Student();
        Student s2 = new Student("Rahul");
        Student s3 = new Student("Amit", 20);

        Label1.Text =
            s1.msg + "<br/>" +
            s2.msg + "<br/>" +
            s3.msg;
    }
  }
}
```

Step 5: Run the Application

1. Save all files.
2. Click Build →Build Solution.
3. Press **F5**.
4. Browser window will open.
5. Enter three numbers in TextBox .
6. Click **Result**.



d. Interfaces

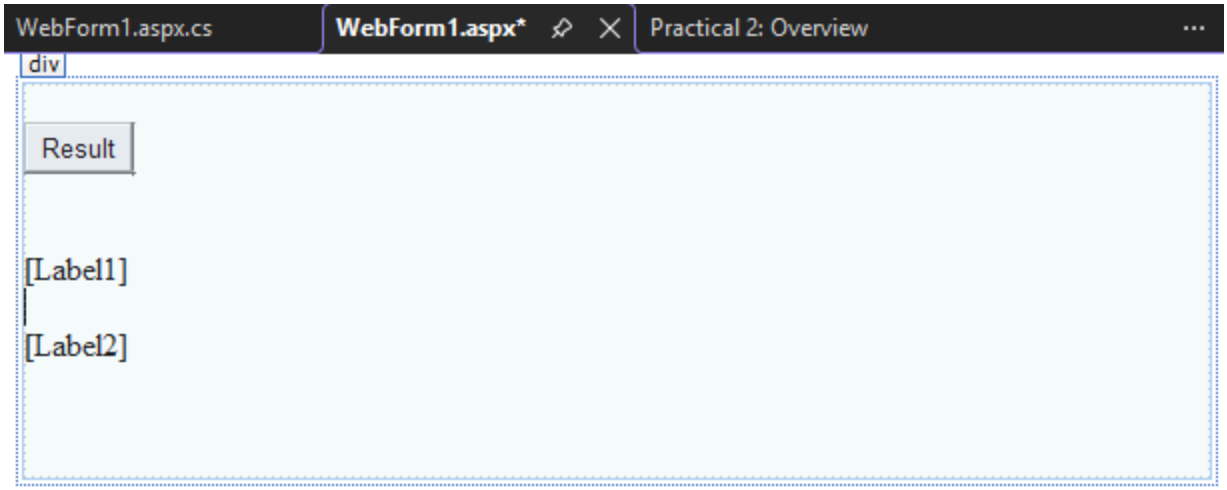
Procedure:

Step 1 and Step 2 are the same as the previous practical.

Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. Button
2. Label





Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical_2
{
   public partial class WebForm1 : System.Web.UI.Page
  {
  public class shape
  {
      public int x, y, z;
      public void getSide(int length, int breadth, int width)
```

```
        {
            x = length;
            y = breadth;
            z = width;
        }
    }
    public interface IPolygon
    {
        int CalculateArea();
    }
    public interface IVolume
    {
        int getVolume();
    }
    class Rectangle : shape, IPolygon, IVolume
    {
        public int CalculateArea()
        {
            return (x * y);
        }
        public int getVolume()
        {
            return (x * y * z);
        }
    }

        protected void Page_Load(object sender, EventArgs e)
        {

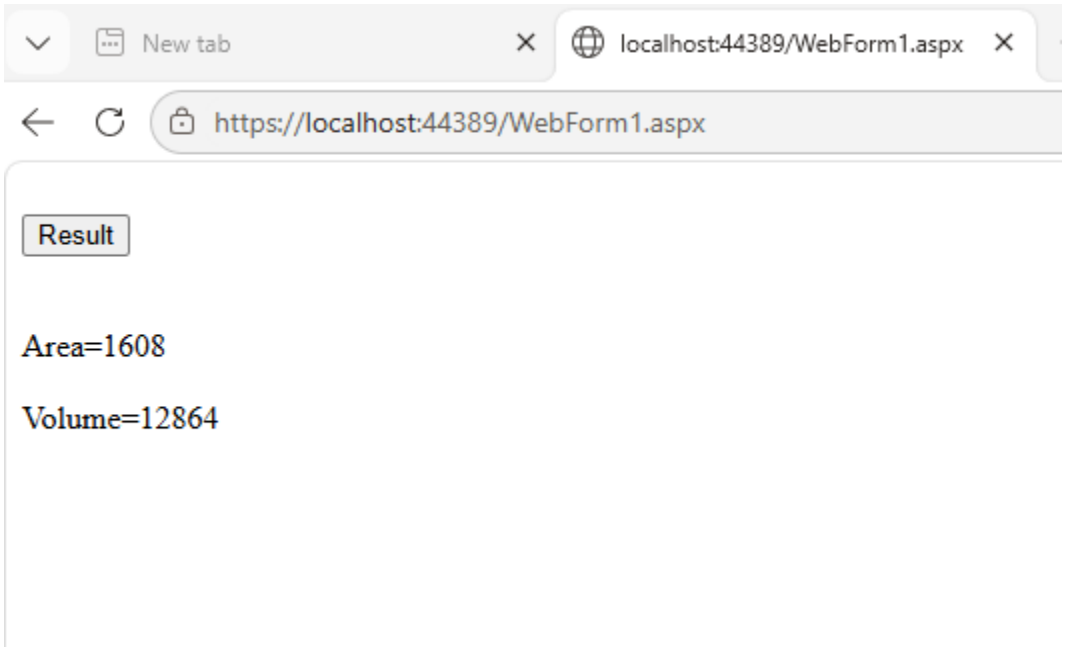
        }

        protected void Button1_Click(object sender, EventArgs e)
        {
            Rectangle r = new Rectangle();
            r.getSide(24, 67, 8);
            Label1.Text = "Area=" + r.CalculateArea() +"<br/>" +"Volume="+ r.getVolume();

        }
    }
}
```

Step 5: Run the Application

1. Save all files.
2. Click Build →Build Solution.
3. Press **F5**.
4. Browser window will open.
5. Enter three numbers in TextBox .
6. Click **Result**.



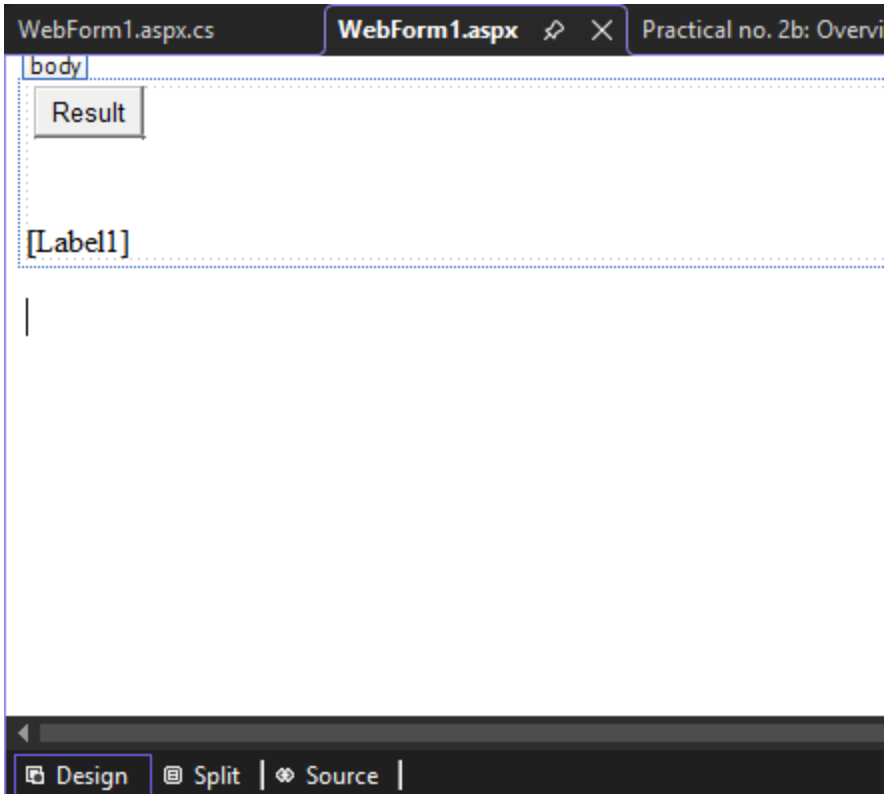
e. Using Delegates and events
Procedure:

Step 1 and Step 2 are the same as the previous practical.

Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. Button
2. Label



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical_2
{
   public partial class WebForm1 : System.Web.UI.Page
   {
      // Delegate Declaration
      public delegate void ResultHandler();

      // Event Declaration
      public event ResultHandler ResultDeclared;
```

```
    protected void Page_Load(object sender, EventArgs e)
    {

    }

    protected void Button1_Click(object sender, EventArgs e)
    {
        // Subscribe Event
        ResultDeclared += ShowMessage;

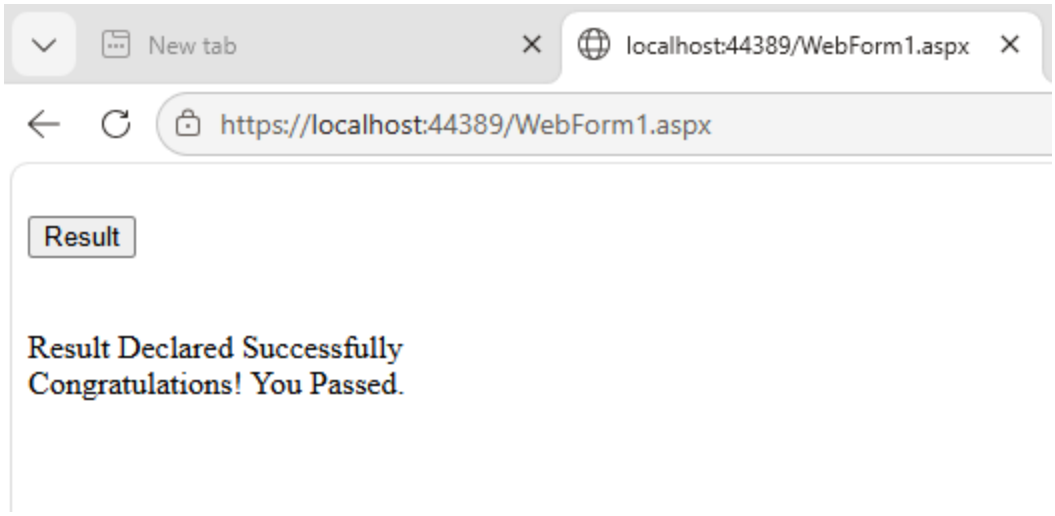
        Label1.Text = "Result Declared Successfully<br/>";

        // Raise Event
        ResultDeclared();
    }

    public void ShowMessage()
    {
        Label1.Text += "Congratulations! You Passed.";
    }
  }
}
```

Step 5: Run the Application

1. Save all files.
2. Click Build →Build Solution.
3. Press **F5**.
4. Browser window will open.
5. Enter three numbers in TextBox .
6. Click **Result.**



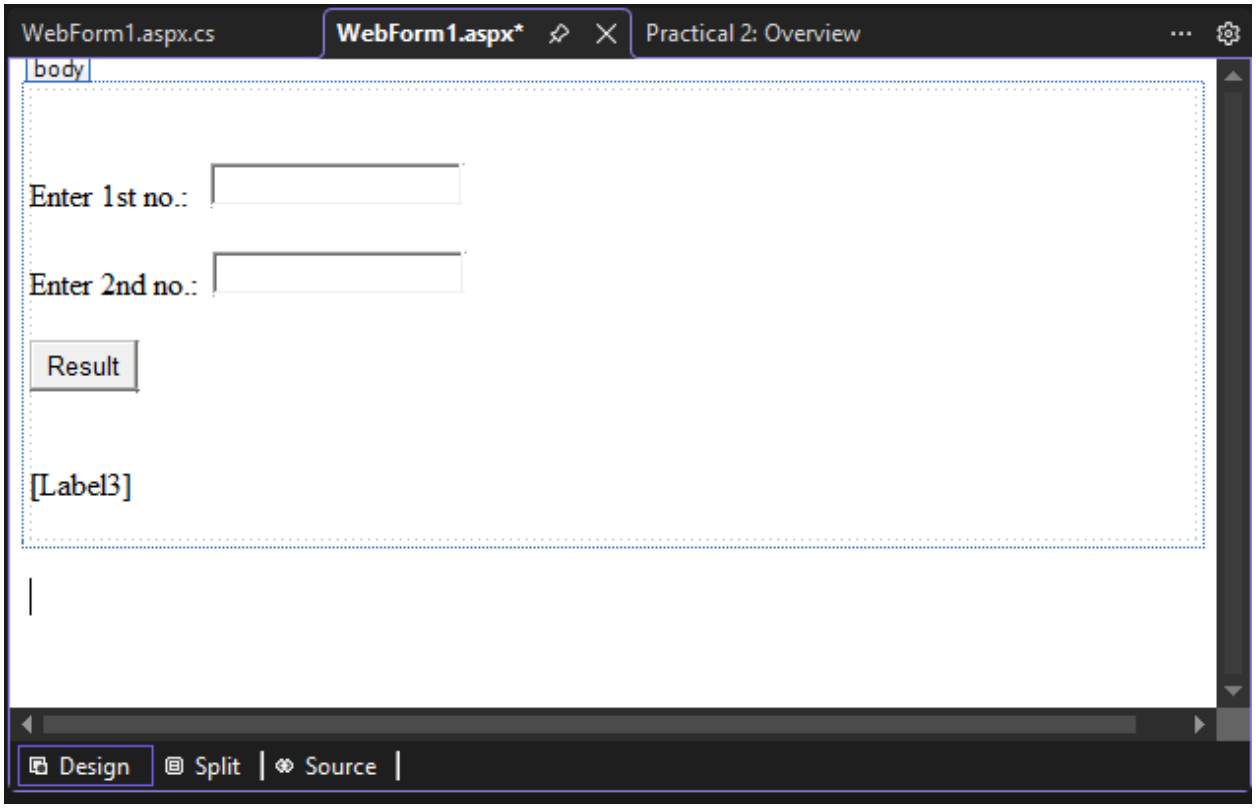
f. Exception Handling
Procedure:

Step 1 and Step 2 are the same as the previous practical.

Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. TextBox
2. Button
3. Label



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical_2
{
   public partial class WebForm1 : System.Web.UI.Page
   {

      protected void Page_Load(object sender, EventArgs e)
      {

      }
```

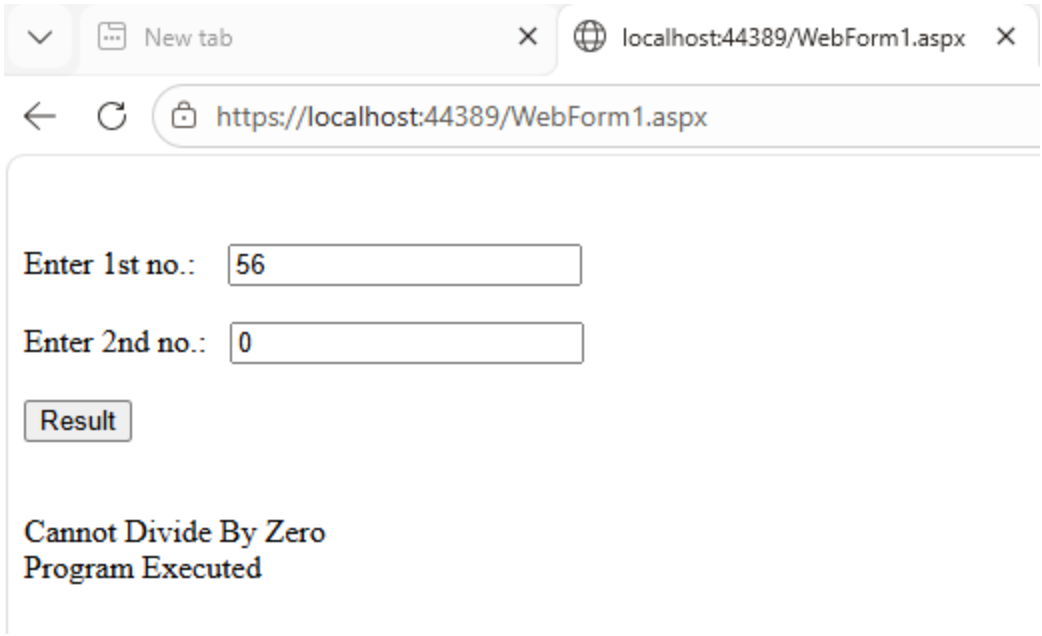
```
    protected void Button1_Click(object sender, EventArgs e)
    {
        try
        {
            int a = Convert.ToInt32(TextBox1.Text);
            int b = Convert.ToInt32(TextBox2.Text);

            int result = a / b;

            Label3.Text = "Result = " + result;
        }
        catch (DivideByZeroException)
        {
            Label3.Text =
            "Cannot Divide By Zero";
        }
        catch (FormatException)
        {
            Label3.Text =
            "Invalid Input";
        }
        finally
        {
            Label3.Text +=
            "<br/>Program Executed";
        }
    }
  }
}
```

Step 5: Run the Application

1. Save all files.
2. Click Build →Build Solution.
3. Press **F5**.
4. Browser window will open.
5. Enter three numbers in TextBox .
6. Click **Result**


New tab
localhost:44389/WebForm1.aspx
https://localhost:44389/WebForm1.aspx
Enter 1st no.:
56
Enter 2nd no.:
0
Result
Cannot Divide By Zero
Program Executed

Practical no. 3A

Aim: Write the program for the following:
a. Create methods add(). multiply(), subtract() ,divide() with suitable parameters and call these methods using concept of C# delegate.

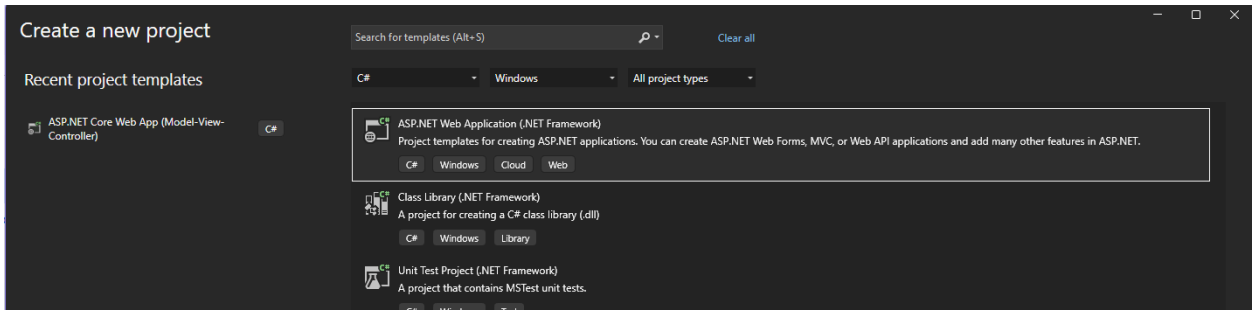
Software Required:

Visual Studio 2026

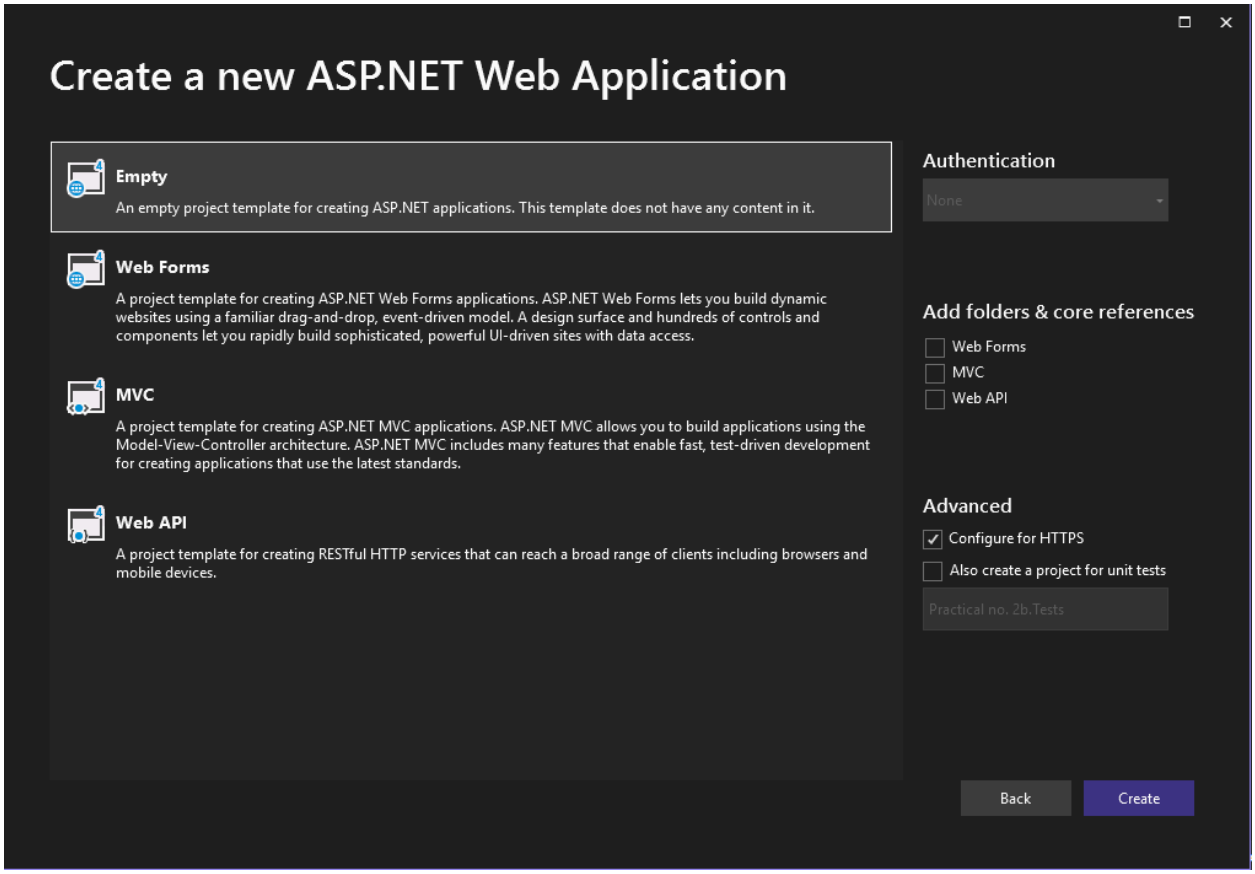
Procedure:

Step 1: Create New Project

1. Open Visual Studio.
2. Click **Create a new project**.
3. Select **ASP.NET Web Application (.NET Framework)**.



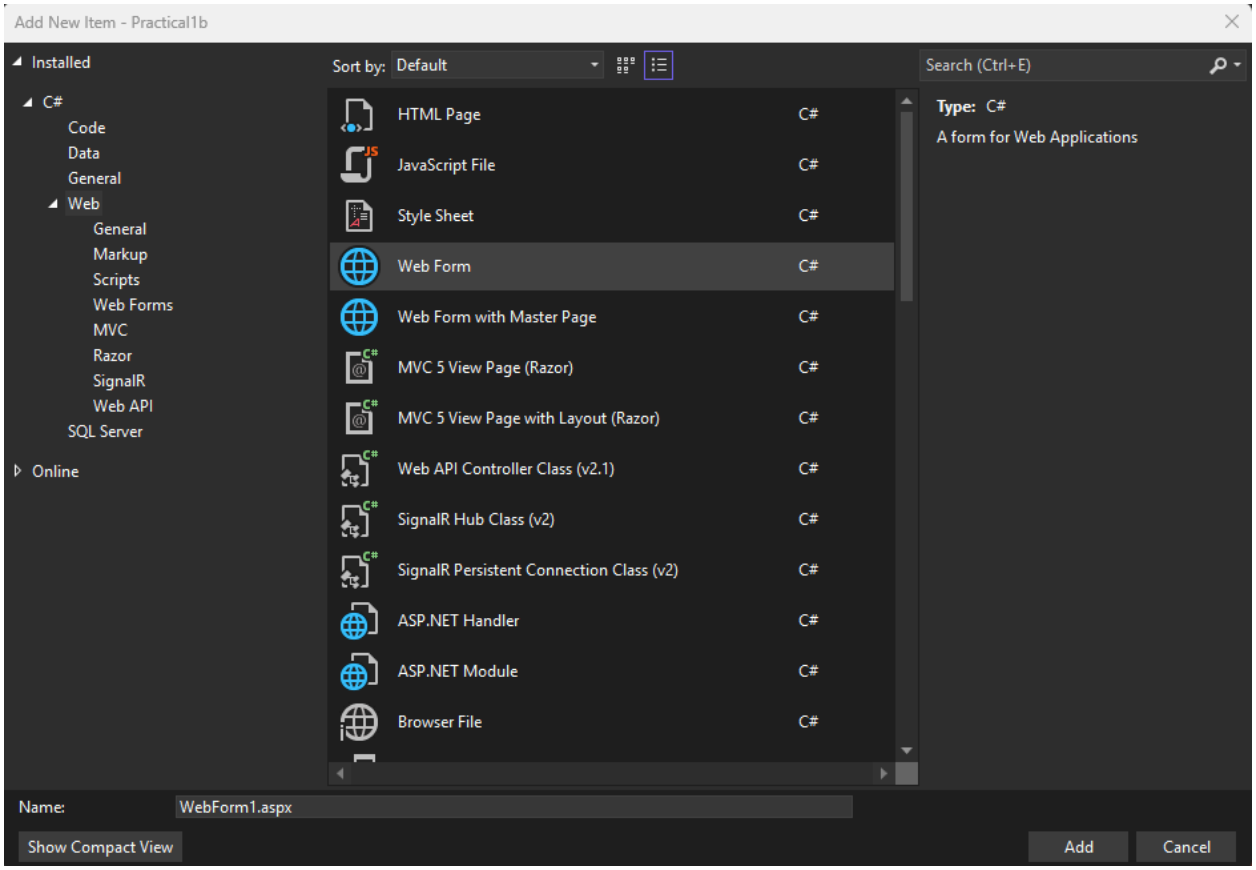
4. Click **Next**.
5. Enter Project Name: **Practical3**
6. Click **Create**.
7. Select **Empty**.



8. Click **Create**.

Step 2: Add a Web Form

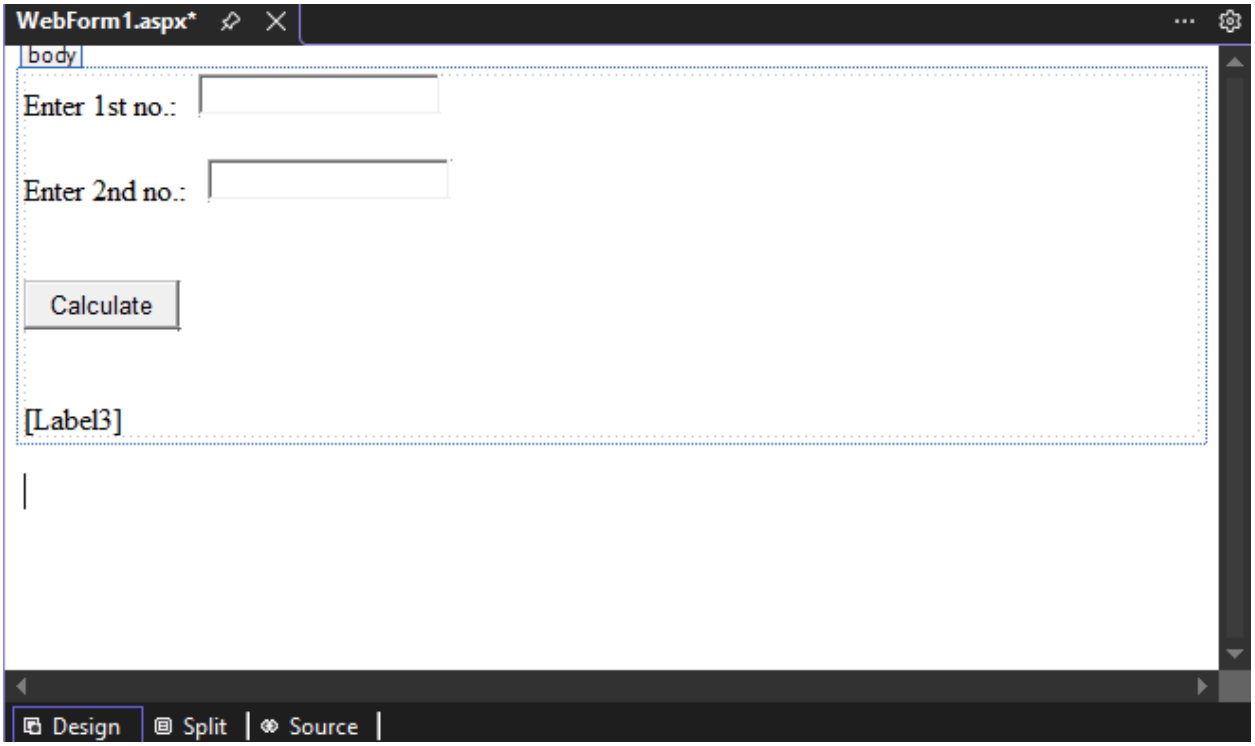
1. In Solution Explorer, right-click the project **Practical3a**
2. Select Add → New Item.
3. Select Web Form.
4. Click **Add**.



Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. TextBox
2. Button
3. Label



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Reflection.Emit;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Practical_3
{
    public partial class WebForm1 : System.Web.UI.Page
  {
    public delegate int MathOperation(int a, int b);

    int Add(int a, int b)
    {
       return a + b;
```

```
    }
    int Subtract(int a, int b)
    {
        return a - b;
    }
    int Multiply(int a, int b)
    {
        return a * b;
    }
    int Divide(int a, int b)
    {
        return a / b;
    }

    protected void Page_Load(object sender, EventArgs e)
    {

    }

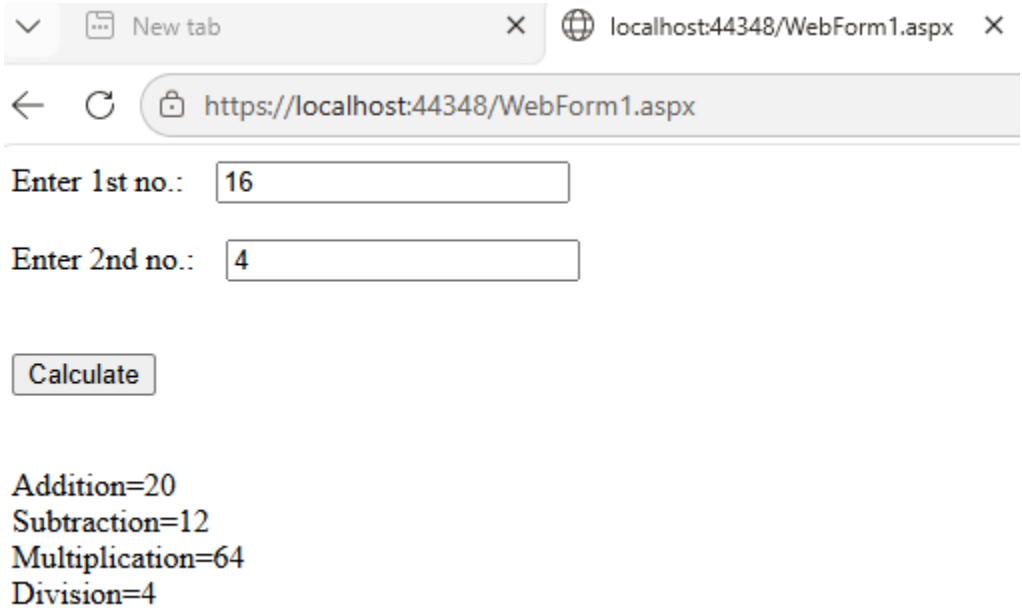
    protected void Button1_Click(object sender, EventArgs e)
    {
        int x = Convert.ToInt32(TextBox1.Text);
        int y = Convert.ToInt32(TextBox2.Text);
        MathOperation add = new MathOperation(Add);
        MathOperation subtract = new MathOperation(Subtract);
        MathOperation multiply = new MathOperation(Multiply);
        MathOperation divide = new MathOperation(Divide);
        Label3.Text = "Addition=" + add(x, y).ToString() + "<br/>";
        Label3.Text += "Subtraction=" + subtract(x, y).ToString() + "<br/>";
        Label3.Text += "Multiplication=" + multiply(x, y).ToString() + "<br/>";
        Label3.Text += "Division=" + divide(x, y).ToString() + "<br/>";

    }
  }
}
```

Step 5: Run the Application

1. Save all files.
2. Click Build → Build Solution.
3. Press **F5**.

4. Browser window will open.
5. Enter three numbers in TextBox .
6. Click **Calculate**.



Practical no. 3B

Aim: Write the program for the following:
b. Write a program using multicast delegate.

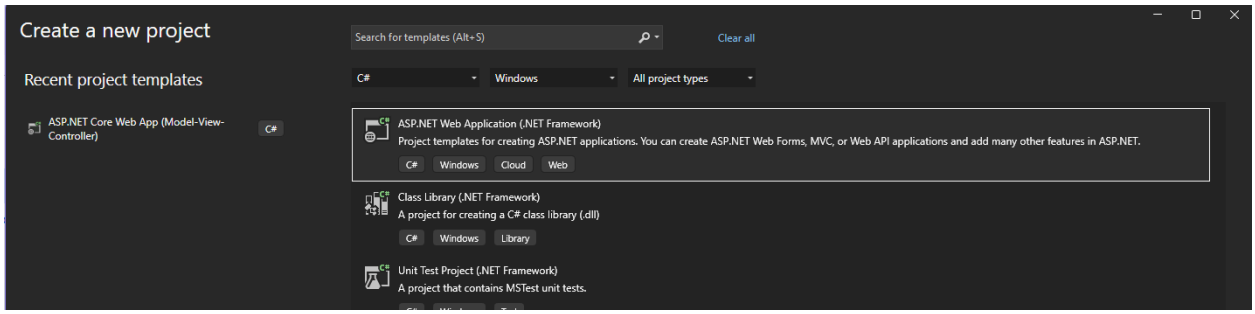
Software Required:

Visual Studio 2026

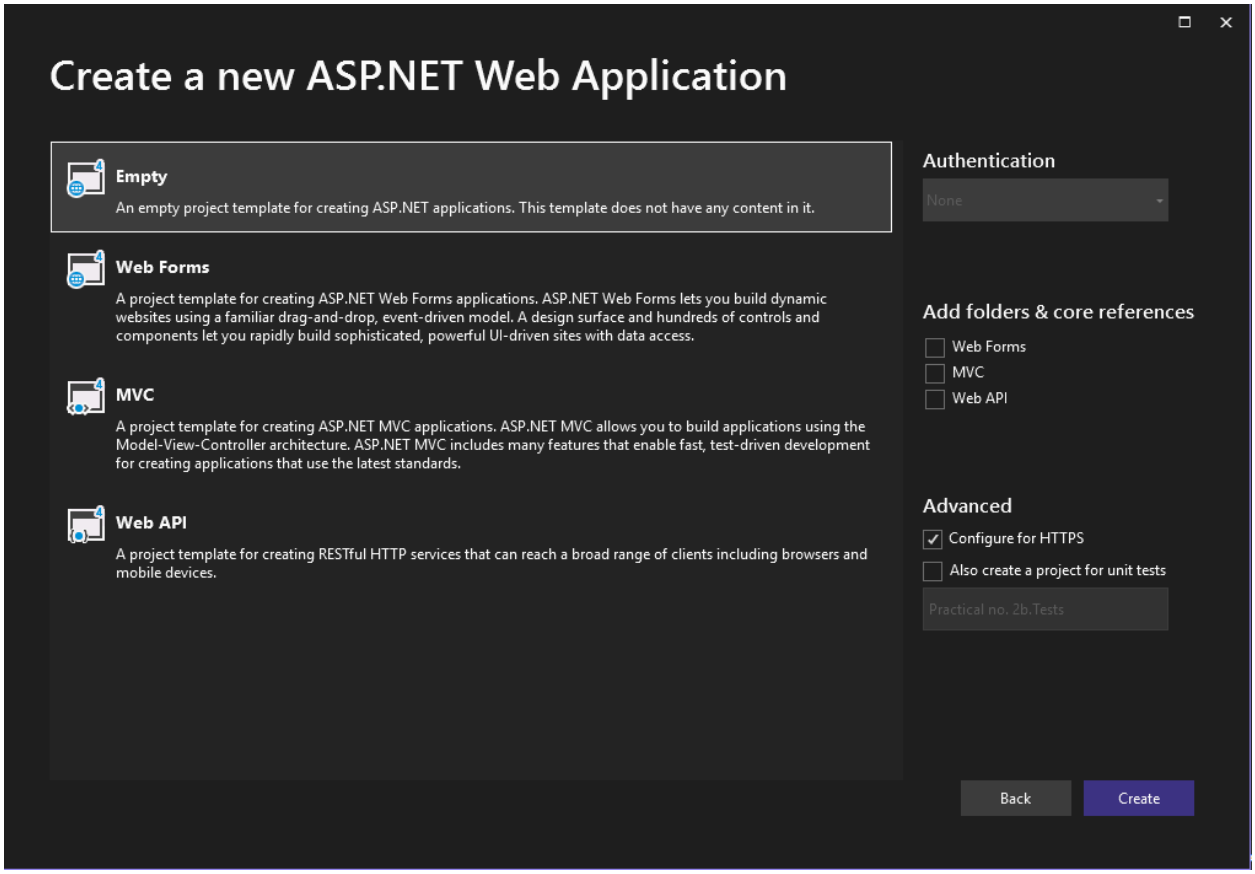
Procedure:

Step 1: Create New Project

1. Open Visual Studio.
2. Click **Create a new project**.
3. Select **ASP.NET Web Application (.NET Framework)**.



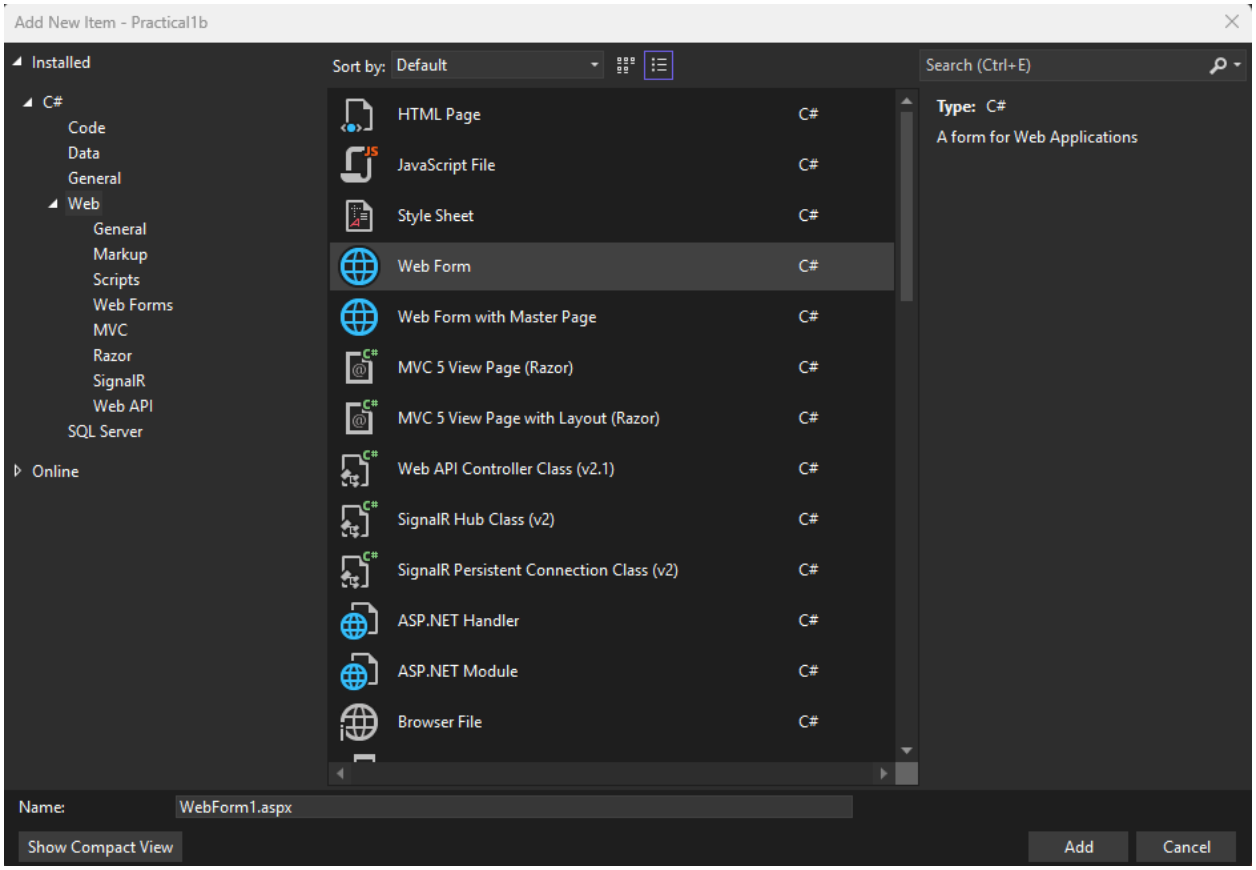
4. Click **Next**.
5. Enter Project Name: **Practical3**
6. Click **Create**.
7. Select **Empty**.



8. Click **Create**.

Step 2: Add a Web Form

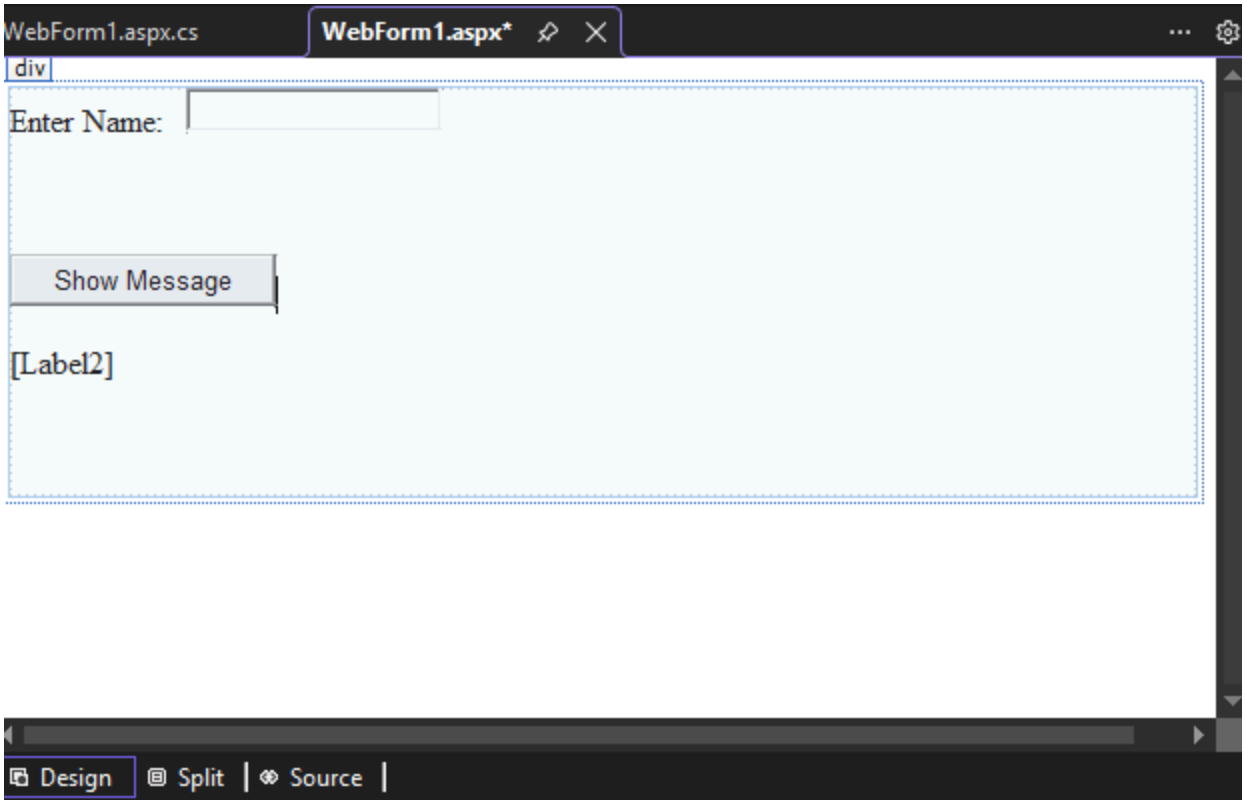
1. In Solution Explorer, right-click the project **Practical3b**
2. Select Add → New Item.
3. Select Web Form.
4. Click **Add**.



Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. TextBox
2. Button
3. Label



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Reflection.Emit;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Xml.Linq;

namespace Practical_3
{

    public partial class WebForm1 : System.Web.UI.Page
    {
        public delegate void GreetingDelegate(string name);
        string result = "";
```

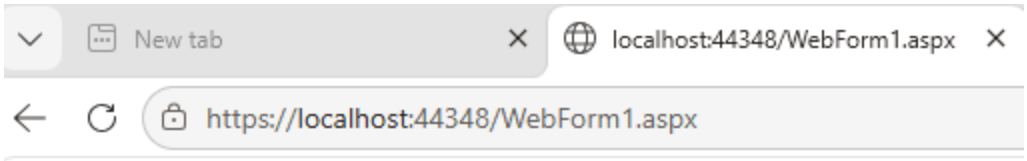
```
    protected void Page_Load(object sender, EventArgs e)
    {

    }
    public void Welcome(string name)
    {
       result += "Welcome " + name + "<br/>";
    }
    public void GoodMorning(string name)
    {
       result += "Good Morning " + name + "<br/>";
    }
    public void ThankYou(string name)
    {
       result += "Thank You " + name + "<br/>";
    }
    protected void Button1_Click(object sender, EventArgs e)
    {
       result = "";
       string name = TextBox1.Text;
       GreetingDelegate gd;
       gd = Welcome;
       gd += GoodMorning;
       gd += ThankYou; gd(name);
      Label2.Text = result;

    }
  }
}
```

Step 5: Run the Application

1. Save all files.
2. Click Build → Build Solution.
3. Press **F5**.
4. Browser window will open.
5. Enter three numbers in TextBox .
6. Click **Show Message**.



Enter Name: Ram

Show Message

Welcome Ram
Good Morning Ram
Thank You Ram

Practical no. 4A

Aim: Create an ASP.NET Web Forms application using simple server controls.
a. Design a Student Registration Form using the following ASP.NET controls:
i. Label and TextBox to accept Student Name
ii. RadioButton to select Gender (Male/Female)
iii. DropDownList to select Course (BSc IT, BSc CS, BCA)
iv. CheckBox for accepting Terms & Conditions
v. Calendar control to select Date
vi. Button control to submit the form
vii. Label control to display the output

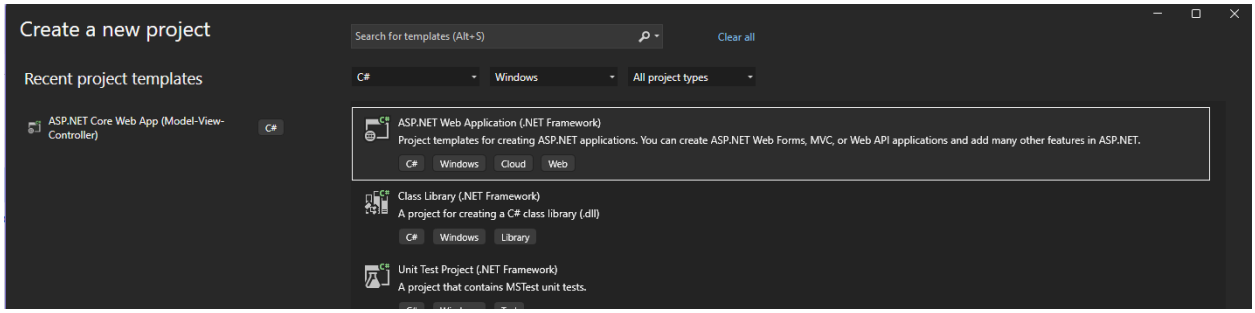
Software Required:

Visual Studio 2022

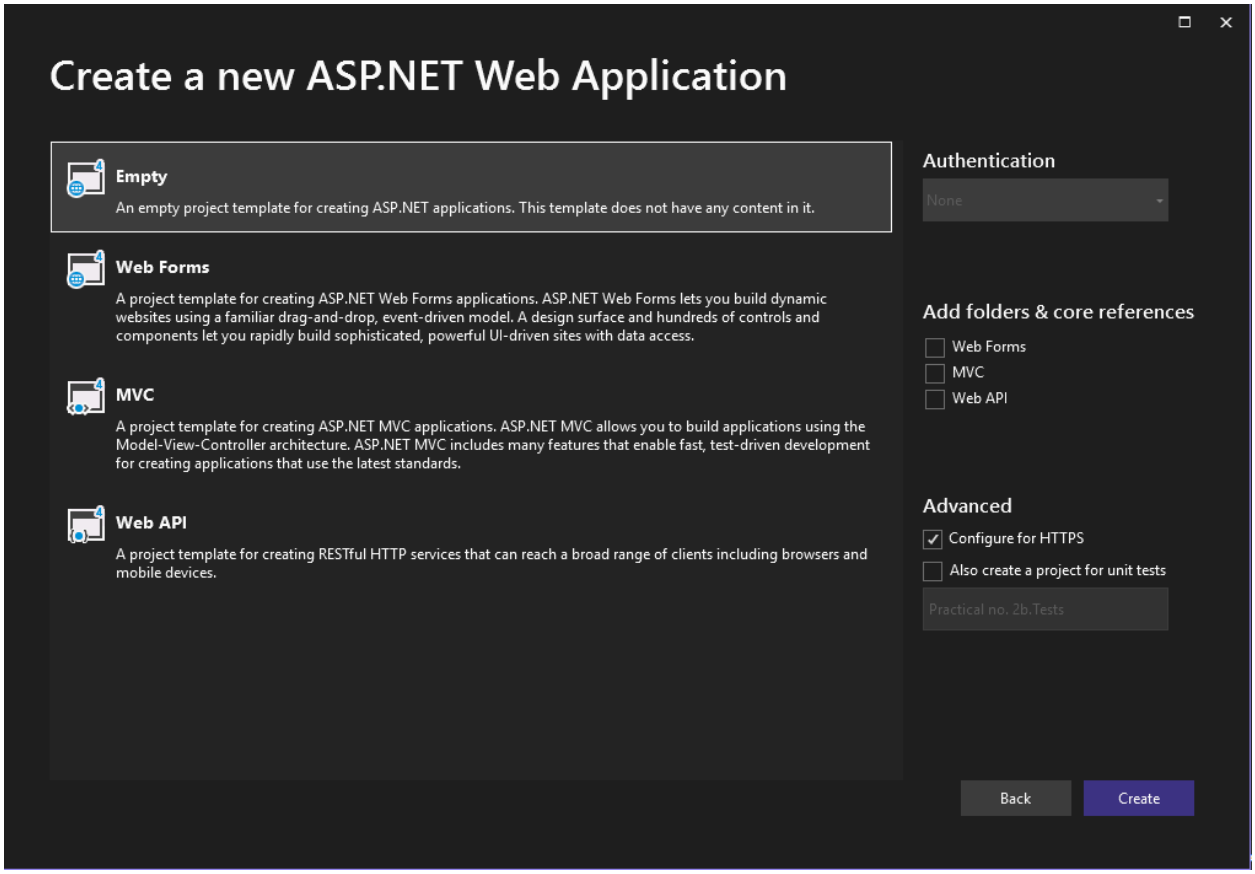
Procedure:

Step 1: Create New Project

1. Open Visual Studio.
2. Click **Create a new project**.
3. Select **ASP.NET Web Application (.NET Framework)**.



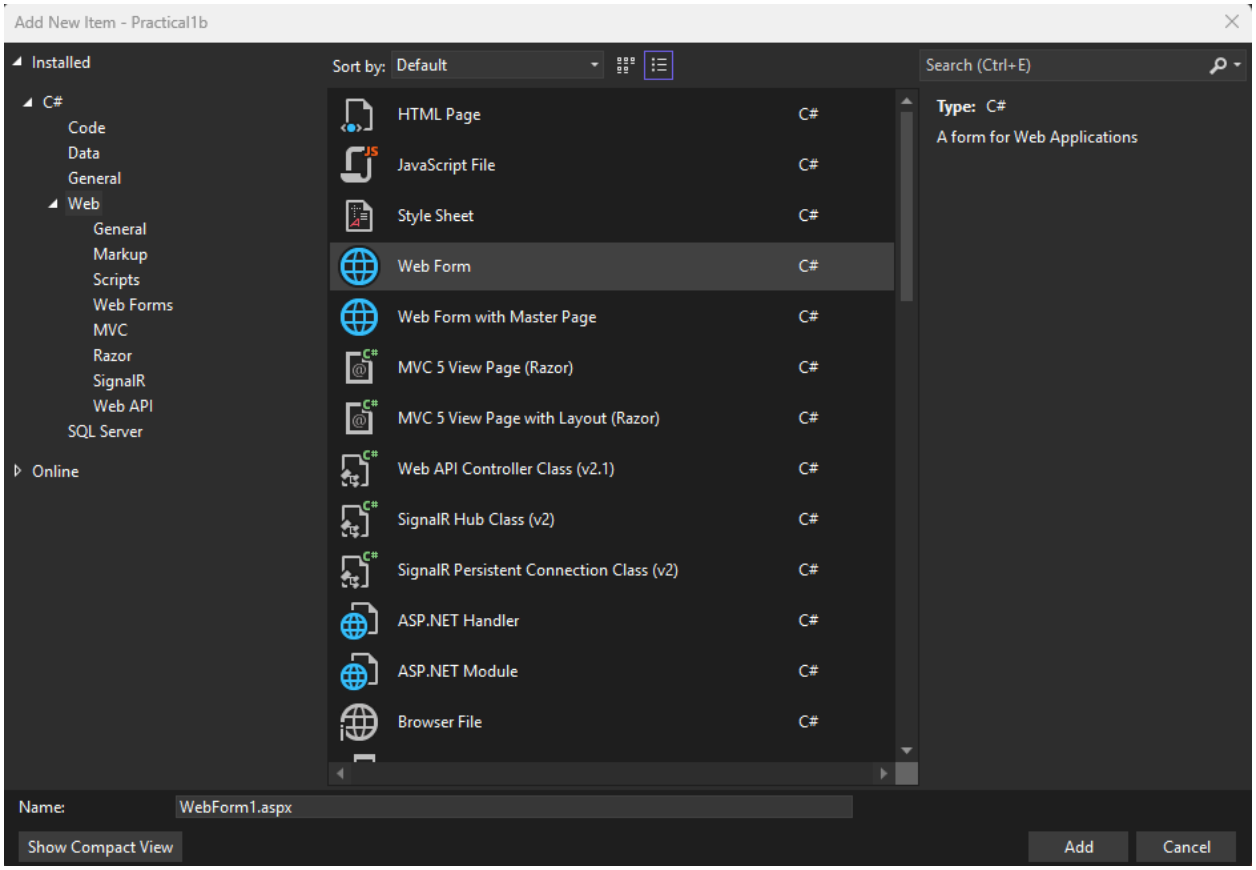
4. Click **Next**.
5. Enter Project Name: **Practical3**
6. Click **Create**.
7. Select **Empty**.



8. Click **Create**.

Step 2: Add a Web Form

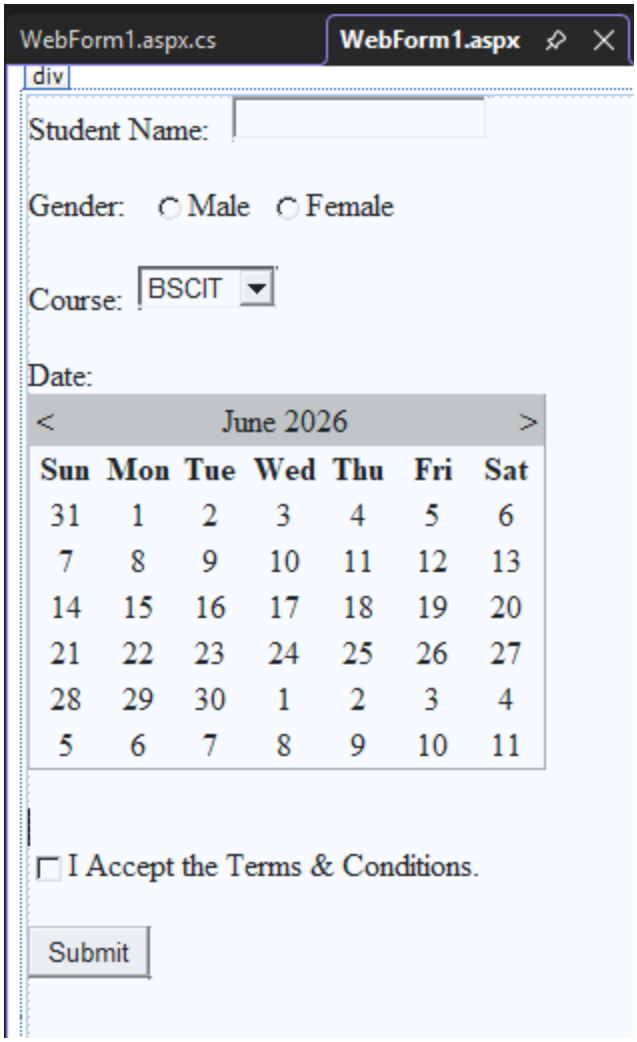
1. In Solution Explorer, right-click the project **Practical4a**
2. Select Add → New Item.
3. Select Web Form.
4. Click **Add**.



Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. TextBox
2. Button
3. Label
4. RadioButton
5. DropDownList
6. CheckBox
7. Calendar control



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Xml.Linq;

namespace Practical_4
{
```

```
public partial class WebForm1 : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {

    }

    protected void Button1_Click(object sender, EventArgs e)
    {
        string gender = "";

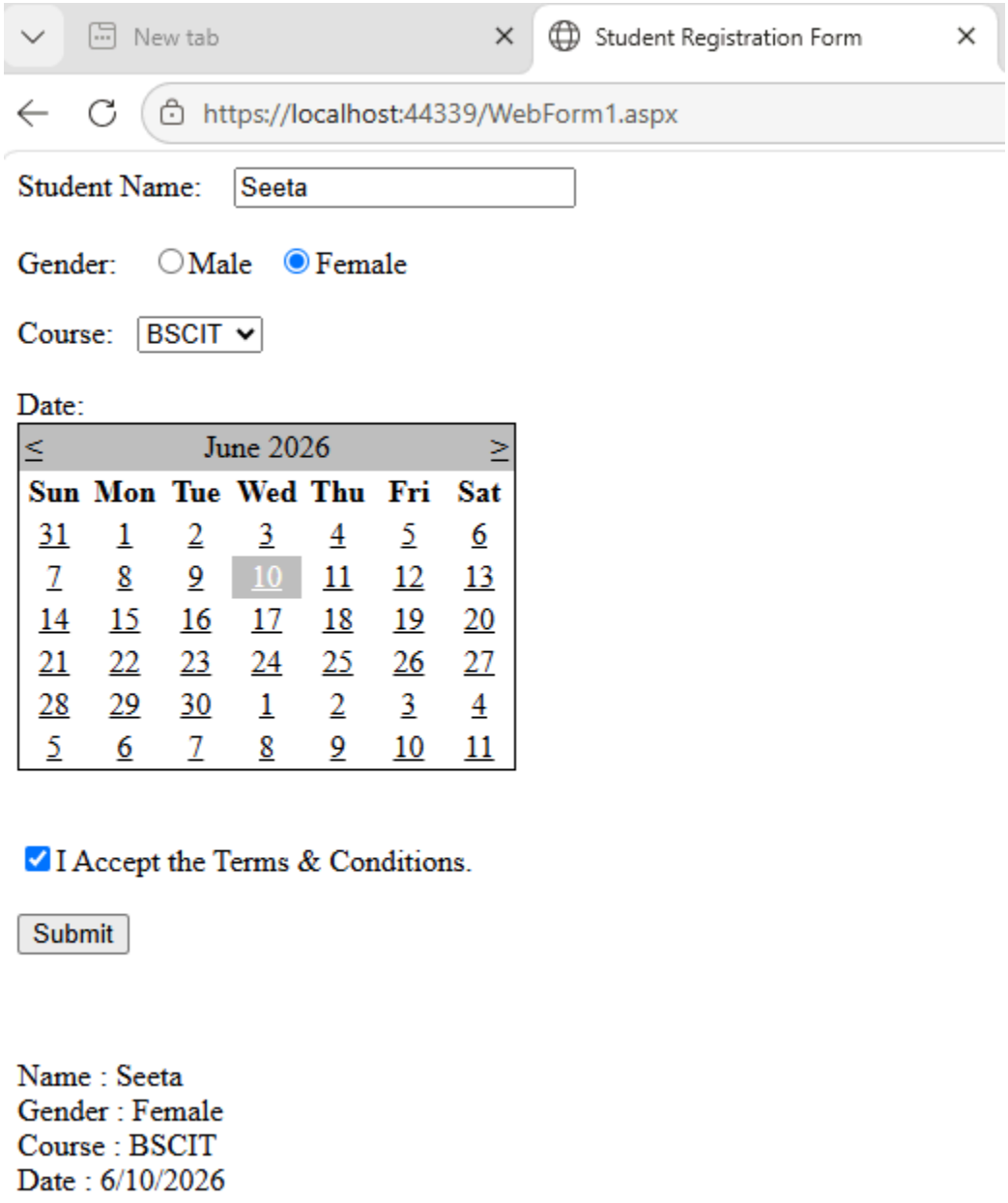
        if (RadioButton1.Checked)
            gender = "Male";
        else if (RadioButton2.Checked)
            gender = "Female";

        if (CheckBox1.Checked)
        {
            Label5.Text =
                "<br/>Name : " + TextBox1.Text +
                "<br/>Gender : " + gender +
                "<br/>Course : " + DropDownList1.SelectedItem.Text +
                "<br/>Date : " +
                Calendar1.SelectedDate.ToShortDateString();
        }
        else
        {
            Label5.Text =
                "Please accept Terms & Conditions.";
        }
    }
}
}
```

Step 5: Run the Application

1. Save all files.
2. Press **Ctrl + F5**.
3. Enter Student Name.
4. Select Gender.
5. Select Course.

6. Accept Terms & Conditions.
7. Select Date from Calendar.
8. Click **Submit**.



Practical no. 4B

Aim: Create an ASP.NET Web Forms application using simple server controls.
b. Create a Registration form to demonstrate use of various Validation controls.

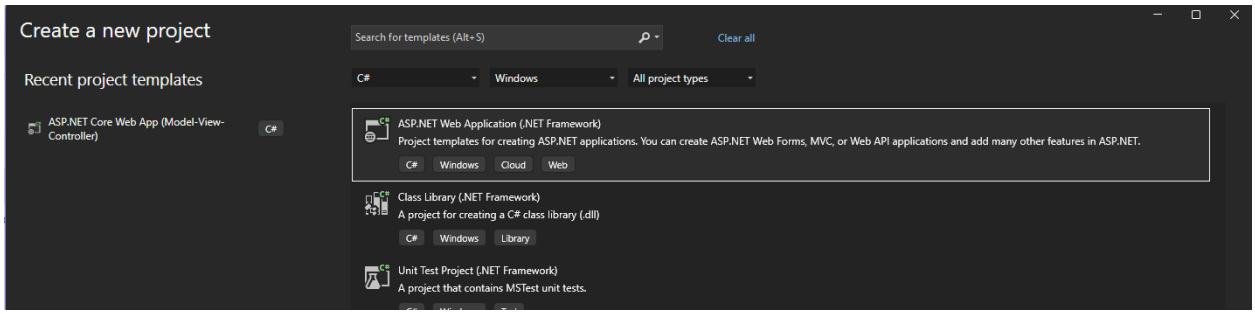
Software Required:

Visual Studio 2022

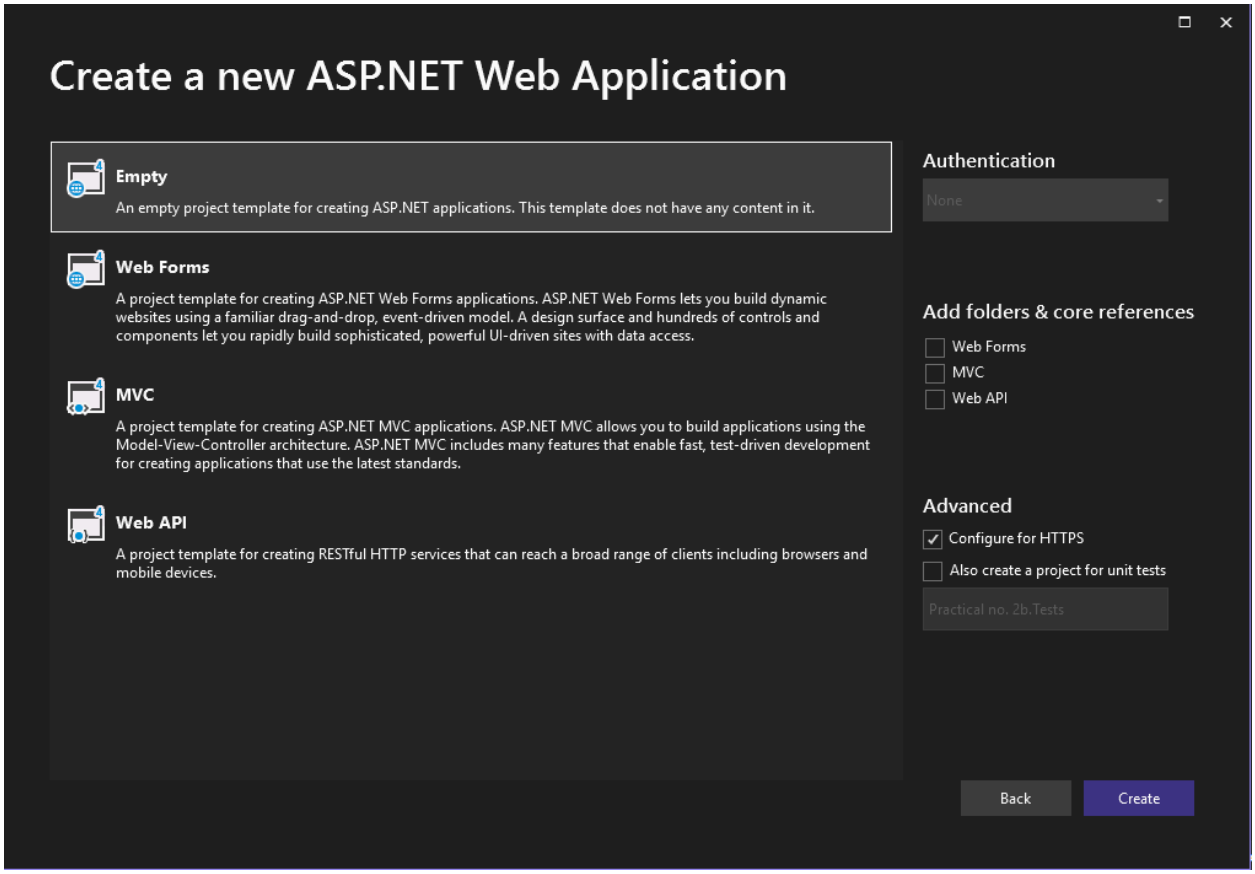
Procedure:

Step 1: Create New Project

1. Open Visual Studio.
2. Click **Create a new project**.
3. Select **ASP.NET Web Application (.NET Framework)**.



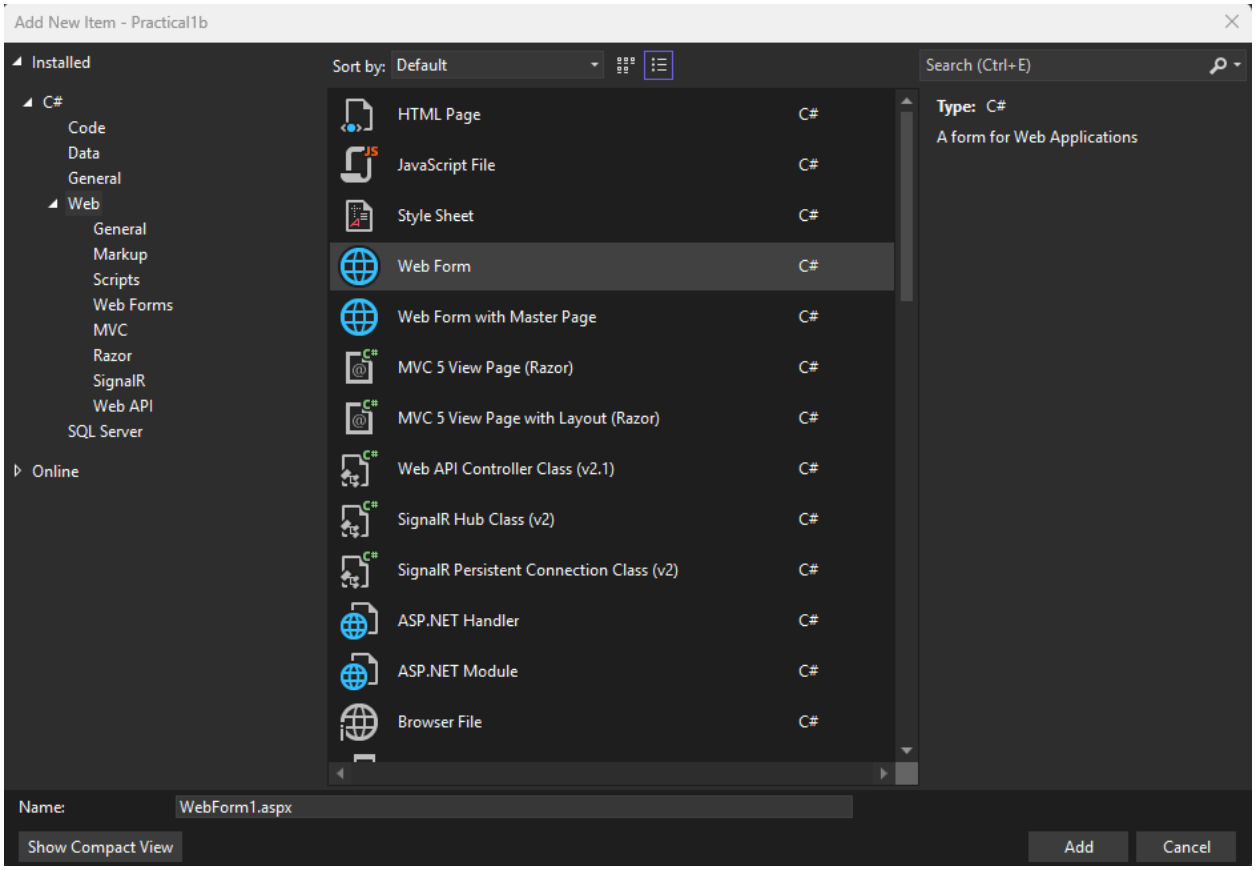
4. Click **Next**.
5. Enter Project Name: **Practical3**
6. Click **Create**.
7. Select **Empty**.



8. Click **Create**.

Step 2: Add a Web Form

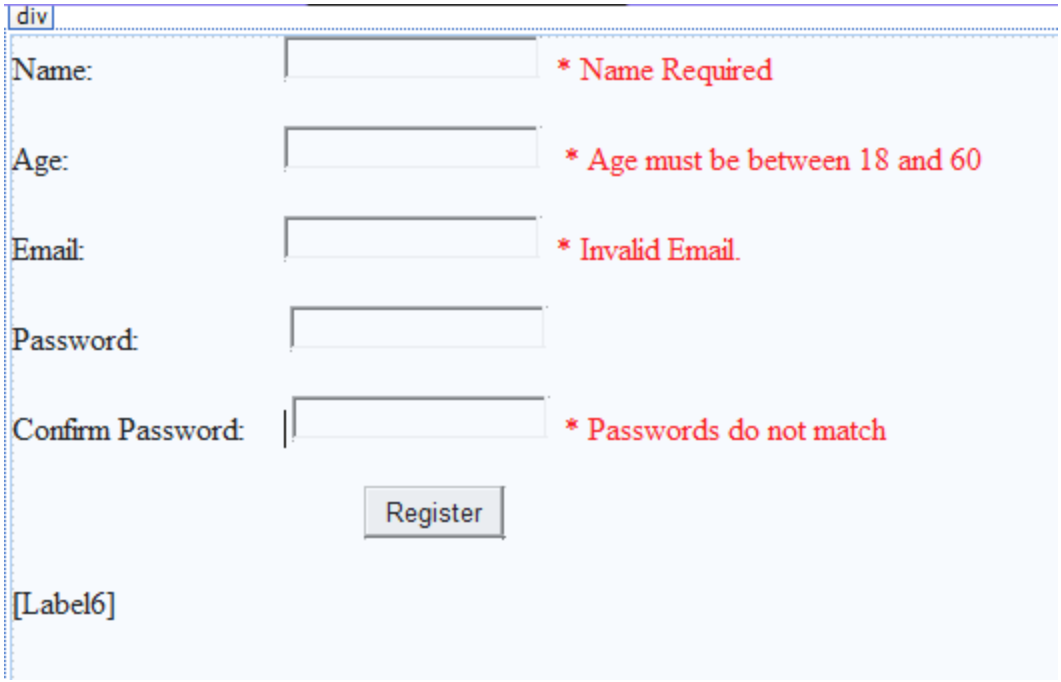
1. In Solution Explorer, right-click the project **Practical4B.**
2. Select Add → New Item.
3. Select Web Form.
4. Click **Add**.



Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. TextBox
2. Button
3. Label
4. RequiredFieldValidator
5. RangeValidator
6. RegularExpressionValidator
7. CompareValidator



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

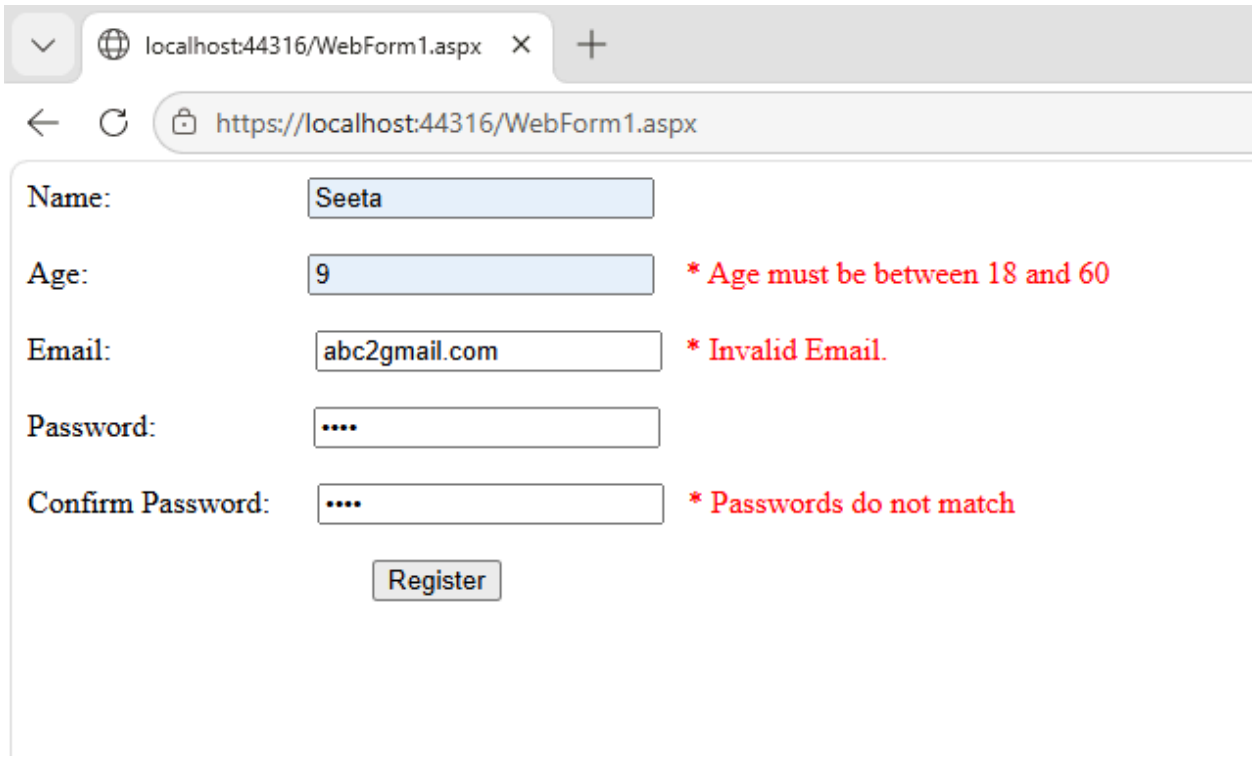
namespace Ptactical_4b
{
   public partial class WebForm1 : System.Web.UI.Page
   {
      protected void Page_Load(object sender, EventArgs e)
      {
         ValidationSettings.UnobtrusiveValidationMode =
           UnobtrusiveValidationMode.None;
      }

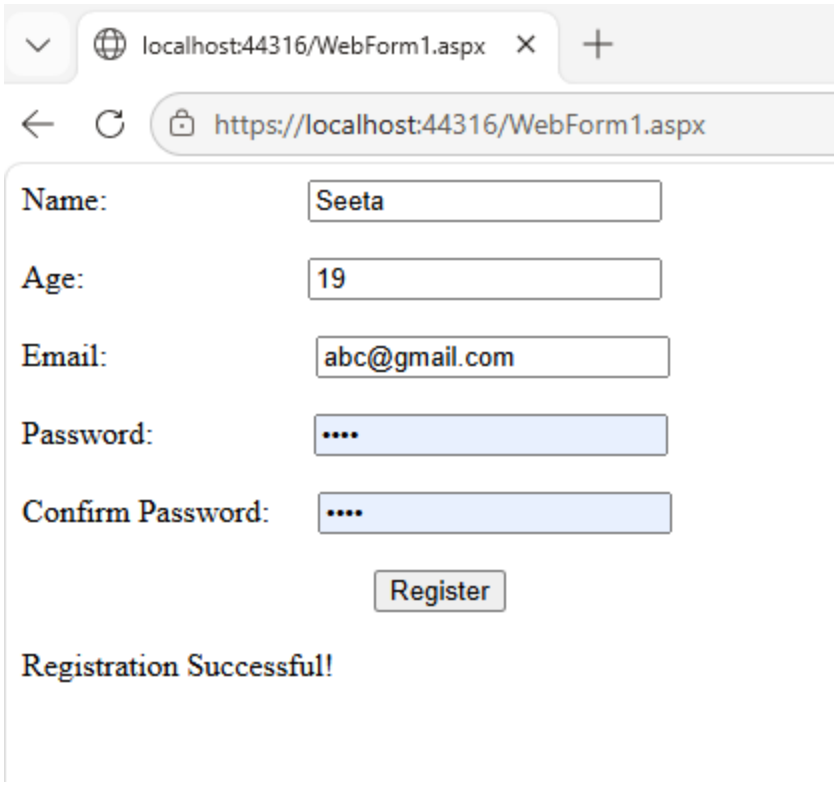
      protected void Button1_Click(object sender, EventArgs e)
      {
```

```
        if (Page.IsValid)
        {
          Label6.Text =
           "Registration Successful!";
        }
      }
    }
}
```

Step 5: Run the Application

1. Save all files.
2. Press **Ctrl + F5**.
3. Leave fields blank or enter invalid values.
4. Click **Register**.
5. Validation messages will appear.
6. Enter correct values and click **Register**.
7. Registration Successful message will be displayed.




localhost:44316/WebForm1.aspx
https://localhost:44316/WebForm1.aspx
Name:
Seeta
Age:
19
Email:
abc@gmail.com
Password:
Confirm Password:
Register
Registration Successful!

Practical no. 4C

Aim: Create an ASP.NET Web Forms application using simple server controls.
c. Create Web Form to demonstrate use of Adrotator Control.

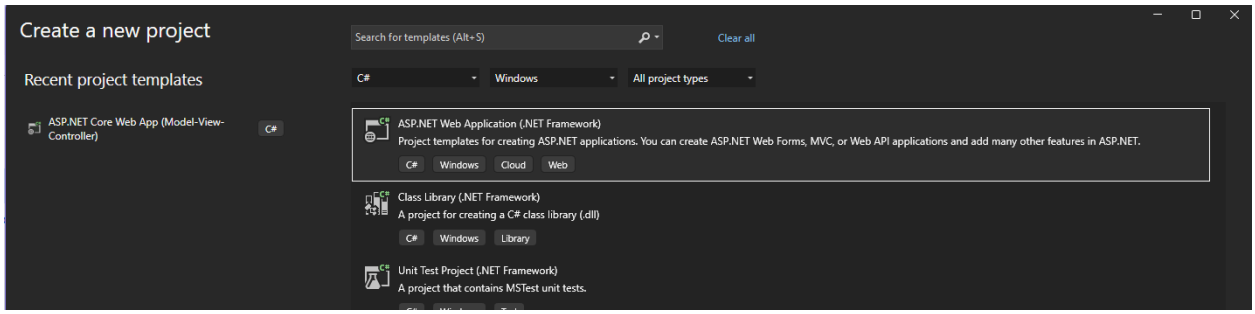
Software Required:

Visual Studio 2022

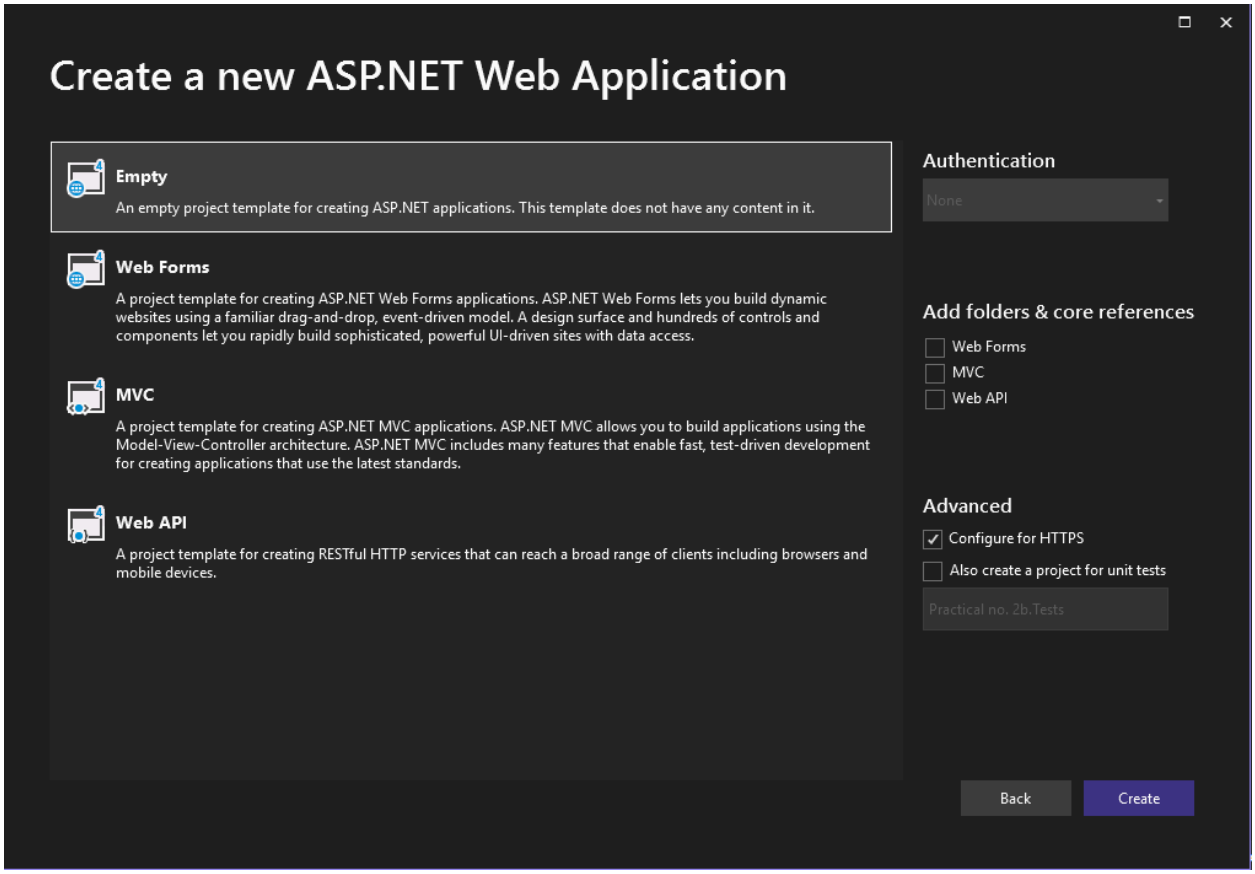
Procedure:

Step 1: Create New Project

1. Open Visual Studio.
2. Click **Create a new project**.
3. Select **ASP.NET Web Application (.NET Framework)**.



4. Click **Next**.
5. Enter Project Name: **Practical3**
6. Click **Create**.
7. Select **Empty**.



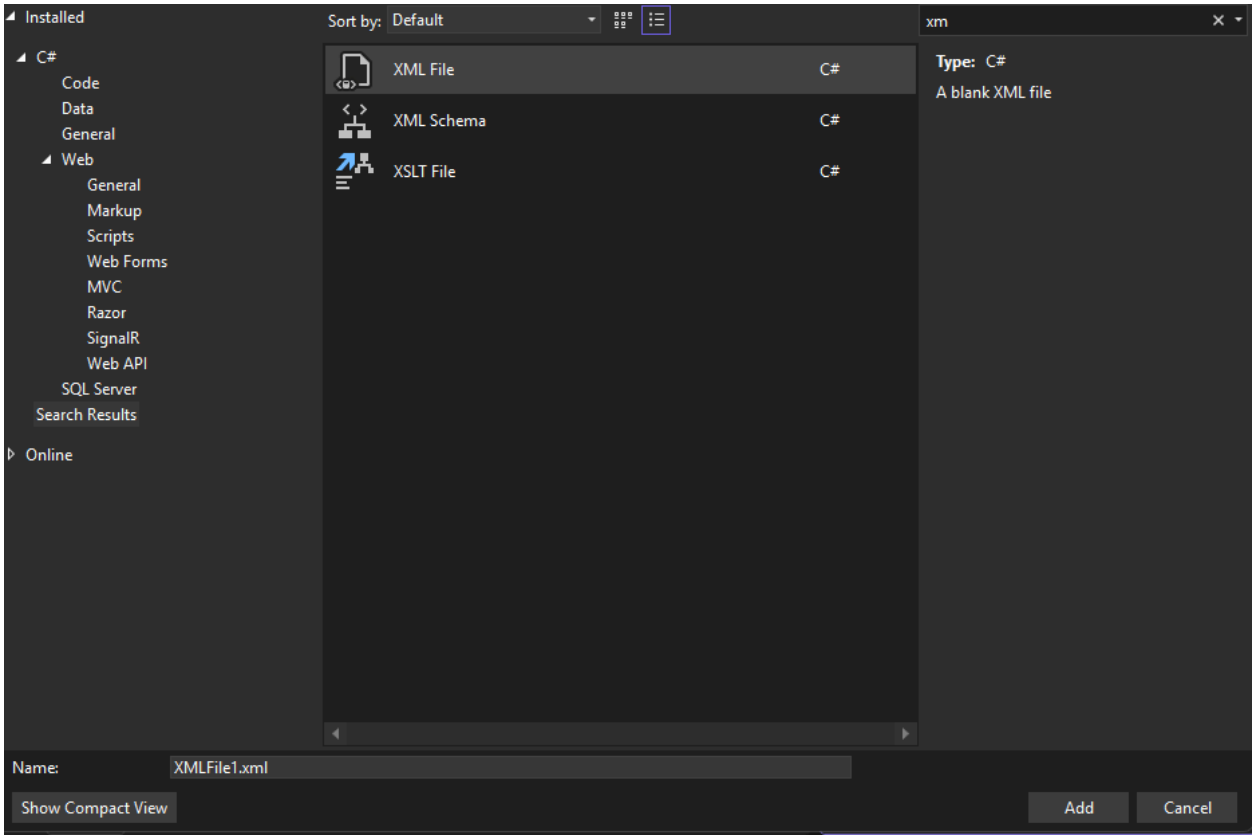
8. Click **Create**.

Step 2: Add Advertisement Images

1. Create the Image folder on your system.
2. Save the Advertisement images in your folder.
3. In Solution Explorer, right-click the project **Practical4c.**
4. Select Add → Existing Item.
5. Navigate to Image Folder.
6. Select all images.
7. Click **Add**.

Step 3: Add an XML File

8. In Solution Explorer, right-click the project **Practical4c.**
9. Select Add → New Item.
10. Select XML File.
11. Click **Add**.



Open **XMLFile1.xml** and write the following code.

XMLFile1.xml

```
<?xml version="1.0" encoding="utf-8" ?>

<Advertisements>

    <Ad>
        <ImageUrl>Images/Bigbasket.png</ImageUrl>
        <NavigateUrl>https://www.bigbasket.com</NavigateUrl>
        <AlternateText>BigBasket Grocery Shopping</AlternateText>
        <Keyword>Grocery</Keyword>
        <Impressions>50</Impressions>
    </Ad>

    <Ad>
        <ImageUrl>Images/blinkit.png</ImageUrl>
```

```
        <NavigateUrl>https://www.blinkit.com</NavigateUrl>
        <AlternateText>Blinkit Grocery Delivery</AlternateText>
        <Keyword>Grocery</Keyword>
        <Impressions>40</Impressions>
    </Ad>

    <Ad>
        <ImageUrl>Images/jiomart.png</ImageUrl>
        <NavigateUrl>https://www.jiomart.com</NavigateUrl>
        <AlternateText>JioMart Online Grocery</AlternateText>
        <Keyword>Grocery</Keyword>
        <Impressions>30</Impressions>
    </Ad>

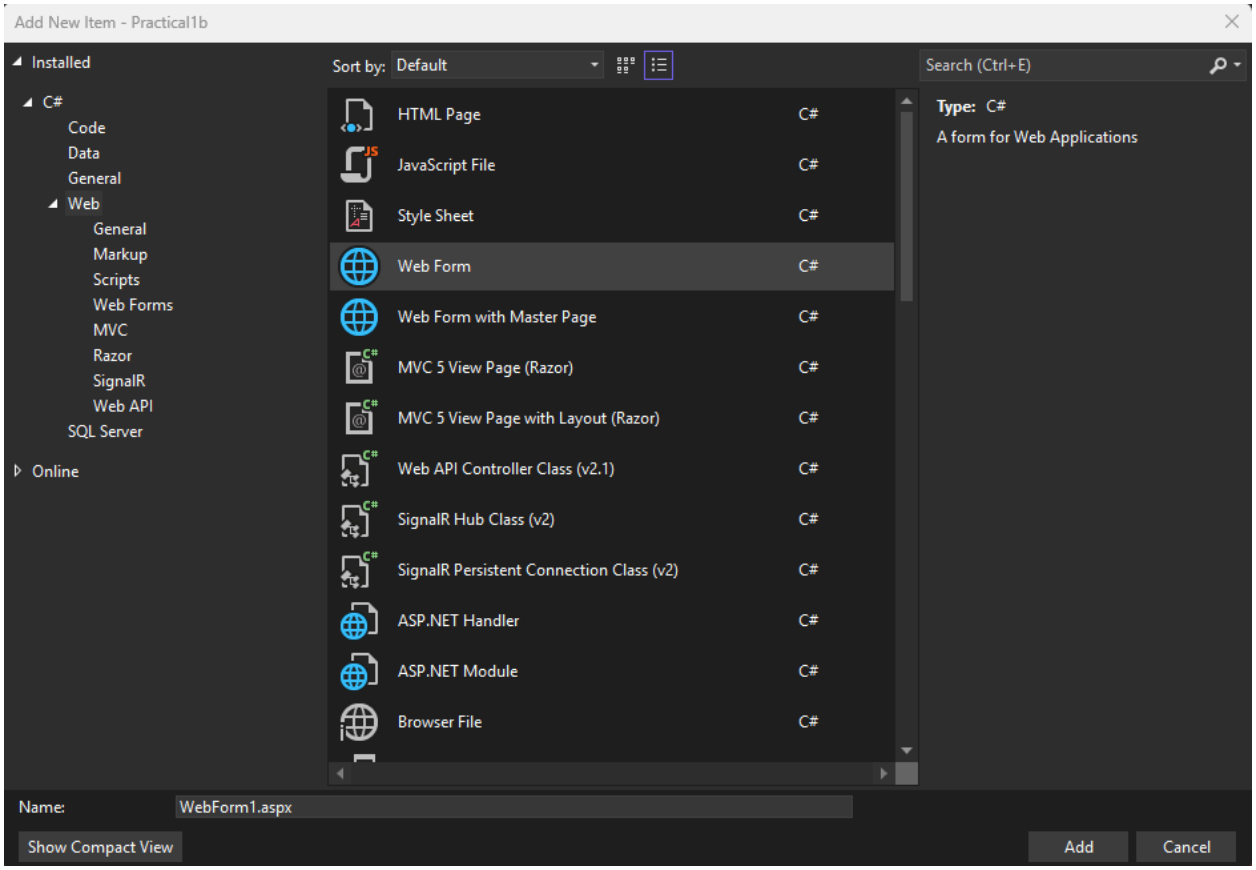
    <Ad>
        <ImageUrl>Images/myntra.jpg</ImageUrl>
        <NavigateUrl>https://www.myntra.com</NavigateUrl>
        <AlternateText>Myntra Fashion Store</AlternateText>
        <Keyword>Fashion</Keyword>
        <Impressions>50</Impressions>
    </Ad>

    <Ad>
        <ImageUrl>Images/ajio.jpg</ImageUrl>
        <NavigateUrl>https://www.ajio.com</NavigateUrl>
        <AlternateText>AJIO Fashion Shopping</AlternateText>
        <Keyword>Fashion</Keyword>
        <Impressions>40</Impressions>
    </Ad>

</Advertisements>
```

Step 4: Add a Web Form

1. In Solution Explorer, right-click the project **Practical4c.**
2. Select Add → New Item.
3. Select Web Form.
4. Click **Add**.



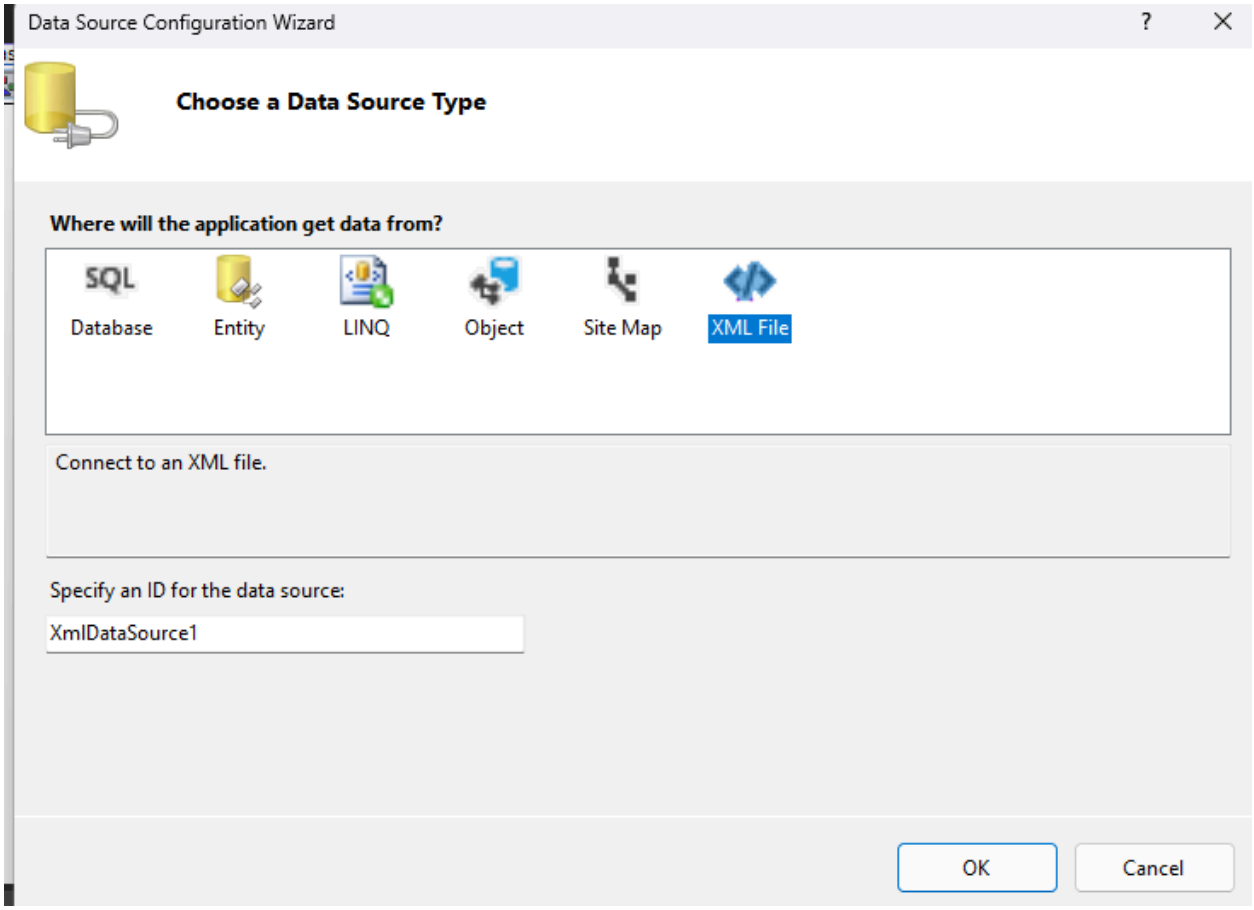
Step 5: Design Web Form

Open **WebForm1.aspx** and add the following controls:

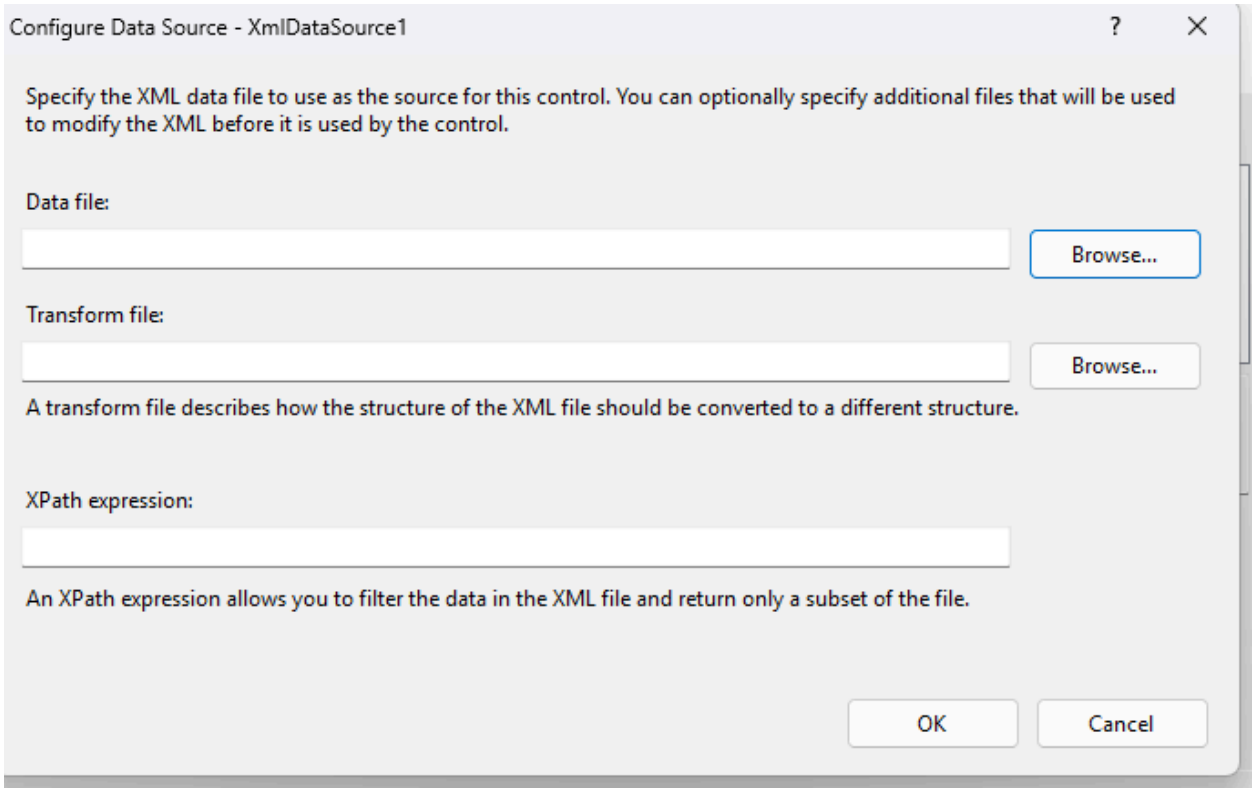
1. AdRotator

Add DataSource to AdRotator

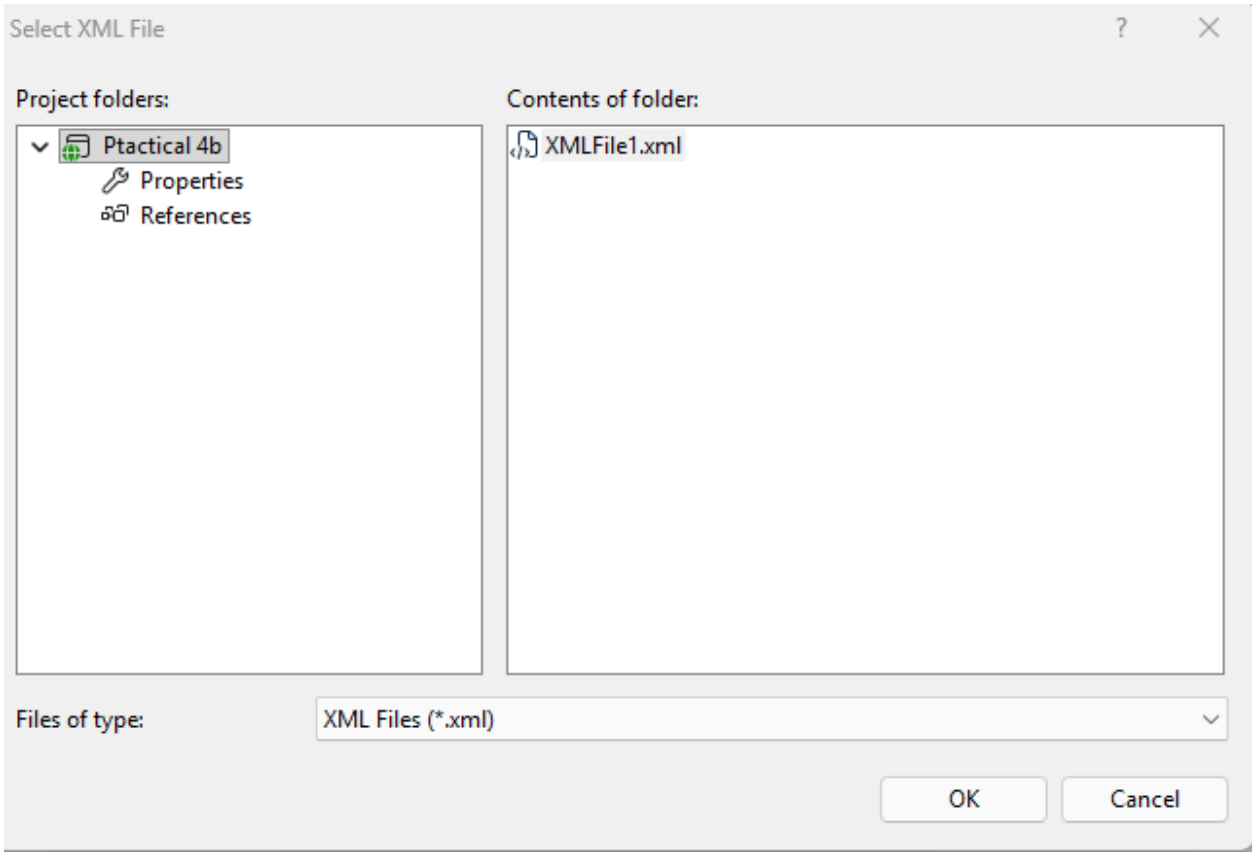
1. Click AdRotator.
2. Select **New Data Source**.
3. Select XML File.

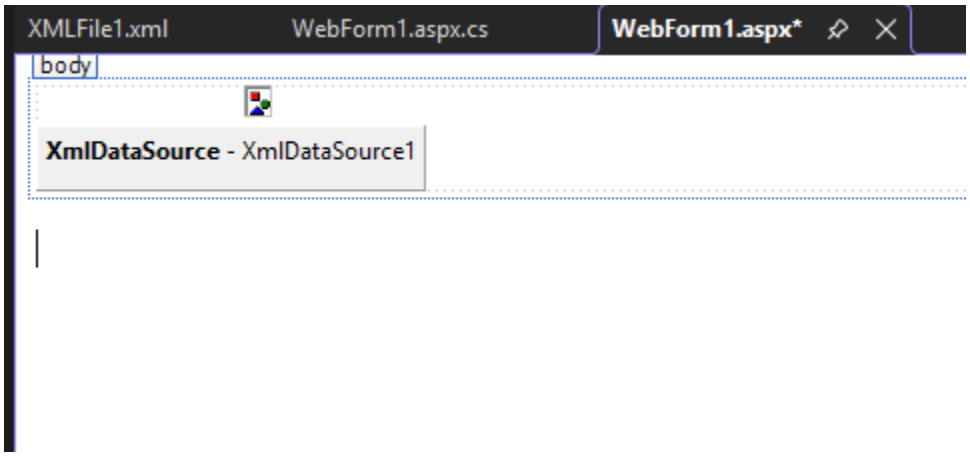


4. Click **OK.**
5. Select the Data file by clicking the Browse button.



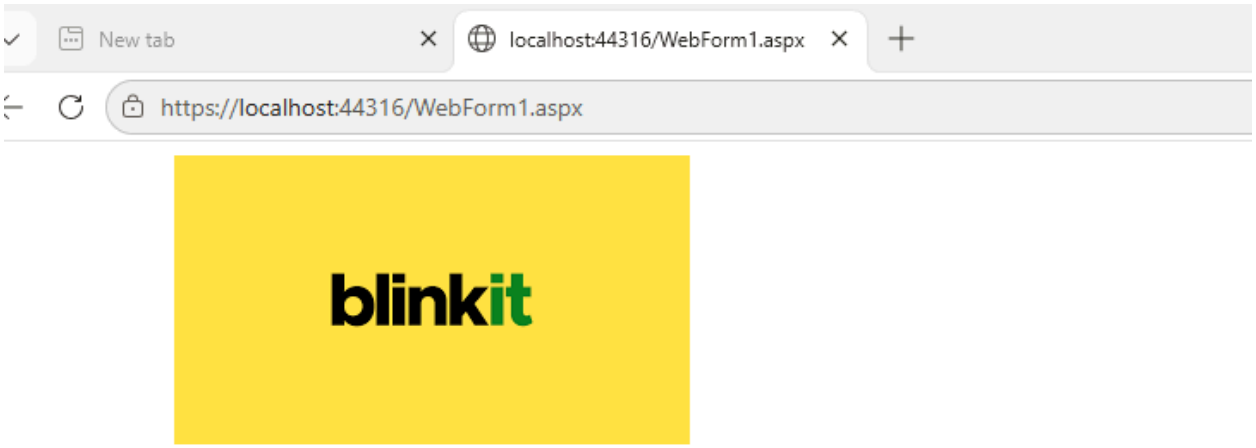
6. Select XMLFile1.xml → OK → OK.

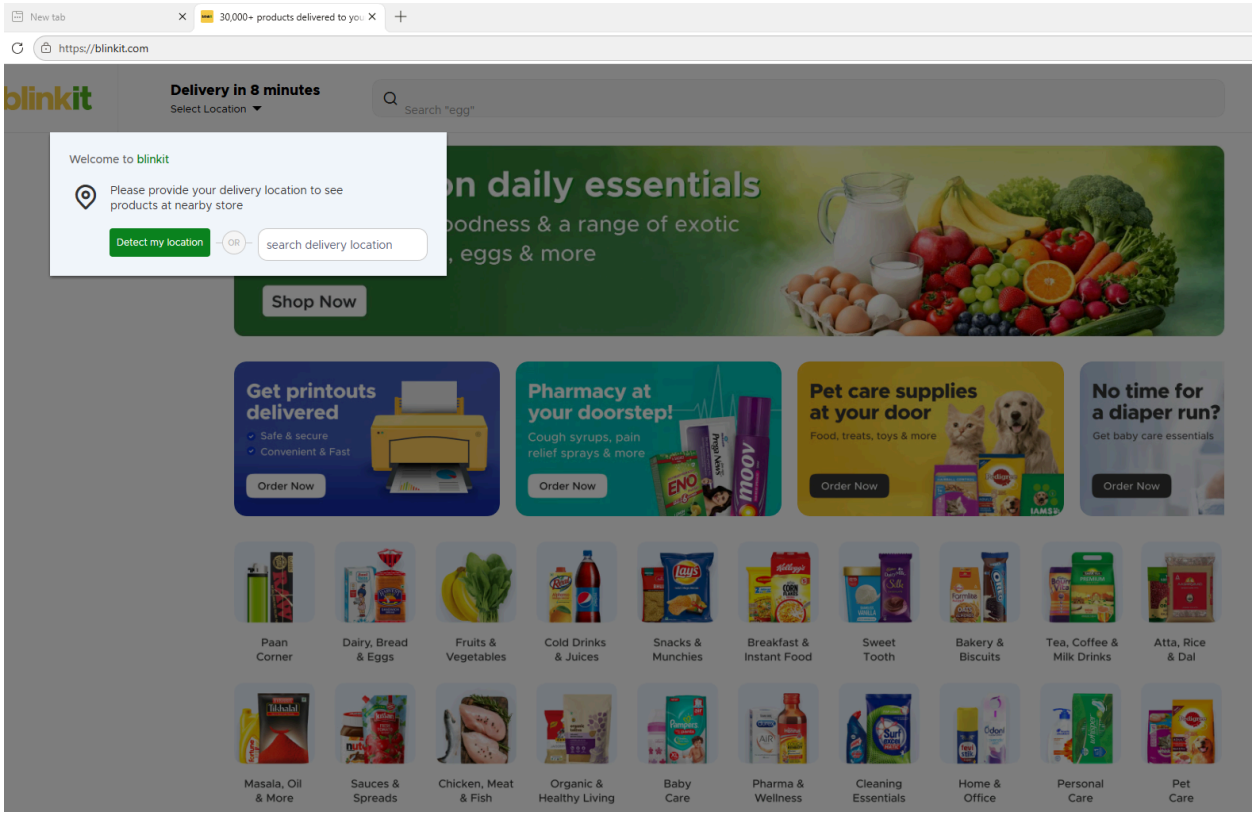




Step 6: Run the Application

1. Save all files.
2. Press **Ctrl + F5**.


New tab
localhost:44316/WebForm1.aspx
https://localhost:44316/WebForm1.aspx
blinkit


New tab
30,000+ products delivered to you
https://blinkit.com
blinkit
Delivery in 8 minutes
Select Location
Search "egg"
Welcome to blinkit
Please provide your delivery location to see products at nearby store
Detect my location
OR
search delivery location
daily essentials
eggs & more
Shop Now
Get printouts delivered
Safe & secure
Convenient & Fast
Order Now
Pharmacy at your doorstep!
Cough syrups, pain relief sprays & more
Order Now
Pet care supplies at your door
Food, treats, toys & more
Order Now
No time for a diaper run?
Get baby care essentials
Order Now
Paan Corner
Dairy, Bread & Eggs
Fruits & Vegetables
Cold Drinks & Juices
Snacks & Munchies
Breakfast & Instant Food
Sweet Tooth
Bakery & Biscuits
Tea, Coffee & Milk Drinks
Atta, Rice & Dal
Masala, Oil & More
Sauces & Spreads
Chicken, Meat & Fish
Organic & Healthy Living
Baby Care
Pharma & Wellness
Cleaning Essentials
Home & Office
Personal Care
Pet Care

Practical no. 5A

Aim: Create the following application:
a. Create a web application to demonstrate the use of different types of Cookies.

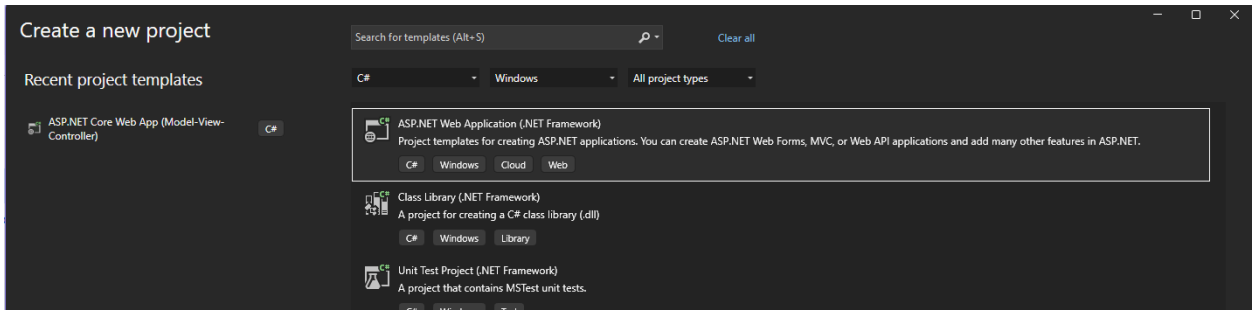
Software Required:

Visual Studio 2022

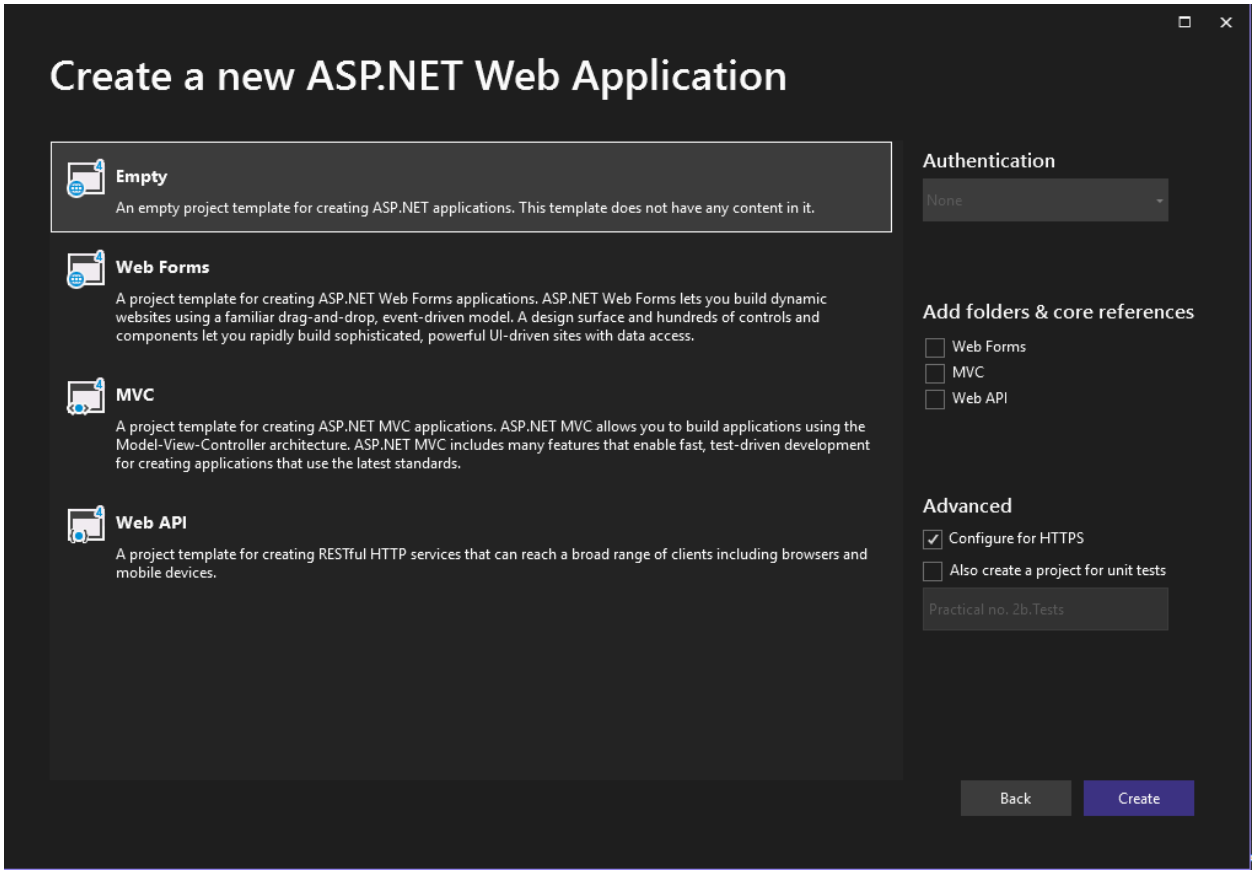
Procedure:

Step 1: Create New Project

1. Open Visual Studio.
2. Click **Create a new project**.
3. Select **ASP.NET Web Application (.NET Framework)**.



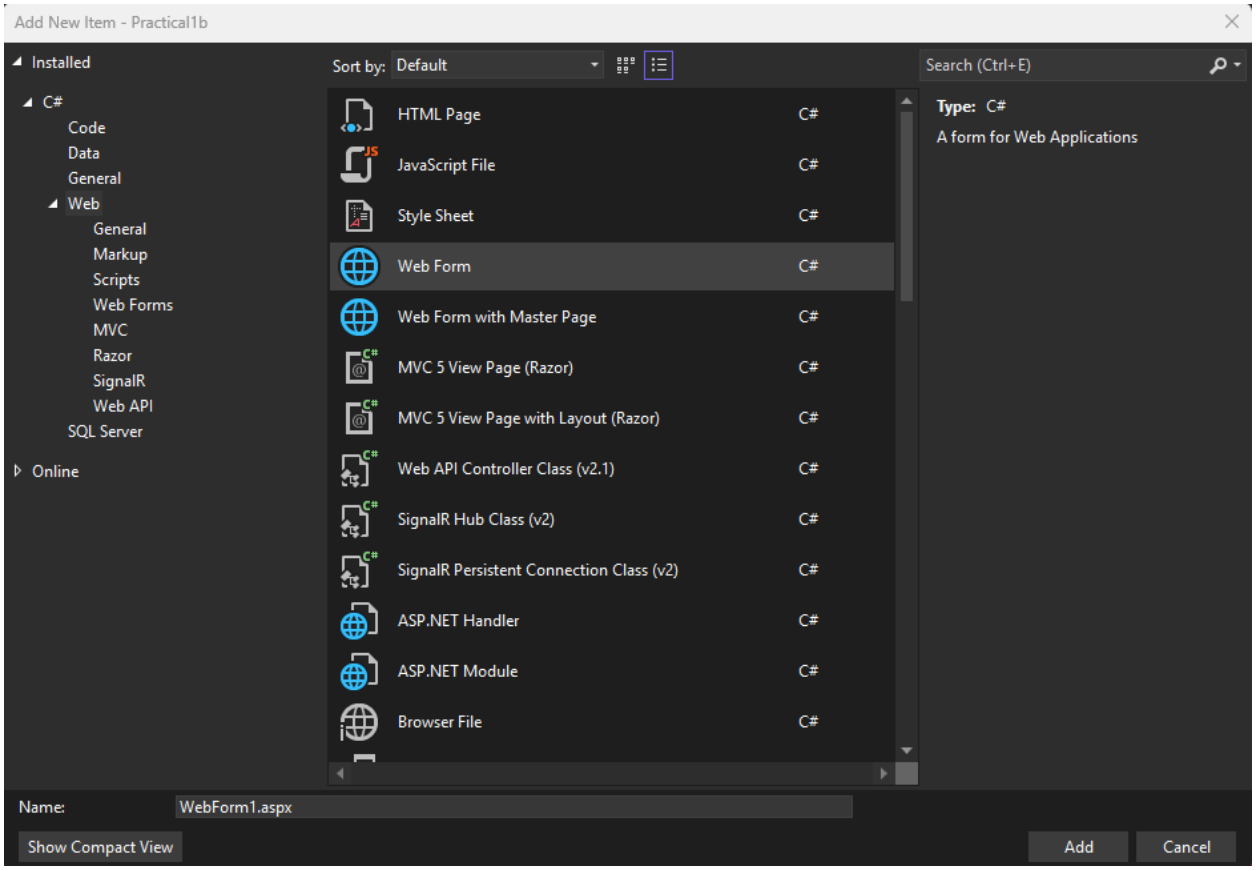
4. Click **Next**.
5. Enter Project Name: **Practical5A**
6. Click **Create**.
7. Select **Empty**.



8. Click **Create**.

Step 2: Add a Web Form

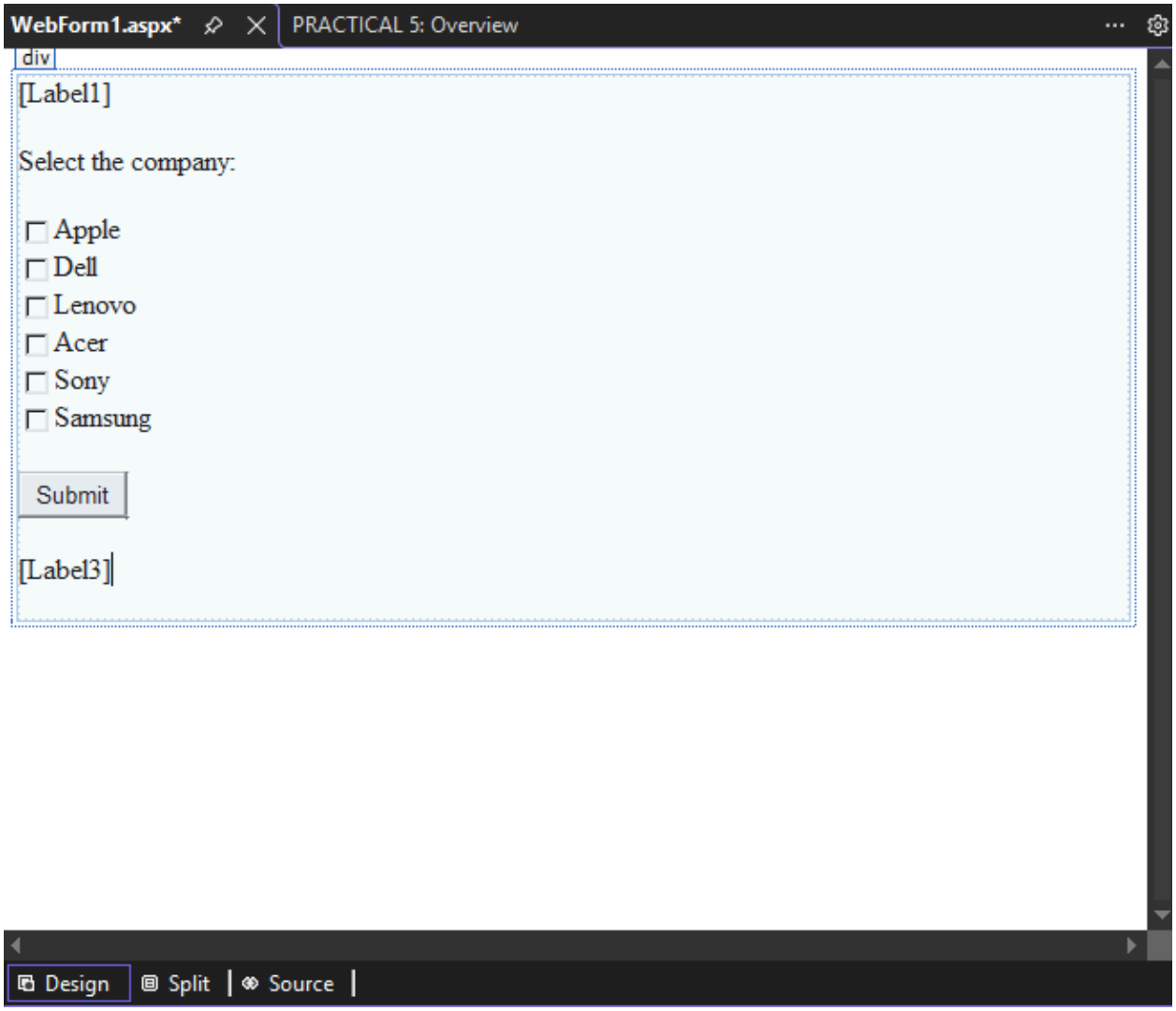
1. In Solution Explorer, right-click the project **Practical4B.**
2. Select Add → New Item.
3. Select Web Form.
4. Click **Add**.



Step 3: Design Web Form

Open **WebForm1.aspx** and add the following controls:

1. TextBox
2. Button
3. Label
4. CheckBox



Step 4: Write C# Code

Open **WebForm1.aspx.cs** and write the following code.

WebForm1.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.ConstrainedExecution;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace PRACTICAL_5
```

```
{
   public partial class WebForm1 : System.Web.UI.Page
   {
      protected void Page_Load(object sender, EventArgs e)
      {
         HttpCookie c1 = new HttpCookie("creator");
         c1.Value = "Ms. Jiteshree Raut";
         Response.Cookies.Add(c1);

         String author = Response.Cookies["creator"].Value;

         Label2.Text = author;
         Response.Cookies["comp"].Expires = DateTime.Now.AddDays(-1);
      }

      protected void Button1_Click(object sender, EventArgs e)
      {
         HttpCookie comp = new HttpCookie("comp");

         if (CheckBox1.Checked)
            comp["apple"] = "Apple";

         if (CheckBox2.Checked)
            comp["dell"] = "Dell";

         if (CheckBox3.Checked)
            comp["lenevo"] = "Lenovo";

         if (CheckBox4.Checked)
            comp["acer"] = "Acer";

         if (CheckBox5.Checked)
            comp["sony"] = "Sony";

         if (CheckBox6.Checked)
            comp["Samsung"] = "Samsung";

         Response.Cookies.Add(comp);

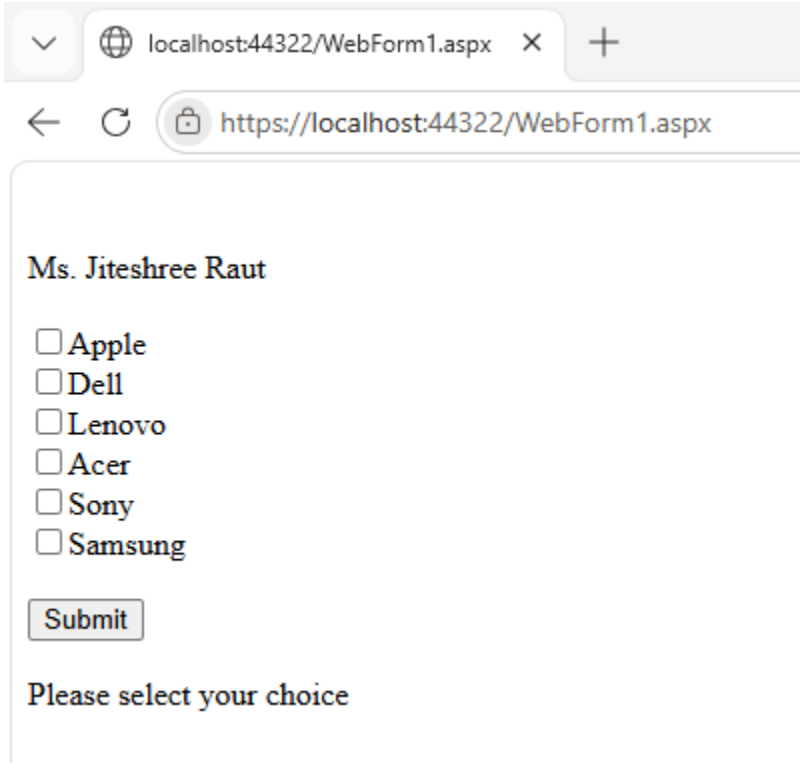
         Label3.Text = "";
```

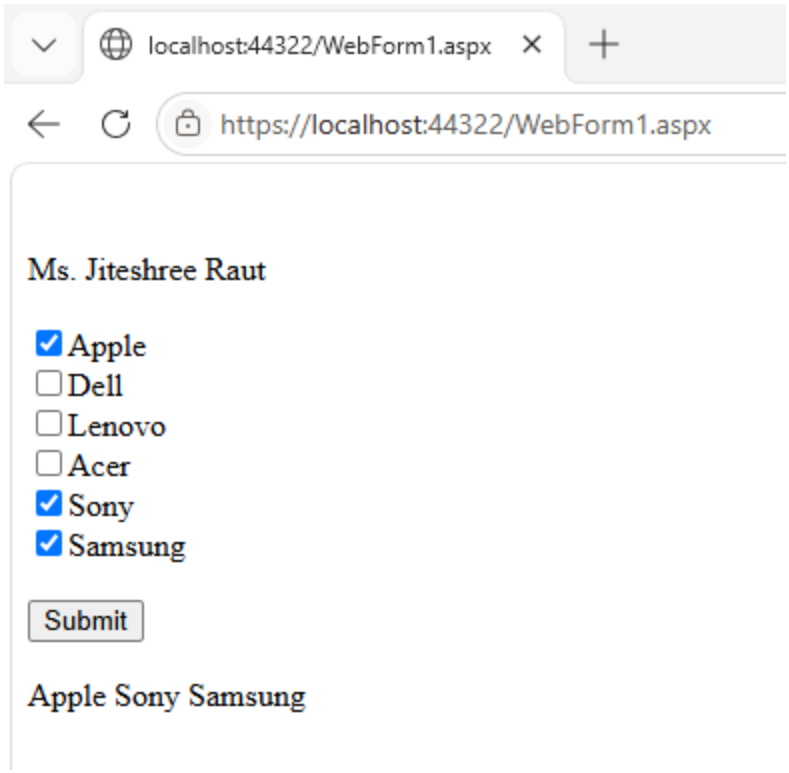
```
            foreach (string key in comp.Values.AllKeys)
            {
                Label3.Text += comp[key] + " ";
            }

            if (Label3.Text == "")
                Label3.Text = "Please select your choice";
        }
    }
}
```

Step 5: Run the Application

1. Save all files.
2. Press **Ctrl + F5**.
3. Leave CheckBox unchecked.
4. Click **Submit**.
5. Error messages will appear.
6. Check some CheckBox and click **Submit**.
7. Selected companies will be displayed.




localhost:44322/WebForm1.aspx
https://localhost:44322/WebForm1.aspx
Ms. Jiteshree Raut
Apple
Dell
Lenovo
Acer
Sony
Samsung
Submit
Apple Sony Samsung

Practical no. 5B

Aim: Create the following application:
b. Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

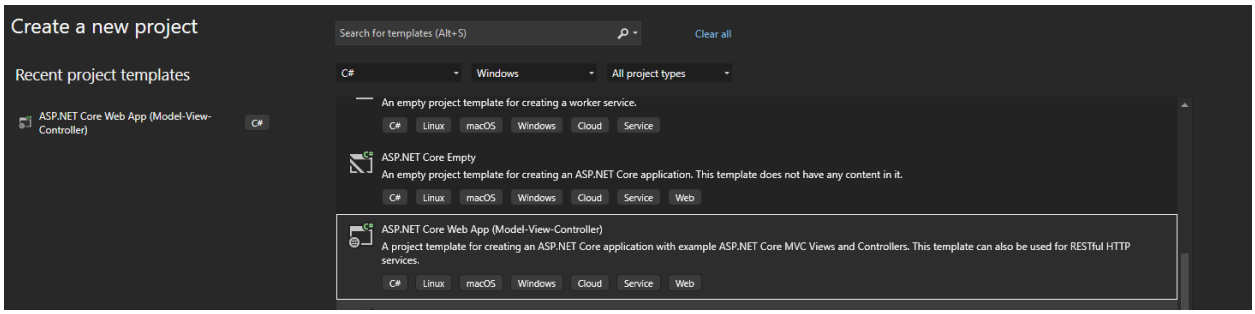
Software Required:

Visual Studio 2026

Procedure:

Step 1: Create New MVC Project

1. Open Visual Studio 2022.
2. Click Create a New Project.
3. Select ASP.NET Core Web App (Model-View-Controller).



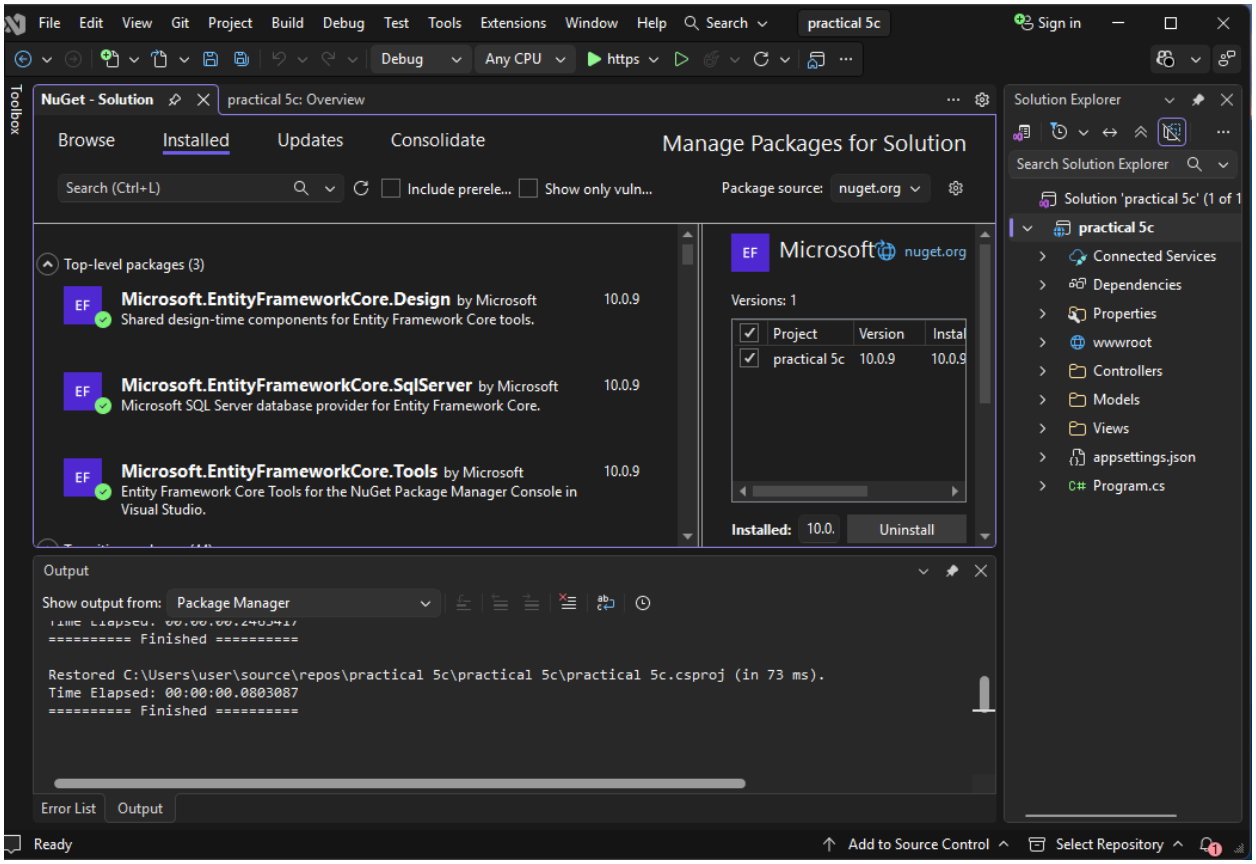
4. Click Next.
5. Enter Project Name: Practical5b
6. Select .NET 8.0.
7. Click Create.

Step 2: Install packages

Tools → NuGet Package Manager → Manage NuGet Packages

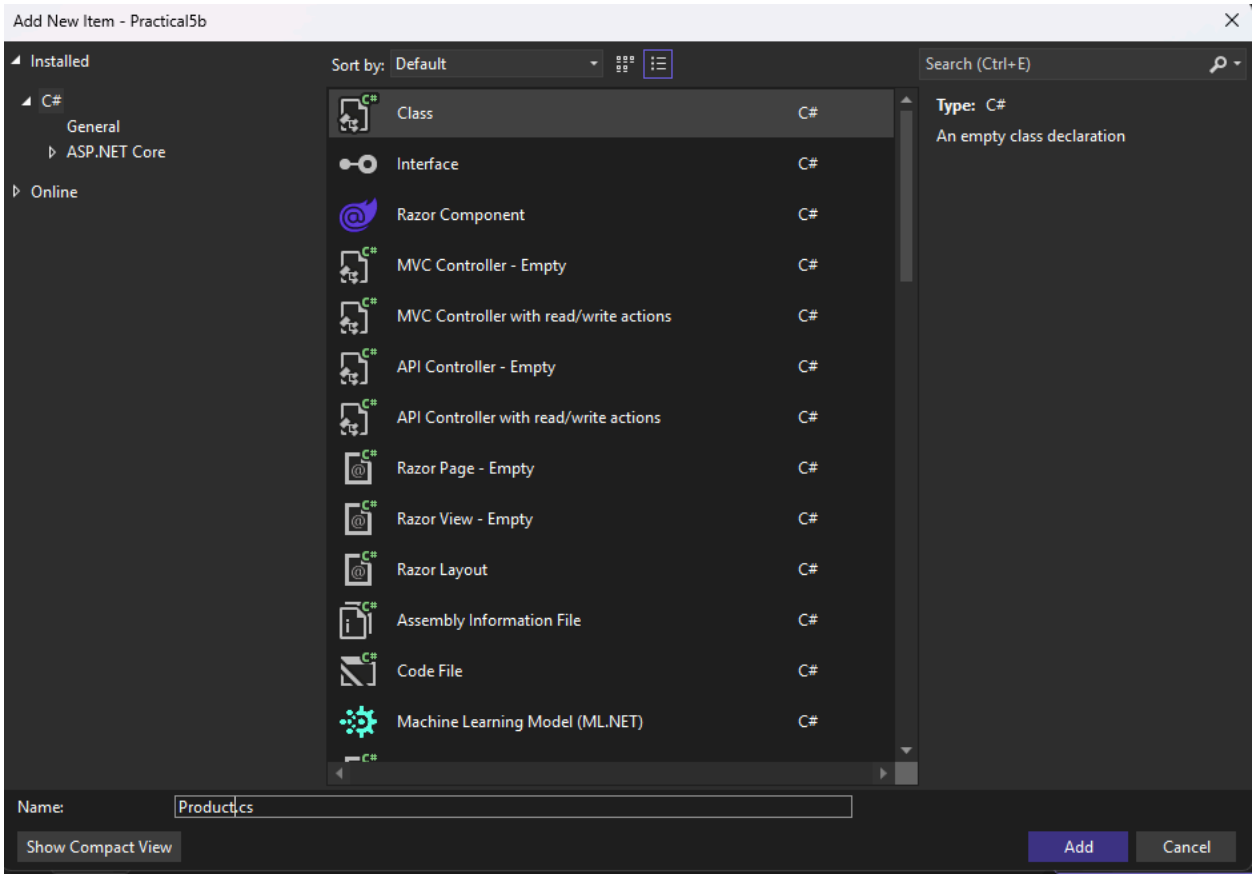
Install:

1. Microsoft.EntityFrameworkCore.Design
2. Microsoft.EntityFrameworkCore.SqlServer
3. Microsoft.EntityFrameworkCore.Tools



Step 3: Create Product Model

1. Right-click Models folder.
2. Add → New Item → Class.
3. Name it: Product.cs
4. Click Add.



Product.cs

```
using System.ComponentModel.DataAnnotations;

namespace Practical5b.Models
{
   public class Product
   {
      public int Id { get; set; }

      [Required]
      public string ProductName { get; set; }

      public decimal Price { get; set; }

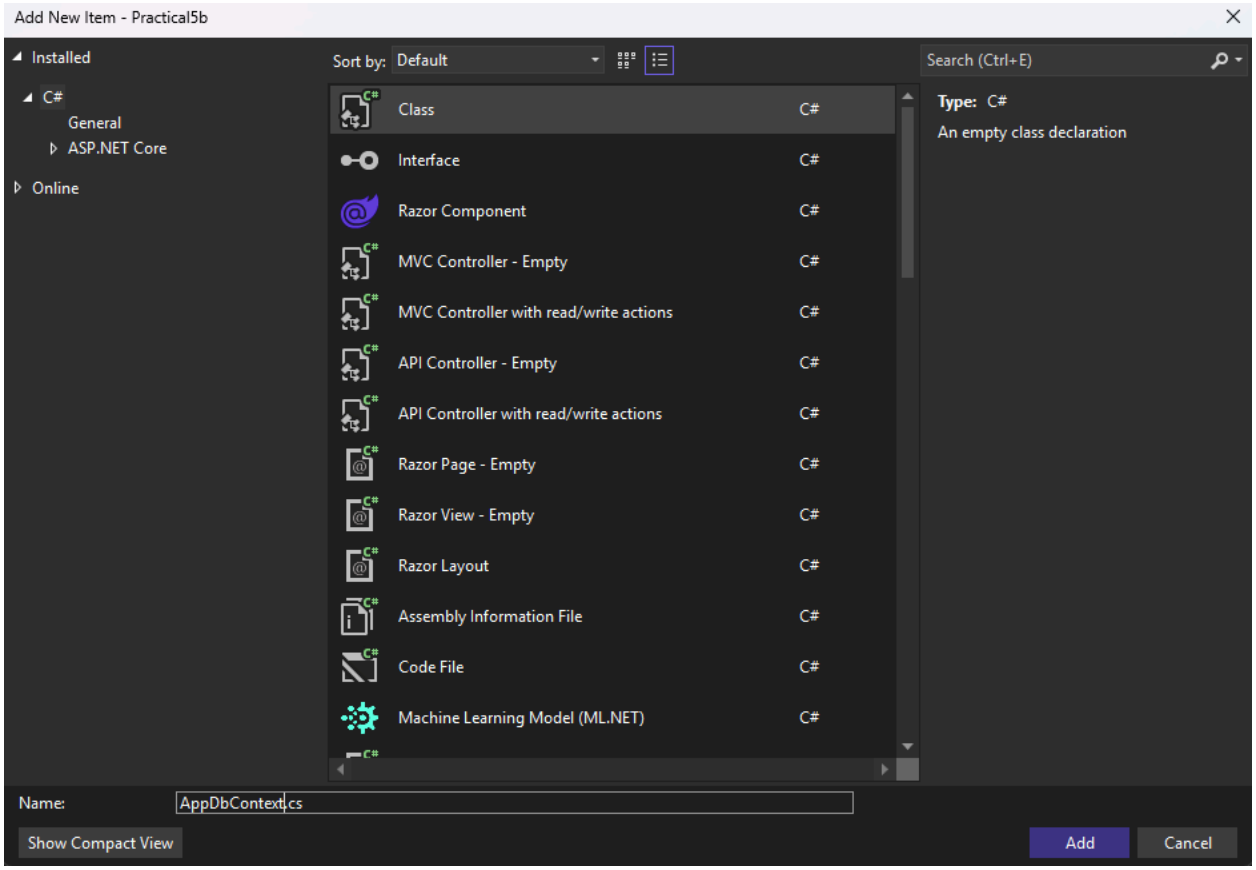
      public int Quantity { get; set; }
   }
}
```

Step 4: Create Data Folder

1. Right-click project.
2. Add → New Folder.
3. Name it: Data

Step 5: Create DbContext Class

1. Right-click the Data folder.
2. Add → New Item → Class.
3. Name it: AppDbContext.cs



AppDbContext.cs

using Microsoft.EntityFrameworkCore;
using Practical5b.Models;

namespace Practical5b.Data
{

```
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Product> Products { get; set; }
    }
}
```

Step 6: Configure Database Connection String
Open Solution Explorer → appsettings.json.

appsettings.json

```
{
  "ConnectionStrings": {
    "ProductConnection":
"Server=(localdb)\\MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;Tru
stServerCertificate=True;"
  },

  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },

  "AllowedHosts": "*"
}
```

Step 7: Configure EF Core in Program.cs
Open Solution Explorer → program.cs.
program.cs

```
using Microsoft.EntityFrameworkCore;
using Practical5b.Data;
```

```
var builder = WebApplication.CreateBuilder(args);

// Add MVC Services
builder.Services.AddControllersWithViews();

// Add Database Context
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("ProductConnection")));

var app = builder.Build();

// Configure HTTP Pipeline
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Product}/{action=Create}/{id?}").WithStaticAssets();

app.Run();
```

Step 8: Create Database Using Migration

Tools → NuGet Package Manager → Package Manager Console

Run following commands in power shell:

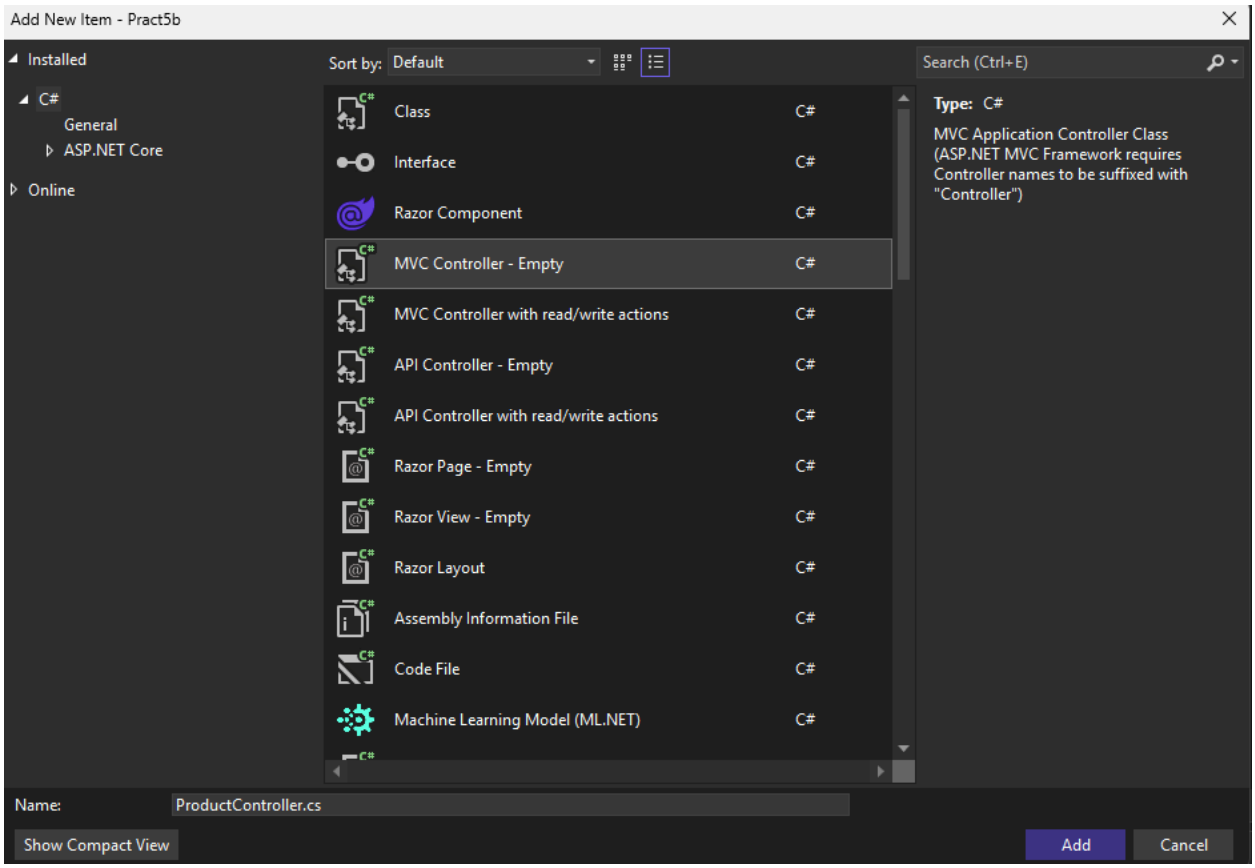
1. Add-Migration InitialCreate
2. Update-Database

EF Core automatically creates: ProductDB Database and Products Table.

Step 9: Create ProductController

Create a ProductController inside the Controllers folder.

1. Right Click Controllers.
2. Click Add.
3. Click New item→ MVC Controller -Empty
4. Name it: ProductController.cs
5. Click Add.



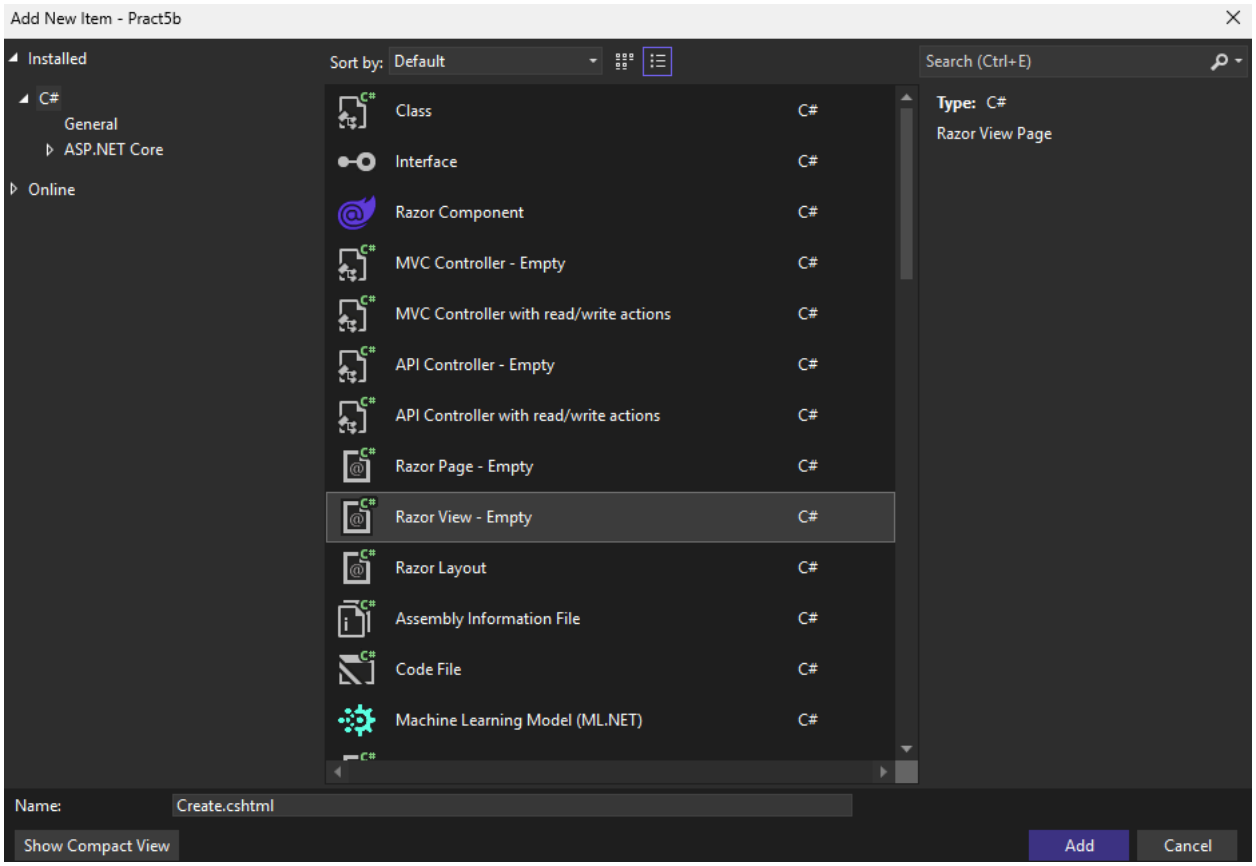

```
using Microsoft.AspNetCore.Mvc;
using Pract5b.Models;
using Pract5b.Data;
namespace Pract5b.Controllers
{
   public class ProductController : Controller
   {
```

```
        private readonly AppDbContext _context;
        public ProductController(AppDbContext context)
        {
            _context = context;
        }
        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Product product)
        {
            if (ModelState.IsValid)
            {
                _context.Products.Add(product);
                _context.SaveChanges();
                ViewBag.Message = "Product inserted successfully!";
                ModelState.Clear();
            }
            return View();
        }
    }
}
```

Step 9: Create View

1. Right-click View Folder.
2. Add → New Folder.
3. Name it: Product
4. Right-click the Product folder.
5. Add → New Item → razor View-Empty
6. Name it: Create.cshtml




```
@model Pract5b.Models.Product

@{
    ViewData["Title"] = "Create";
}

<h2>Add Product</h2>

<form asp-action="Create" method="post">

    <div>
        <label>Product Name</label>
        <input asp-for="ProductName" />
    </div>

    <br />

    <div>
        <label>Price</label>
```

```
        <input asp-for="Price" type="number" step="0.01" />
    </div>

    <br />

    <div>
        <label>Quantity</label>
        <input asp-for="Quantity" type="number" />
    </div>

    <br />

    <button type="submit">Insert Product</button>

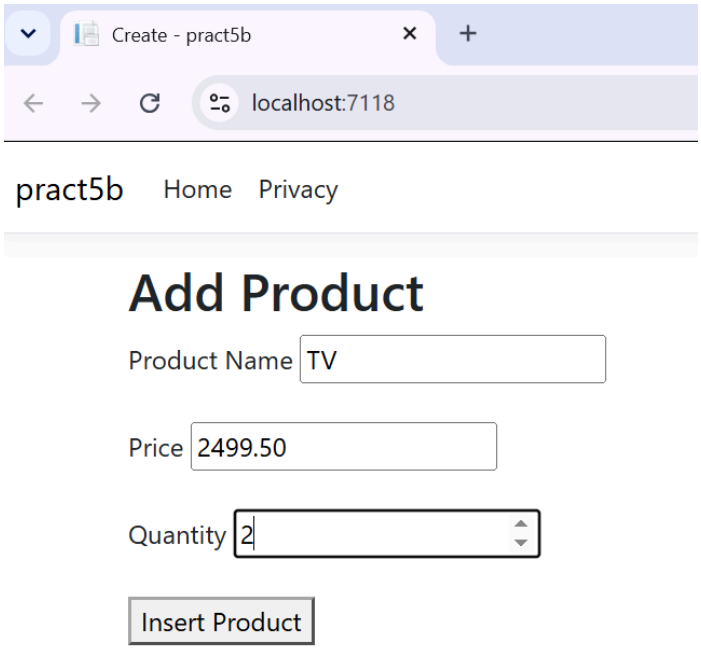
</form>

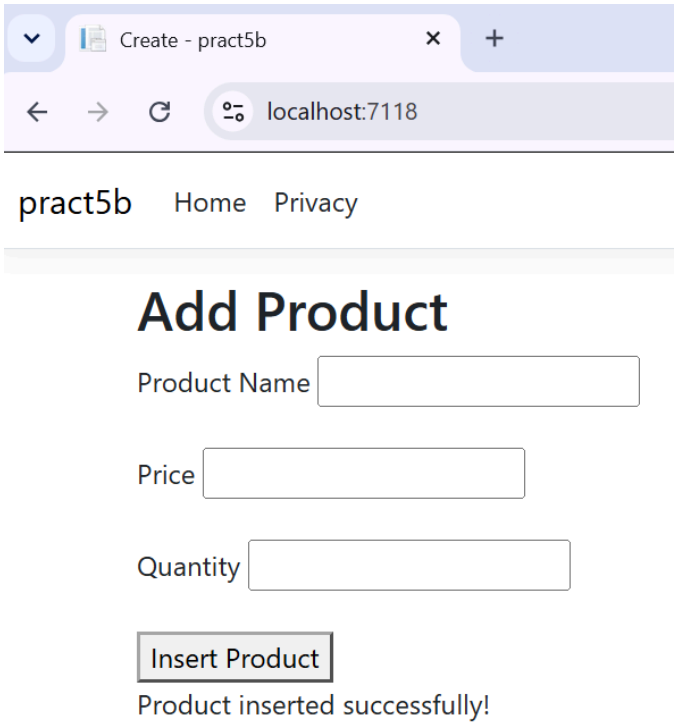
@if (ViewBag.Message != null)
{
    <p>@ViewBag.Message</p>
}
```

Step 10: Build Project
Ctrl + Shift + B

Step 11: Run Application

Press F5.


Create - pract5b
localhost:7118
pract5b
Home
Privacy
Add Product
Product Name
TV
Price
2499.50
Quantity
2
Insert Product


Create - pract5b
localhost:7118
pract5b
Home
Privacy
Add Product
Product Name
Price
Quantity
Insert Product
Product inserted successfully!

Practical no. 6A

Aim: a. Create a simple website for a College Information Portal and convert it into a Progressive Web Application.

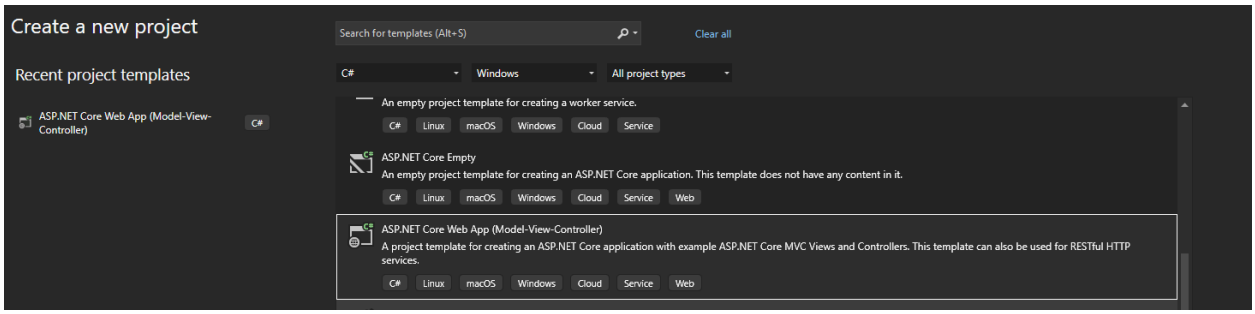
Software Required:

Visual Studio 2026

Procedure:

Step 1: Create New MVC Project

1. Open Visual Studio 2022.
2. Click Create a New Project.
3. Select ASP.NET Core Web App (Model-View-Controller).



4. Click Next.
5. Enter Project Name: Practical6b
6. Select .NET 8.0.
7. Click Create.

Step 2: Open Index.cshtml

Open:

Views → Home → Index.cshtml

Delete the existing code and enter:

```
@{
    ViewData["Title"] = "College Information Portal";
}

<div class="text-center">
    <h1>College Information Portal</h1>
    <p>Welcome to our College Information Portal</p>
</div>
```

```
<hr />

<h2>About College</h2>
<p>
   Our college provides quality education and develops
   technical and professional skills among students.
</p>

<h2>Courses Offered</h2>
<ul>
   <li>B.Sc. Information Technology</li>
   <li>B.Sc. Computer Science</li>
   <li>B.Com.</li>
   <li>B.A.</li>
</ul>

<h2>College Facilities</h2>
<ul>
   <li>Computer Laboratory</li>
   <li>Library</li>
   <li>Internet Facility</li>
   <li>Sports Facility</li>
</ul>

<h2>Contact Us</h2>
<p>
   Email: info@college.edu<br />
   Phone: 0250-1234567<br />
   Address: College Road, Maharashtra
</p>
```

Step 3: Open site.css

Open:

wwwroot → css → site.css

Add the following CSS:

```
body {
   font-family: Arial, sans-serif;
}

h1 {
   color: #0d6efd;
```

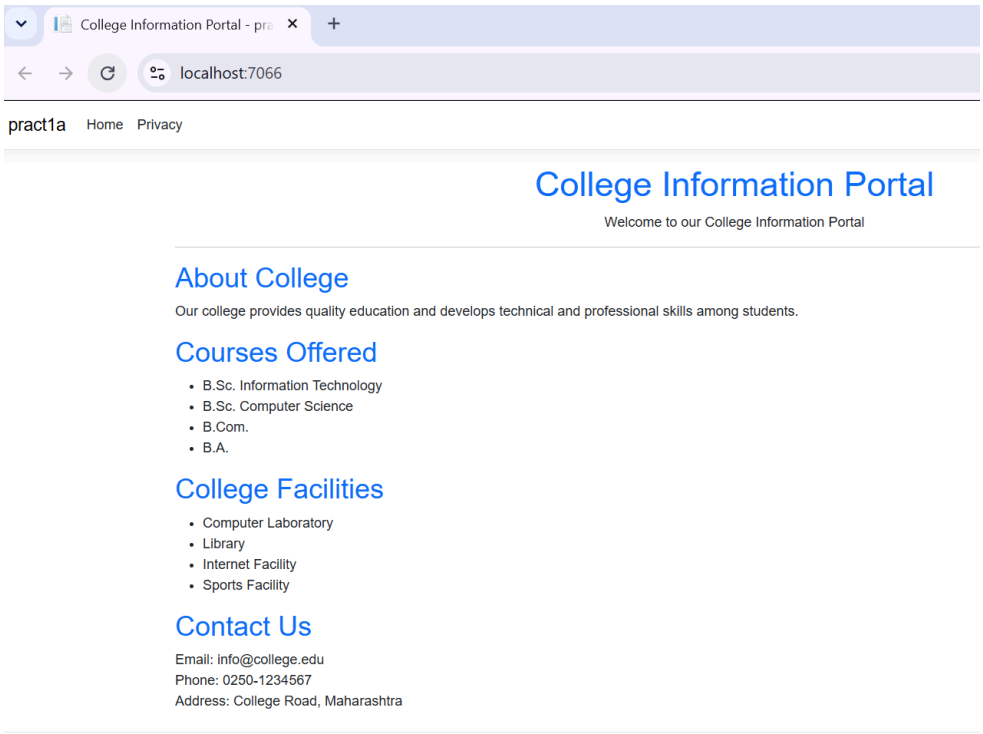
```
}

h2 {
    color: #0d6efd;
}

.text-center {
    text-align: center;
}
```

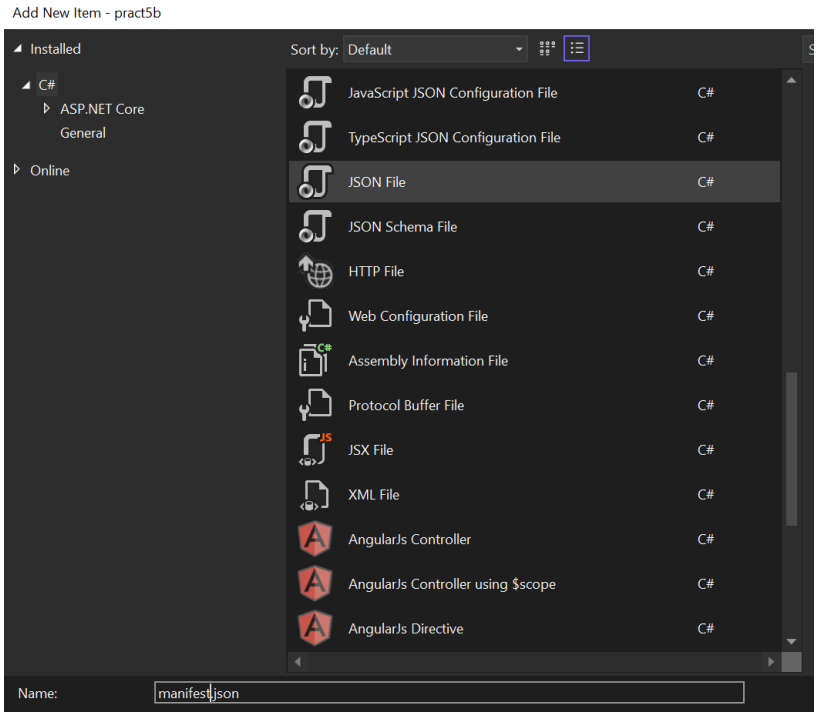
Step 4: Run and Test the Website

Press Ctrl + F5



Step 5: Create manifest.json

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select json file.
4. Enter File Name: **manifest.json**
5. Click Add.



Enter:

```
{
 "name": "College Information Portal",
 "short_name": "College Portal",
 "start_url": "/",
 "display": "standalone",
 "background_color": "#ffffff",
 "theme_color": "#0d6efd",
 "icons": [
   {
    "src": "/icons/icon-192.png",
    "sizes": "192x192",
    "type": "image/png"
   },
   {
    "src": "/icons/icon-512.png",
    "sizes": "512x512",
    "type": "image/png"
   }
 ]
}
```

Step 6: Add App Icon

1. Right Click → **wwwroot** folder

2. Click Add → New Folder
3. Enter Folder Name: **icons**
4. Paste one image in the **icons** folder.

Step 7: Edit _Layout.cshtml
Open:

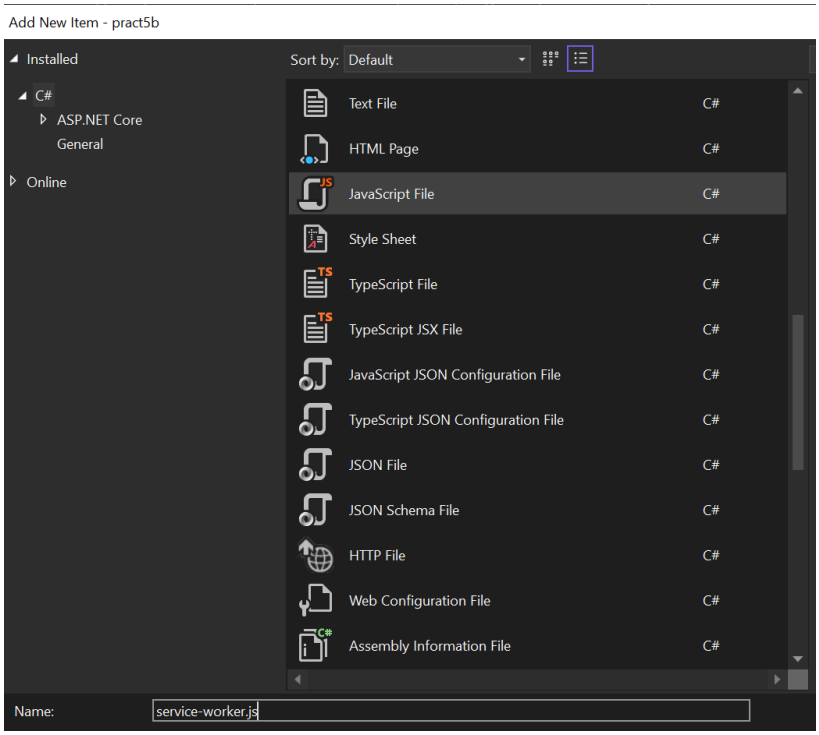
Views → Shared → _Layout.cshtml

Inside the <head> section, add:

```
<link rel="manifest" href="/manifest.json">
<meta name="theme-color" content="#0d6efd">
```

Step 8: Create Service Worker

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select JavaScript file.
4. Enter File Name: **service-worker.js**
5. Click Add.



Write the following code in the service-worker.js file.

service-worker.js

```
const CACHE_NAME = "college-portal-v1";

const FILES_TO_CACHE = [
    "/",
    "/css/site.css",
    "/manifest.json"
];

self.addEventListener("install", function (event) {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(function (cache) {
                return cache.addAll(FILES_TO_CACHE);
            })
    );
});

self.addEventListener("fetch", function (event) {
    event.respondWith(
        caches.match(event.request)
            .then(function (response) {
                return response || fetch(event.request);
            })
    );
});
```

Step 9: Register the Service Worker
Open:

wwwroot → js → site.js

Add:

```
if ("serviceWorker" in navigator) {

    window.addEventListener("load", function () {

        navigator.serviceWorker.register(
            "/service-worker.js"
        );

    });

}
```

Step 10: Run the application
Press Ctrl + F5 and open the application in Chrome or Edge.

Step 11: Check the Manifest
Press F12 → Application → Manifest.

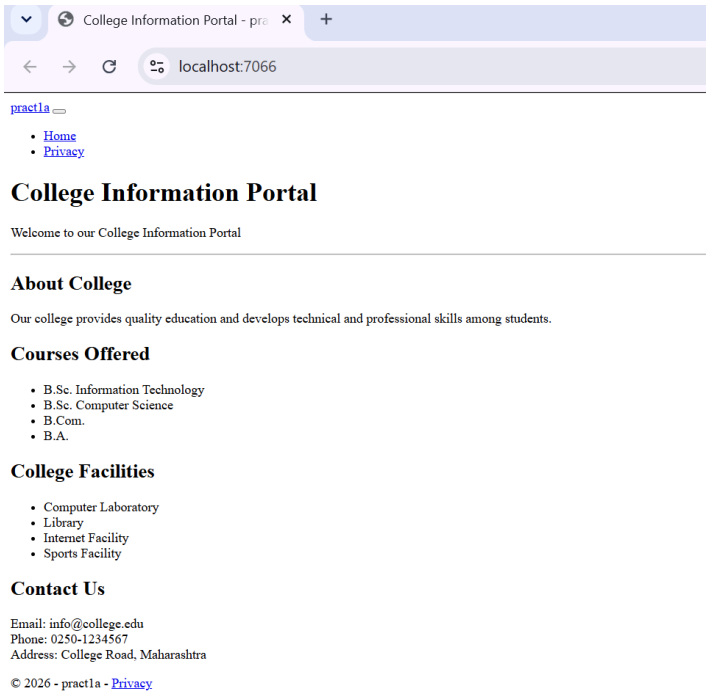
Check that the browser displays:

- College Information Portal
- College Portal
- Start URL: /
- Display: standalone
- Application icons

Step 12: Check the Service Worker
Go to F12 → Application → Service Workers.

Check that service-worker.js is registered and running.

Step 13: Test Offline Functionality
Open F12 → Network. → Select the Offline option and refresh the page.



Step 14: Install the PWA

1. Click Chrome Menu: ⋮ (Three Dots)
2. Select: Cast, Save and Share
3. Then click: Install Page as App or Install College Information Portal

How to Uninstall the PWA

1. Open installed application.
2. Click: ⋮ Menu
3. Select: Uninstall College Information Portal
4. Click: Remove

Practical no. 6B

Aim: Create a PWA that displays a custom offline page when the user is not connected to the internet.

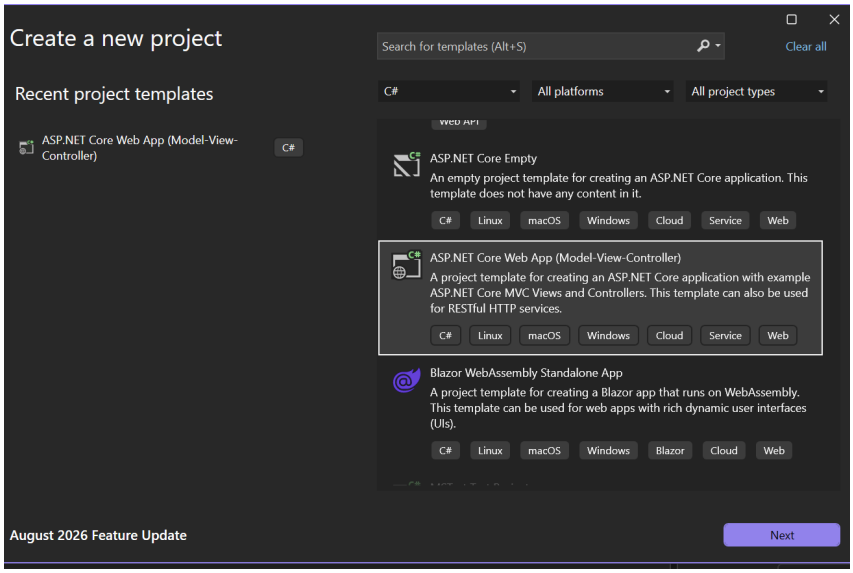
Software Required:

Visual Studio 2026

Procedure:

STEP 1: CREATE NEW PROJECT

1. Open Visual Studio 2026.
2. Select Create a new project.
3. Search for ASP.NET Core Web App (Model-View-Controller).
4. Click Next.
5. Set the Project Name to Practical No. 6b.
6. Select a suitable location and click Next.
7. Select the installed/latest .NET version and keep the default options.
8. Click Create.



STEP 2: CREATE HOME PAGE

Open Views → Home → Index.cshtml and replace its contents with:

```
@{
    ViewData["Title"] = "PWA Offline Demo";
}

<div style="text-align:center; margin-top:50px;">
```

```
    <h1>Welcome to My PWA</h1>

    <p>
        This is a Progressive Web Application.
    </p>

    <p>
        The application provides a custom page when
        the Internet connection is unavailable.
    </p>

</div>
<script>

    if ("serviceWorker" in navigator) {

        window.addEventListener("load", function () {

            navigator.serviceWorker
                .register("/service-worker.js")
                .then(function () {
                    console.log("Service Worker registered successfully.");
                })
                .catch(function (error) {
                    console.log("Service Worker registration failed:", error);
                });

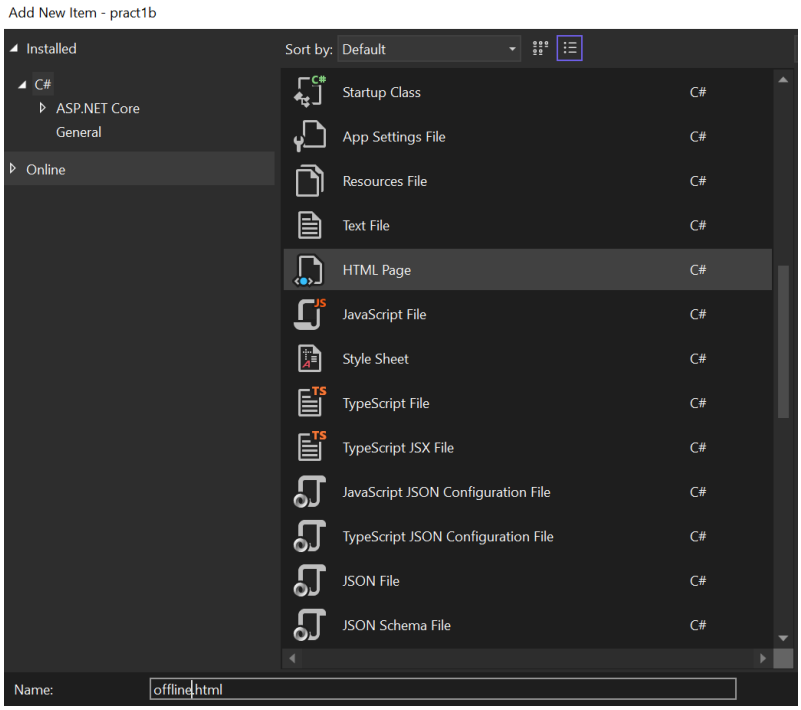
        });

    }

</script>
```

Step 3: Create the Custom Offline Page

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select html page.
4. Enter File Name: **offline.html**
5. Click Add.



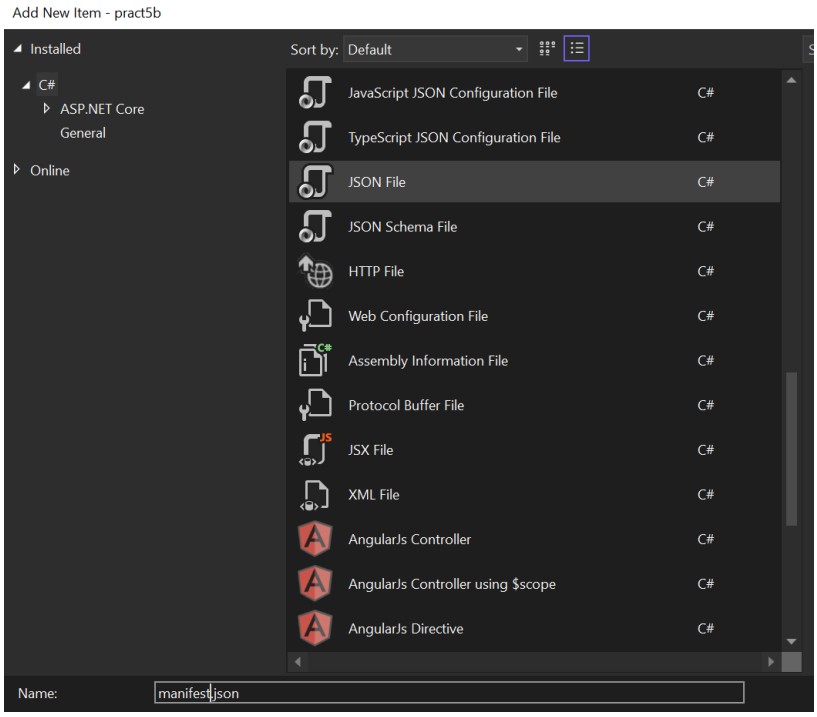
Write the following code.

offline.html

```
<!DOCTYPE html>
<html>
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>You are Offline</title>
</head>
<body style="text-align:center; font-family:Arial; padding:60px;">
   <h1> You are Offline</h1>
   <p>It looks like you are not connected to the Internet.</p>
   <p>Please check your Internet connection and try again.</p>
   <button onclick="location.reload()">Try Again</button>
</body>
</html>
```

Step 4: Create manifest.json

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select json file.
4. Enter File Name: **manifest.json**
5. Click Add.



Enter:

```
{
  "name": "PWA Offline Demo",
  "short_name": "PWA Demo",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#007bff"
}
```

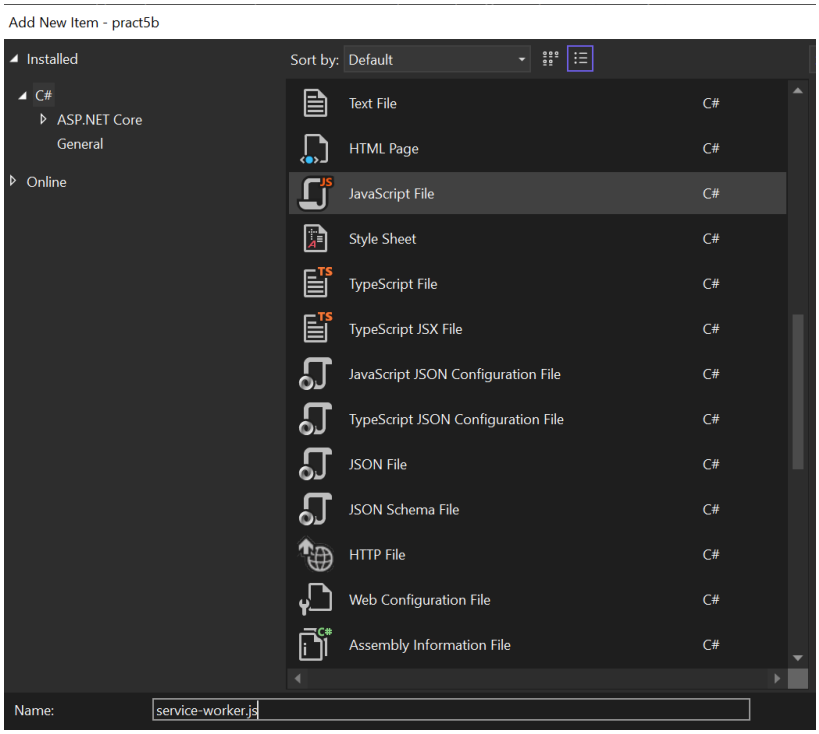
Step 4 : Add the Manifest
Open Views → Shared → _Layout.cshtml.

Inside the <head> section, add:

```
<link rel="manifest" href="/manifest.json">
<meta name="theme-color" content="#007bff">
```

Step 8: Create Service Worker

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select JavaScript file.
4. Enter File Name: **service-worker.js**
5. Click Add.



Write the following code in the service-worker.js file.

service-worker.js

```
const CACHE_NAME = "pwa-cache-v1";

const FILES_TO_CACHE = [
    "/",
    "/offline.html",
    "/manifest.json"
];

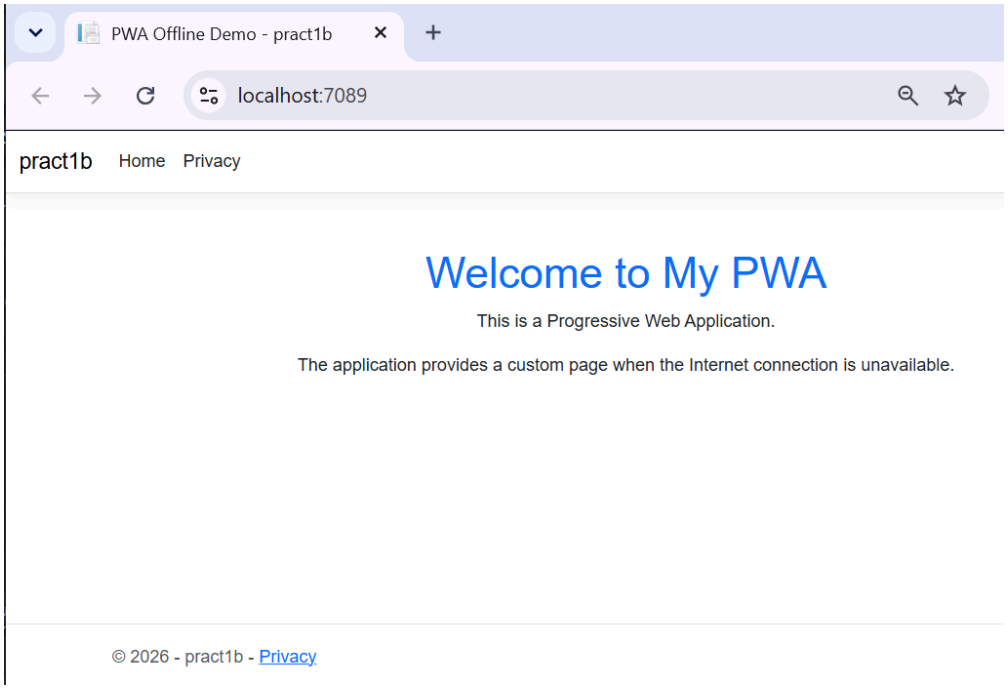
self.addEventListener("install", function (event) {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(function (cache) {
                return cache.addAll(FILES_TO_CACHE);
            })
    );
});

self.addEventListener("fetch", function (event) {
    event.respondWith(
        fetch(event.request)
            .catch(function () {
                return caches.match("/offline.html");
            })
```

```
    );
});
```

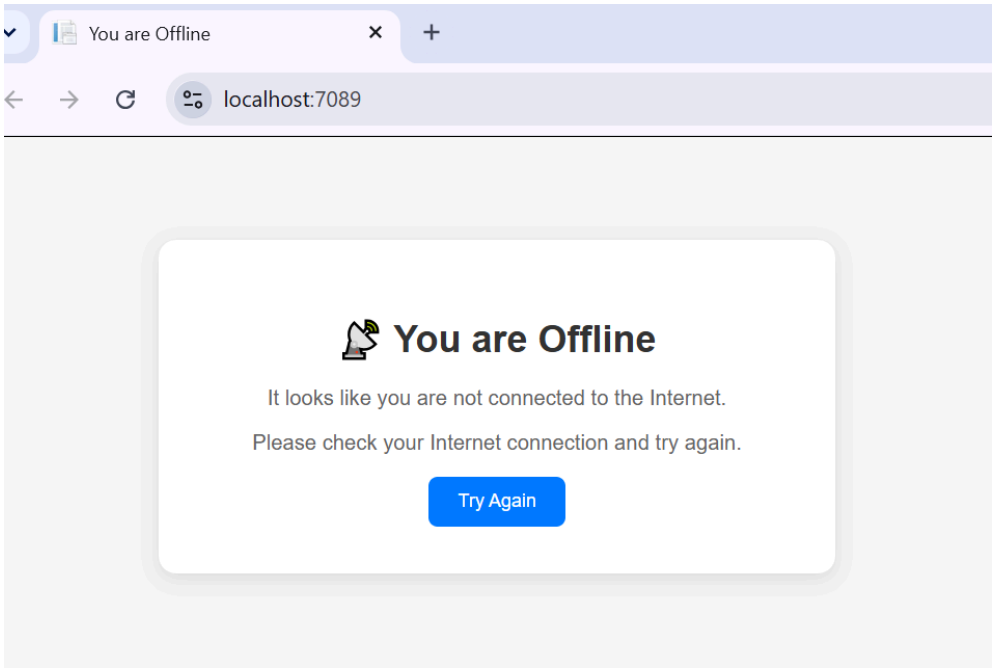
Step 9: Run the Application

Press F5



Step 10: Test Offline Mode

1. Press: F12
2. Open: Network Tab
3. Select: Offline
4. Refresh page.


You are Offline
localhost:7089
You are Offline
It looks like you are not connected to the Internet.
Please check your Internet connection and try again.
Try Again

Practical no. 7A

Aim: Develop a PWA that works in offline mode using Service Worker.

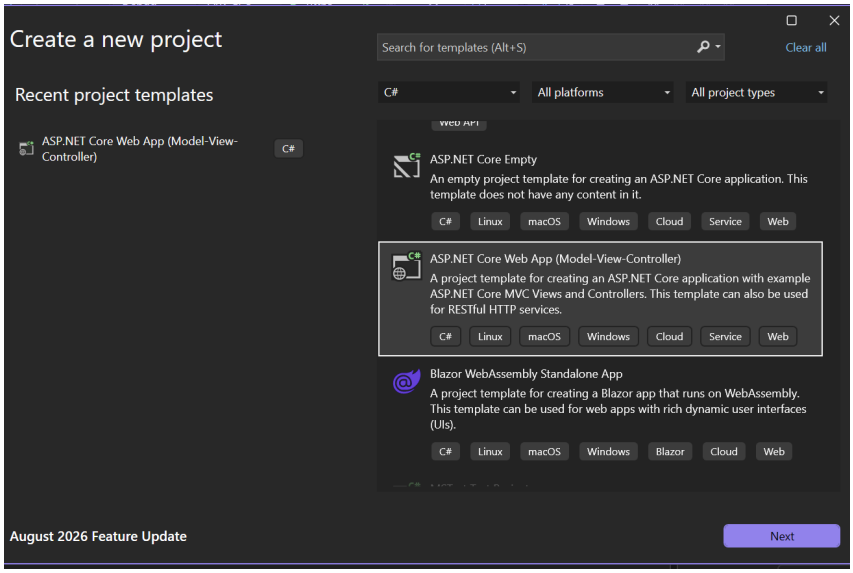
Software Required:

Visual Studio 2026

Procedure:

STEP 1: CREATE NEW PROJECT

1. Open Visual Studio 2026.
2. Click: Create a New Project
3. Open Visual Studio 2026.
4. Select Create a new project.
5. Search for ASP.NET Core Web App (Model-View-Controller).
6. Click Next.
7. Set the Project Name to Practical No. 6b.
8. Select a suitable location and click Next.
9. Select the installed/latest .NET version and keep the default options.
10. Click Create.



STEP 2: CREATE HOME PAGE

Open Views → Home → Index.cshtml and replace its contents with:

```
@{
    ViewData["Title"] = "Offline PWA";
}
```

```
<div class="text-center">

    <h1>Offline PWA</h1>

    <h2>Student Information Portal</h2>

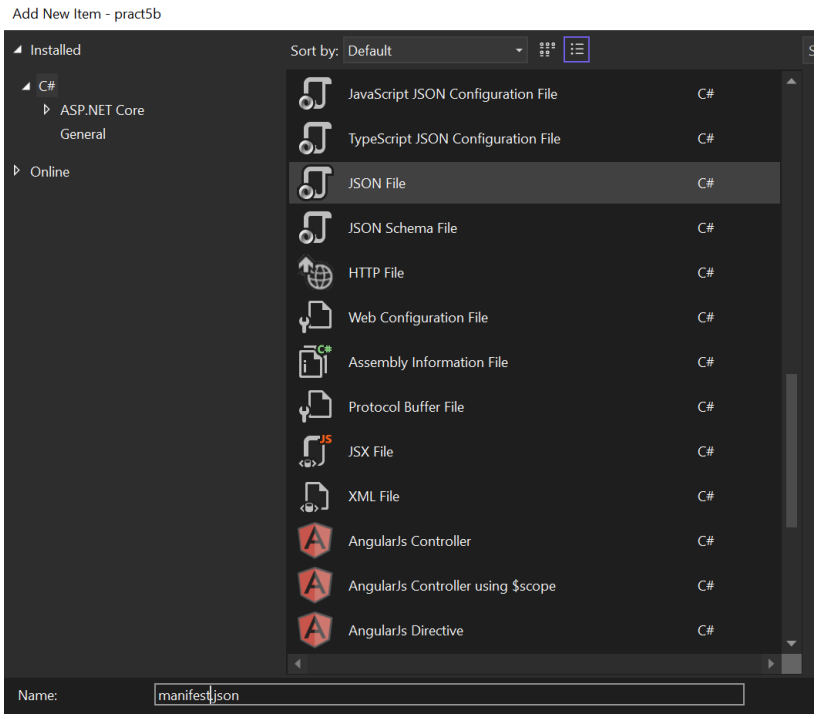
    <p>Welcome to the Student Information Portal.</p>

    <p>This page can also be accessed offline.</p>

</div>
<script src="~/js/site.js"></script>
```

Step 3: Create Manifest File

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select json file.
4. Enter File Name: **manifest.json**
5. Click Add.



Write the following code in the manifest.json file.

manifest.json

```
{

  "name": "Offline PWA",
   "short_name": "OfflinePWA",
   "start_url": "/",
   "display": "standalone",
   "background_color": "#ffffff",
   "theme_color": "#007bff"

}
```

Step 4: Create Service Worker

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select Javascript file.
4. Enter File Name: **service-worker.js**
5. Click Add.

```
const CACHE_NAME = "my-pwa-cache";

self.addEventListener("install", function (event) {

   event.waitUntil(
      caches.open(CACHE_NAME).then(function (cache) {

         return cache.add("/");

      })
   );

});

self.addEventListener("fetch", function (event) {

   event.respondWith(

      caches.match(event.request).then(function (response) {
```

```
        if (response) {
            return response;
        }

        return fetch(event.request);

    })

  );

});
```

Step 5: Register the Service Worker
Open:

wwwroot → js → site.js

Add:

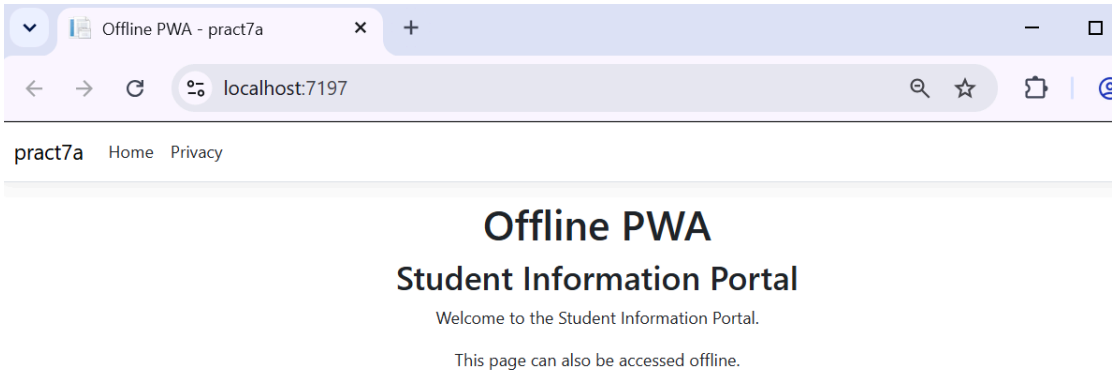
```
if ("serviceWorker" in navigator) {

  navigator.serviceWorker.register("/service-worker.js");

}
```

Step 6: Run the Application

Press F5



Step 7: Check the Service Worker
Press F12 → Application → Service Workers.

Check that service-worker.js is registered and running.

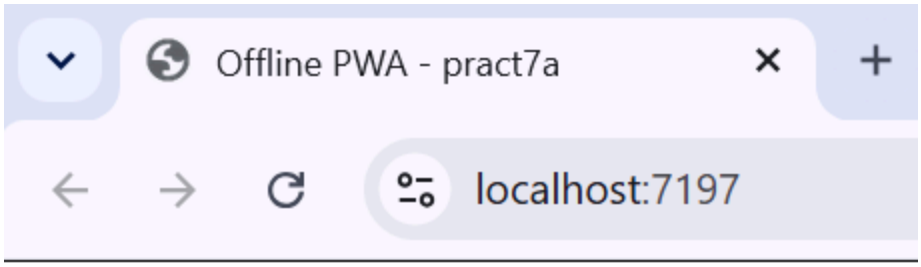
Step 8: Check Cache Storage

In Developer Tools, go to: Application→Cache Storage→Click: my-pwa-cache.

You should see: /

Step 10: Test Offline Mode

1. Press: F12
2. Open: Network Tab
3. Select: Offline
4. Refresh page.



pract7a

- Home
- Privacy

Offline PWA

Student Information Portal

Welcome to the Student Information Portal.

This page can also be accessed offline.

© 2026 - pract7a - Privacy

Practical no. 7B

Aim: Create a Student Record Management PWA.

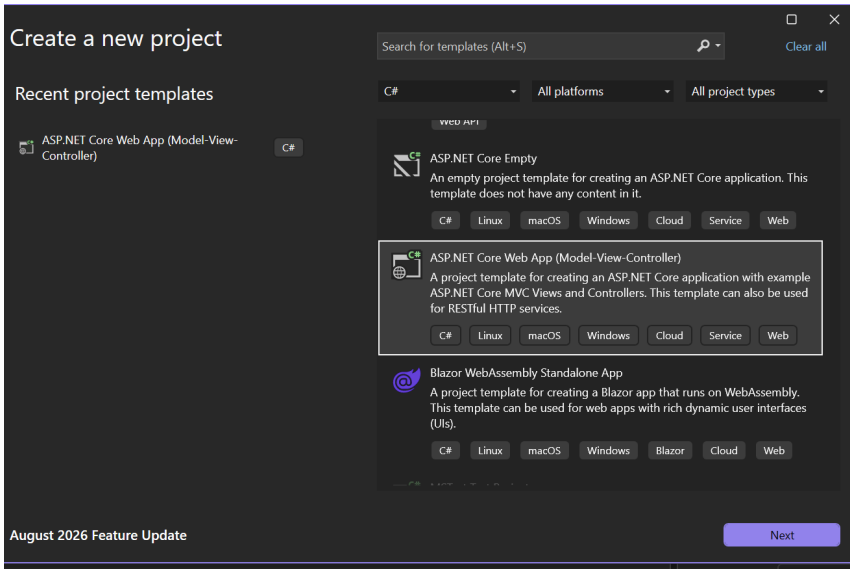
Software Required:

Visual Studio 2026

Procedure:

STEP 1: CREATE NEW PROJECT

1. Open Visual Studio 2026.
2. Select Create a new project.
3. Search for ASP.NET Core Web App (Model-View-Controller).
4. Click Next.
5. Set the Project Name to Practical No. 6b.
6. Select a suitable location and click Next.
7. Select the installed/latest .NET version and keep the default options.
8. Click Create.



STEP 2: CREATE HOME PAGE

Open Views → Home → Index.cshtml and replace its contents with:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Records</title>

    <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
    <link rel="stylesheet" href="site.css">

</head>

<body>

    <h1>Student Record Management</h1>

    <h2>Student Records</h2>

    <table>
        <tr>
            <th>Roll No</th>
            <th>Name</th>
            <th>Course</th>
            <th>Marks</th>
        </tr>

        <tr>
            <td>101</td>
            <td>Rahul</td>
            <td>BSc IT</td>
            <td>85</td>
        </tr>

        <tr>
            <td>102</td>
            <td>Priya</td>
            <td>BSc IT</td>
            <td>91</td>
        </tr>

        <tr>
            <td>103</td>
            <td>Amit</td>
            <td>BSc IT</td>
            <td>78</td>
        </tr>
    </table>

    <script src="site.js"></script>

</body>
```

```
</html>
```

Step 3: Open site.css
Open:

wwwroot → css → site.css

Add the following CSS:

```
body {
    font-family: Arial;
    text-align: center;
    background-color: #f2f2f2;
}

h1 {
    background-color: blue;
    color: white;
    padding: 20px;
}

table {
    margin: auto;
    border-collapse: collapse;
    background-color: white;
}

th, td {
    border: 1px solid black;
    padding: 10px;
}

th {
    background-color: blue;
    color: white;
}
```

Step 4: Register the Service Worker
Open:

wwwroot → js → site.js

Add:

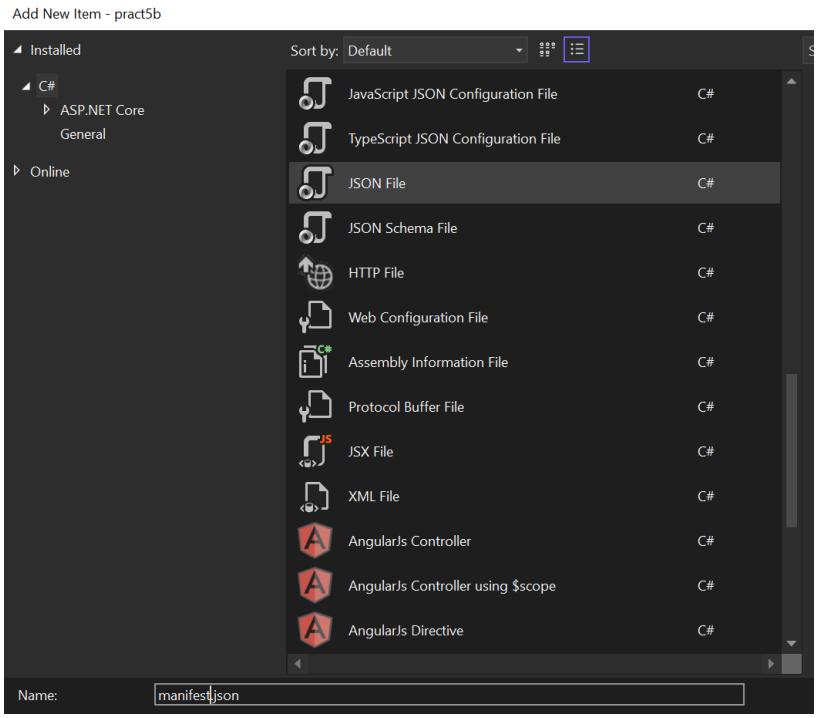
```
if ("serviceWorker" in navigator) {

    navigator.serviceWorker.register("service-worker.js")
        .then(function () {
            console.log("Service Worker Registered");
```

```
    })
    .catch(function (error) {
        console.log("Service Worker Error:", error);
    });
}
```

Step 5: Create manifest.json

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select json file.
4. Enter File Name: **manifest.json**
5. Click Add.



Enter:

```
{
 "name": "Student Record Management",
 "short_name": "Student PWA",
 "start_url": "/",
 "display": "standalone",
 "theme_color": "blue",
 "background_color": "white"
}
```

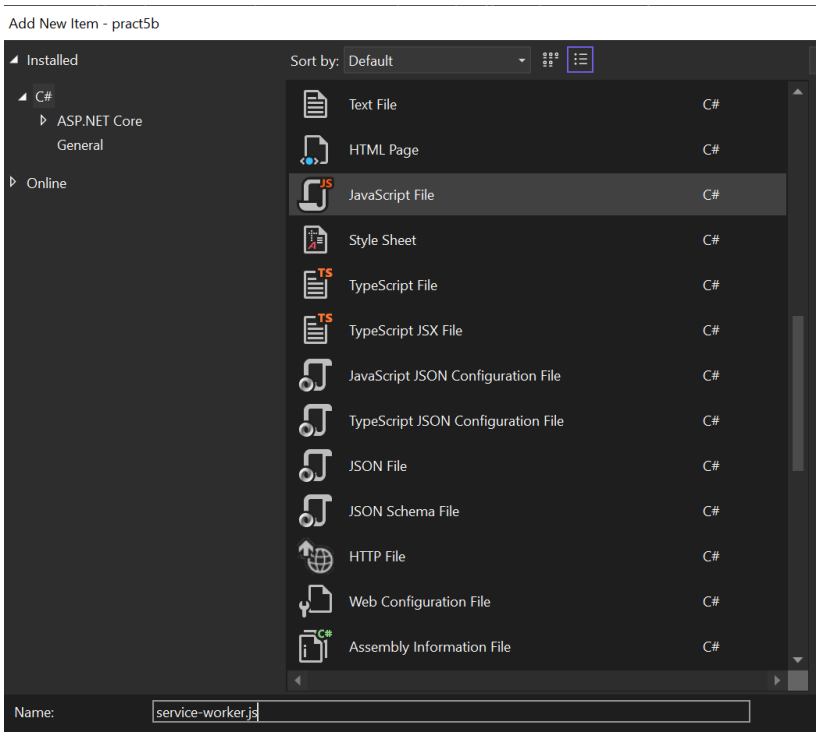
Step 6 : Register the Manifest
Open Views → Shared → _Layout.cshtml.

Inside the <head> section, add:

<link rel="manifest" href="/manifest.json">
<meta name="theme-color" content="#0d6efd">

Step 7: Create Service Worker

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select JavaScript file.
4. Enter File Name: **service-worker.js**
5. Click Add.



Write the following code in the service-worker.js file.

service-worker.js

```
const CACHE_NAME = "student-pwa-v1";
const FILES = [
    "/",
    "/Home/Index",
    "/css/site.css",
    "/js/site.js",
```

```
  "/manifest.json"
];

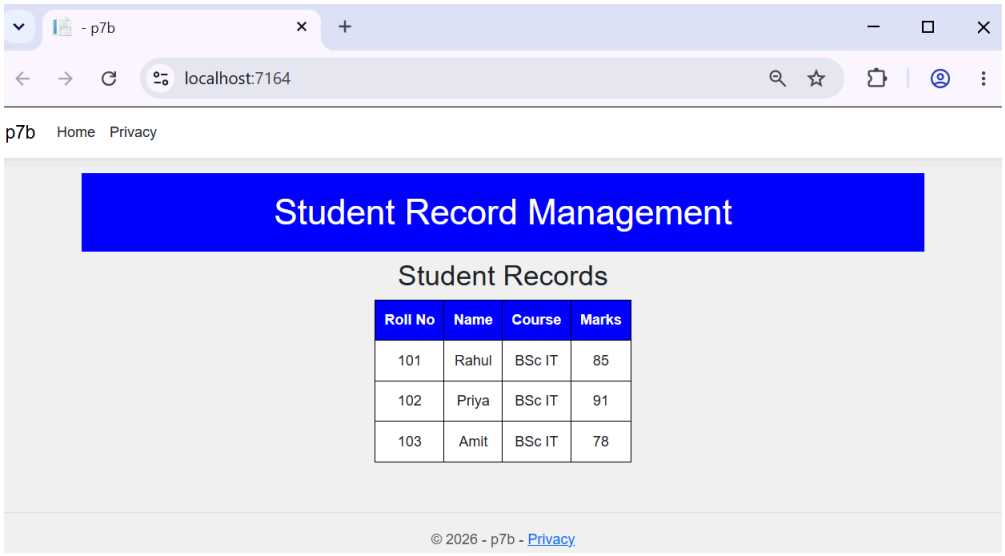
self.addEventListener("install", function (event) {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(function (cache) {
        return cache.addAll(FILES);
      })
  );

});

self.addEventListener("fetch", function (event) {
  event.respondWith(
    caches.match(event.request)
      .then(function (response) {
        return response || fetch(event.request);
      })
  );
});
```

Step 8: Run the Application

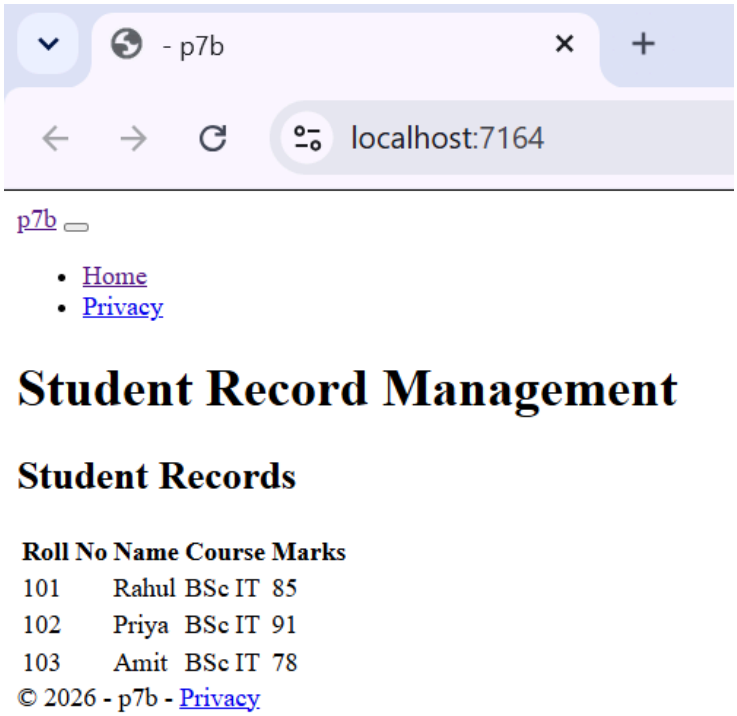
Press F5



Step 9: Test Offline Mode

1. Press: F12

2. Open: Network Tab
3. Select: Offline
4. Refresh page.



Step 10: Install the PWA

1. Click Chrome Menu: ⋮ (Three Dots)
2. Select: Cast, Save and Share
3. Then click: Install Page as App or Install College Information Portal

Practical no. 8A

Aim: Create an HTML page that would be the starting point of the application. This HTML will contain a link to the file named manifest.json. This is an important file that would be created in the next step.

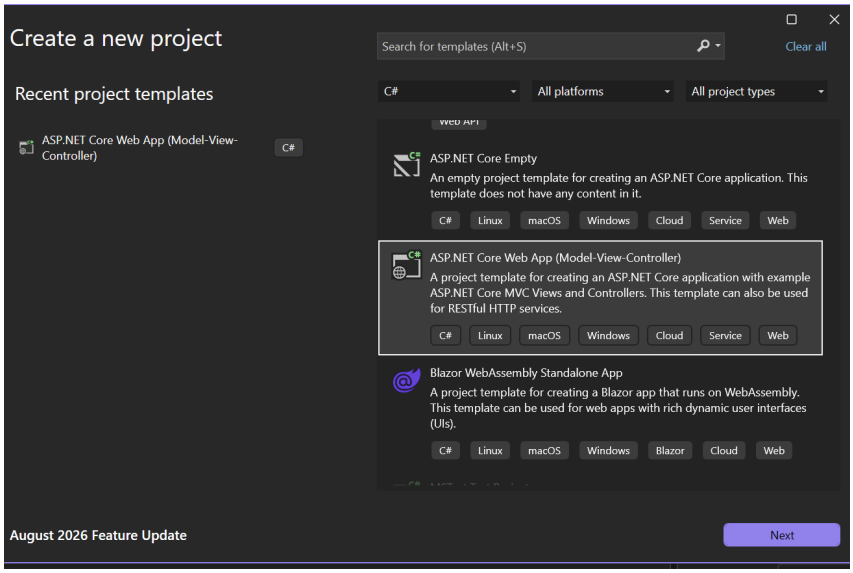
Software Required:

Visual Studio 2026

Procedure:

STEP 1: CREATE NEW PROJECT

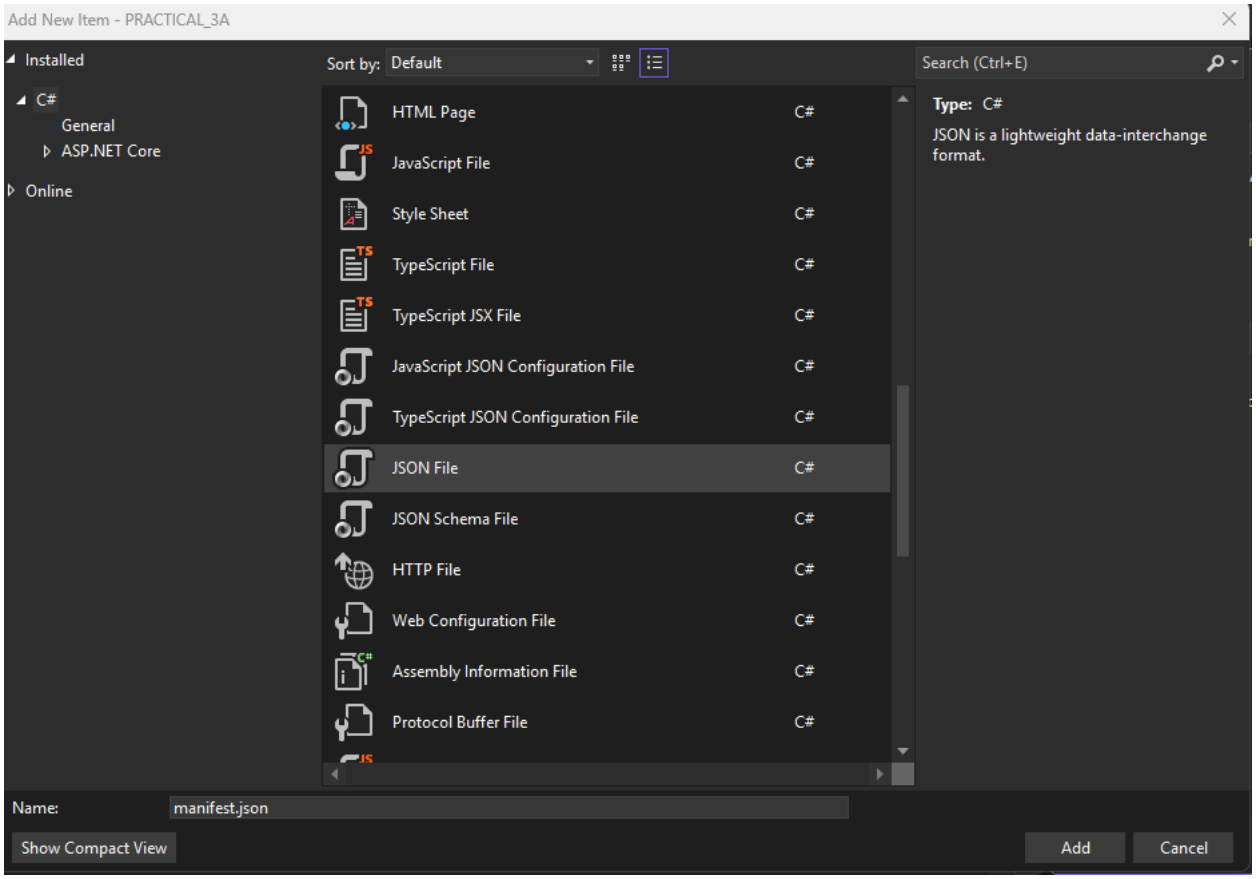
1. Open Visual Studio 2026.
2. Click: Create a New Project
3. Open Visual Studio 2026.
4. Select Create a new project.
5. Search for ASP.NET Core Web App (Model-View-Controller).
6. Click Next.
7. Set the Project Name to Practical No. 6b.
8. Select a suitable location and click Next.
9. Select the installed/latest .NET version and keep the default options.
10. Click Create.



Step 5: Create Manifest File

1. Right Click → **wwwroot** folder
2. Click Add → New item

3. Select JSON file.
4. Enter File Name: **manifest.json**
5. Click Add.



Write the following code in the manifest.json file.

manifest.json

```
{
 "name": "My First PWA",
 "short_name": "MyPWA",
 "start_url": "/",
 "display": "standalone",
 "background_color": "#ffffff",
 "theme_color": "#007bff"
}
```

STEP 3: CREATE HOME PAGE

Open Views → Home → Index.cshtml and replace its contents with:

```
@{
    Layout = null;
}

<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />

    <title>My First PWA</title>
    <link rel="manifest" href="/manifest.json" />

</head>

<body>

    <h1>Welcome to My First PWA</h1>

    <p>This is the starting page of my Progressive Web Application.</p>

</body>
</html>
```

Step 4: Run the Application
Ctrl + F5

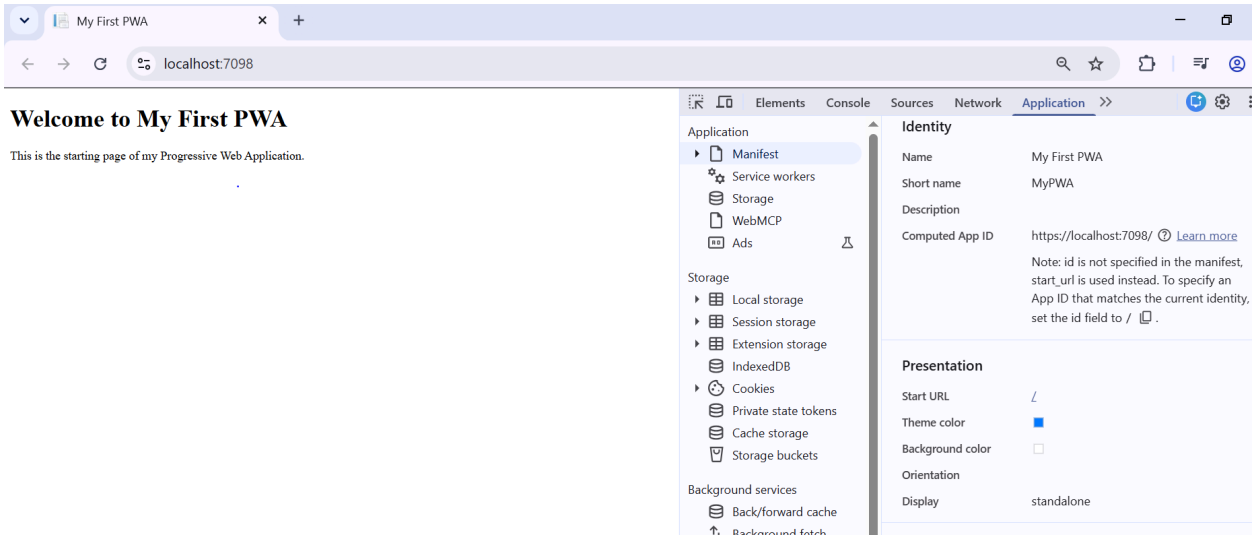


Step 5: Verify Manifest File

1. Press: F12
2. Open: Application → Manifest

You should see:

1. Application Name
2. Short Name
3. Theme Color
4. Background color
5. Display Mode



Practical no. 8B

Aim: Create a simple Web API that returns Student data in JSON format.

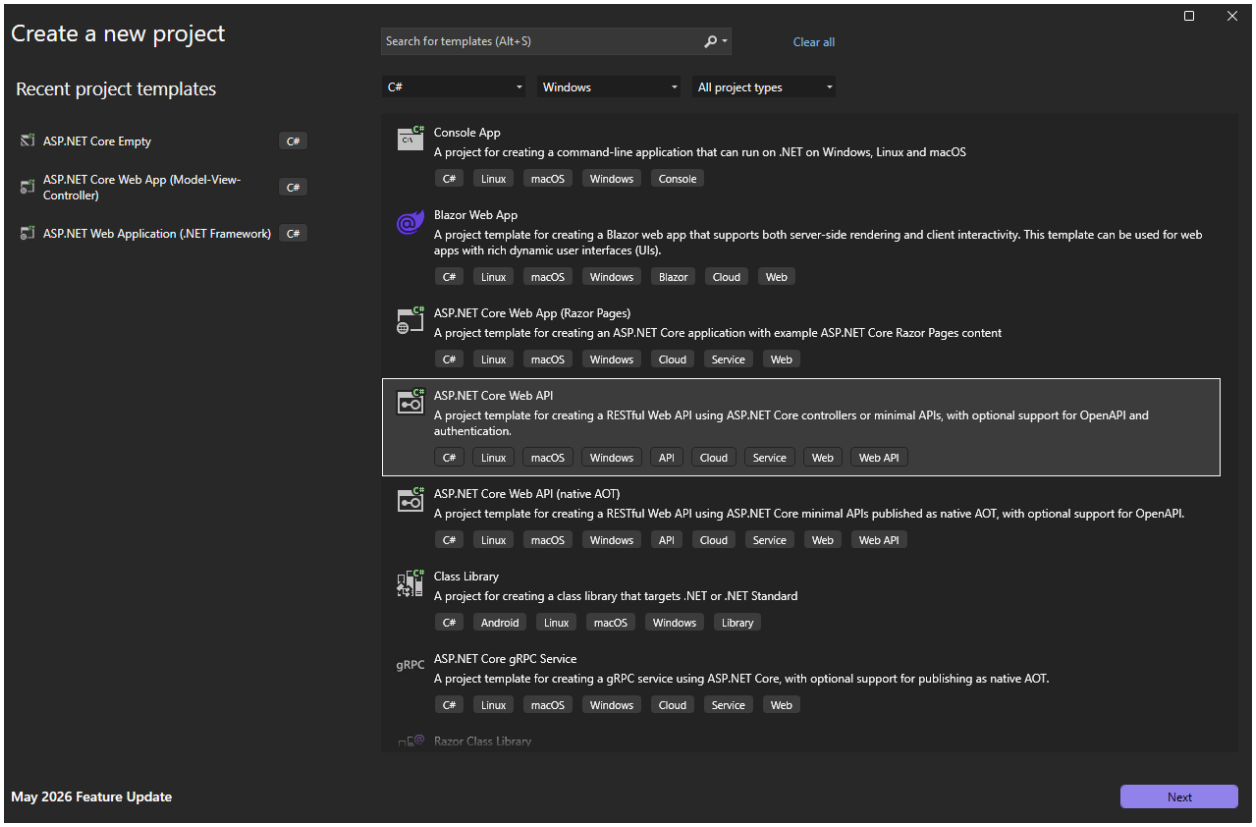
Software Required:

Visual Studio 2026

Procedure:

STEP 1: CREATE NEW PROJECT

1. Open Visual Studio 2026.
2. Click: Create a New Project
3. Select: **ASP.NET Core Web API**
4. Click Next.
5. Enter: Project Name: Practical 8B
6. Click Next.
7. Select: Framework: .NET 10 (or latest)
   Authentication Type: None
   Check: Configure for HTTPS
8. Click Create.



Step 2: Create Student Model

1. Click View.
2. Click Solution Explorer..
3. Right click project name.
4. Add→ New folder.
5. Enter Name: **Models**
6. Right Click Models.
7. Click Add.
8. Click New item.
9. Select Class.
10. Give Name **Student.cs.**
11. Click Add.

Student.cs

```
namespace Practical_8b.Models
{
   public class Student
   {
      public int Id { get; set; }

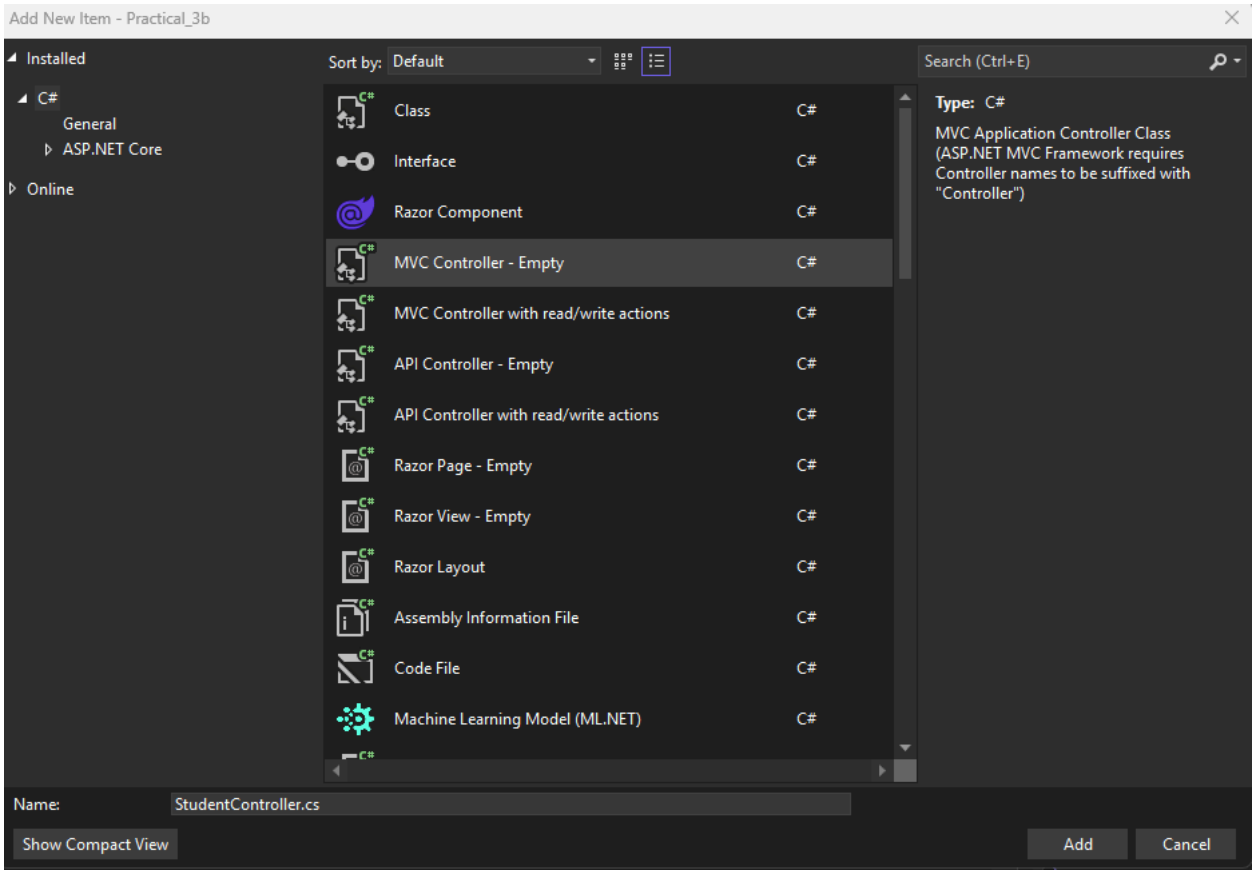
      public string Name { get; set; }

      public string Course { get; set; }
   }
}
```

Step 3: Create Controller

Create a StudentController inside the Controllers folder.

1. Right Click Controllers.
2. Click Add.
3. Click New item.
4. Select MVC Controller - Empty.
5. Give Name **StudentController**.
6. Click Add.



StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
using Practical_3b.Models;

namespace Practical_8b.Controllers
{
   [ApiController]
   [Route("api/[controller]")]
   public class StudentController : ControllerBase
   {
      [HttpGet]
      public IActionResult GetStudents()
      {
         List<Student> students = new List<Student>()
         {
            new Student
            {
```

```
                Id = 101,
                Name = "Rahul",
                Course = "BSc IT"
            },

            new Student
            {
                Id = 102,
                Name = "Priya",
                Course = "BSc CS"
            },

            new Student
            {
                Id = 103,
                Name = "Amit",
                Course = "BCA"
            }
        };

        return Ok(students);
    }
  }
}
```

Step 4: Modify Program.cs

Solution Explorer → Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Add Controller Services
builder.Services.AddControllers();

var app = builder.Build();

// Configure HTTP Request Pipeline
app.UseHttpsRedirection();

// Map Controllers
app.MapControllers();
```

app.Run();

Step 8: Build the Project

Click: Build → Build Solution

OR

Ctrl + Shift + B

Step 9: Run the Application

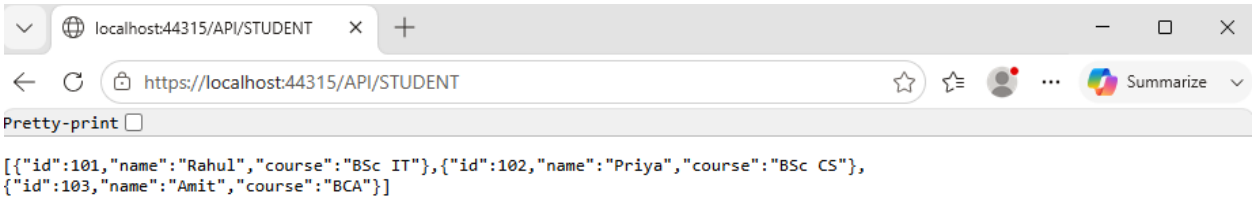
Ctrl + F5

Step 10: Test API

Manually add: /api/student

Example:

https://localhost:44315/api/student



Practical no. 9A

Aim: Create a PWA with a Web App Manifest file including:
App name
Icons
Start URL
Display mode
Implement Push Notifications in a PWA.

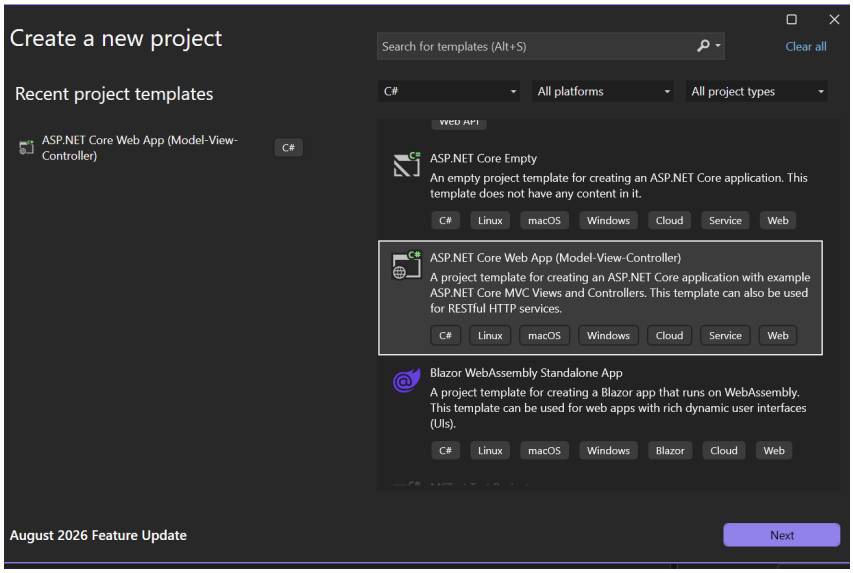
Software Required:

Visual Studio 2026

Procedure:

STEP 1: CREATE NEW PROJECT

1. Open Visual Studio 2026.
2. Click: Create a New Project
3. Open Visual Studio 2026.
4. Select Create a new project.
5. Search for ASP.NET Core Web App (Model-View-Controller).
6. Click Next.
7. Set the Project Name to Practical No. 9A.
8. Select a suitable location and click Next.
9. Select the installed/latest .NET version and keep the default options.
10. Click Create.



STEP 2: CREATE HOME PAGE

Open Views → Home → Index.cshtml and replace its contents with:

```
@{
    ViewData["Title"] = "PWA Home";
}

<div class="text-center">

    <h1>TYIT Student PWA</h1>

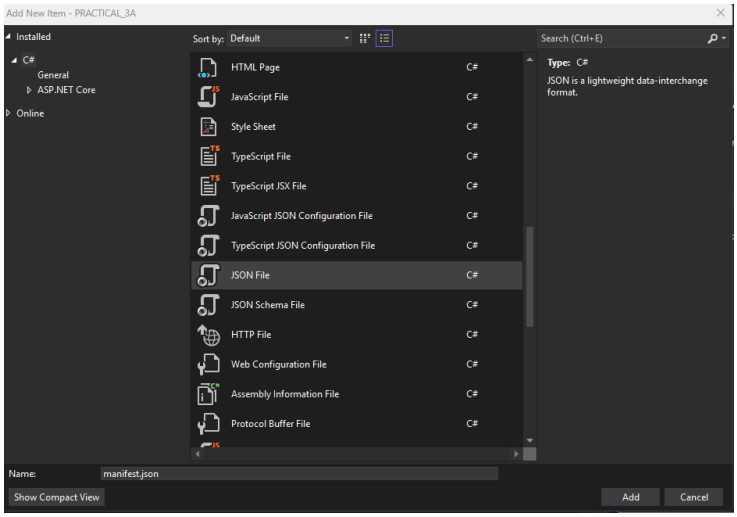
    <p>Progressive Web Application</p>

    <button id="notifyButton" class="btn btn-primary">
        Enable Notification
    </button>

</div>
<script src="/js/site.js"></script>
```

Step 5: Create Manifest File

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select JSON file.
4. Enter File Name: **manifest.json**
5. Click Add.



Write the following code in the manifest.json file.

manifest.json

```
{
 "name": "TYIT Student PWA",
 "short_name": "TYIT PWA",
 "start_url": "/",
 "display": "standalone",
 "background_color": "#ffffff",
 "theme_color": "#0d6efd",
 "icons": [
  {
   "src": "/icons/sdsm.png",
   "sizes": "192x192",
   "type": "image/png"
  },
  {
   "src": "/icons/sdsm.png",
   "sizes": "512x512",
   "type": "image/png"
  }
 ]
}
```

Step 6: Add App Icon

1. Right Click → **wwwroot** folder
2. Click Add → New Folder
3. Enter Folder Name: **icons**
4. Paste one image in the **icons** folder.

Step 7: Edit _Layout.cshtml
Open:

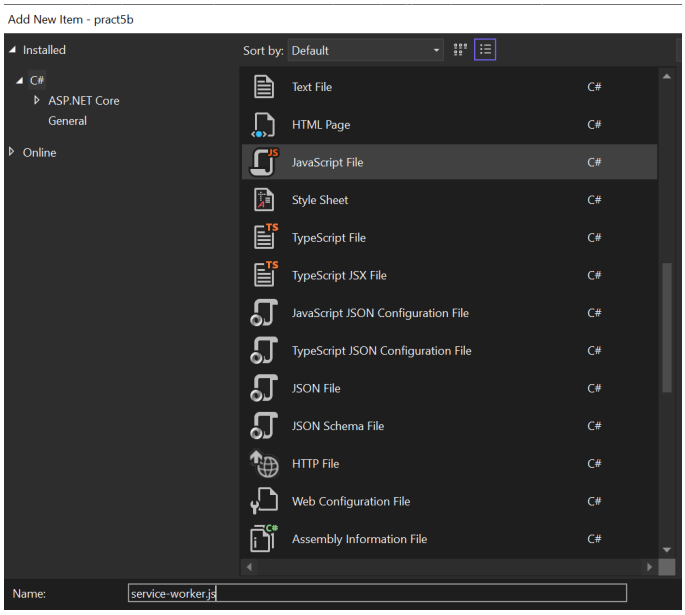
Views → Shared → _Layout.cshtml

Inside the <head> section, add:

```
<link rel="manifest" href="/manifest.json">
```

Step 8: Create Service Worker

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select JavaScript file.
4. Enter File Name: **service-worker.js**
5. Click Add.



Write the following code in the service-worker.js file.

service-worker.js

```
self.addEventListener("install", function (event) {
    console.log("Service Worker Installed");
});

self.addEventListener("activate", function (event) {
    console.log("Service Worker Activated");
});
```

Step 9: Register the Service Worker
Open:

wwwroot → js → site.js

Add:

```
// Register Service Worker

if ("serviceWorker" in navigator) {

    window.addEventListener("load", function () {

        navigator.serviceWorker.register("/service-worker.js")
            .then(function (registration) {
```

```
            console.log("Service Worker Registered");

        })
        .catch(function (error) {

            console.log("Service Worker Registration Failed", error);

        });

    });

}

// Notification Button

document.getElementById("notifyButton").addEventListener("click", function () {

    if ("Notification" in window) {

        Notification.requestPermission().then(function (permission) {

            if (permission === "granted") {

                new Notification("TYIT Student PWA", {
                    body: "Welcome! Notification is working successfully.",
                    icon: "/icons/sdsm.png"
                });

            }

        });

    } else {

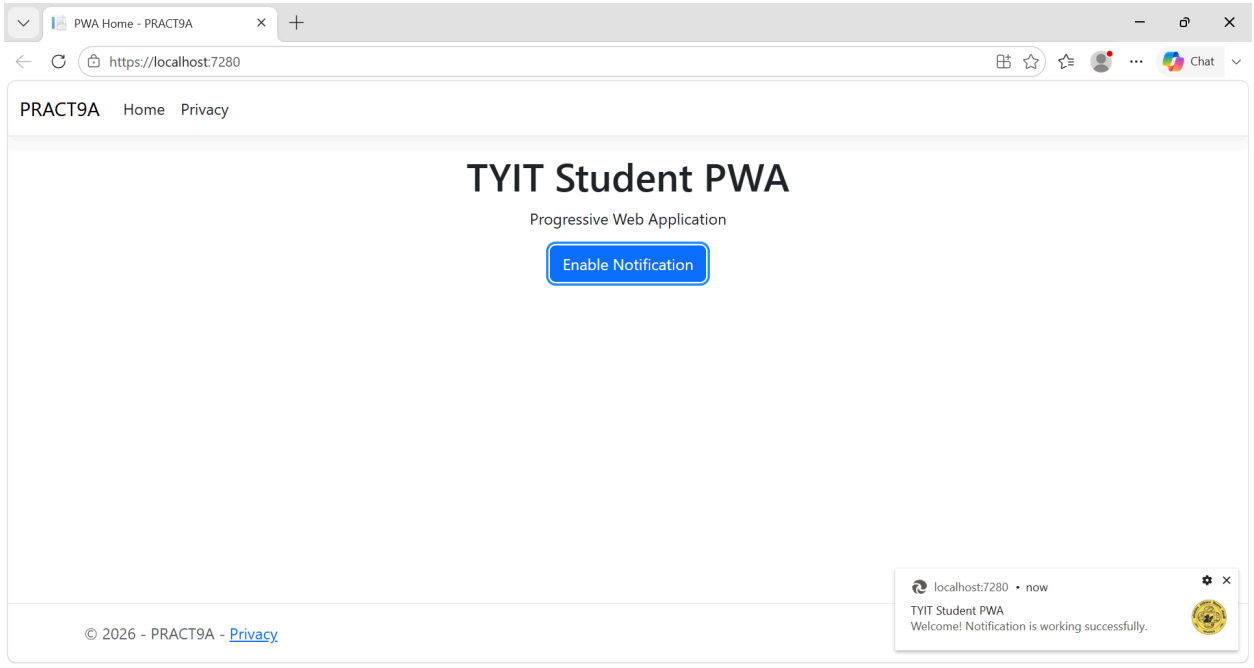
        alert("Notifications are not supported by this browser.");

    }

});
```

Step 10: Run the application
Press Ctrl + F5


PWA Home - pract9a
Get notifications?
localhost:5090
pract9a
http://localhost:5090 wants to
Show notifications
Allow
Block
TYIT Student PWA
Progressive Web Application
Enable Notification


PWA Home - PRACT9A
https://localhost:7280
Chat
PRACT9A
Home
Privacy
TYIT Student PWA
Progressive Web Application
Enable Notification
localhost:7280 • now
TYIT Student PWA
Welcome! Notification is working successfully.
© 2026 - PRACT9A - Privacy

Practical no. 9B

Aim: Develop an E-Commerce PWA with product listing and offline cart functionality.

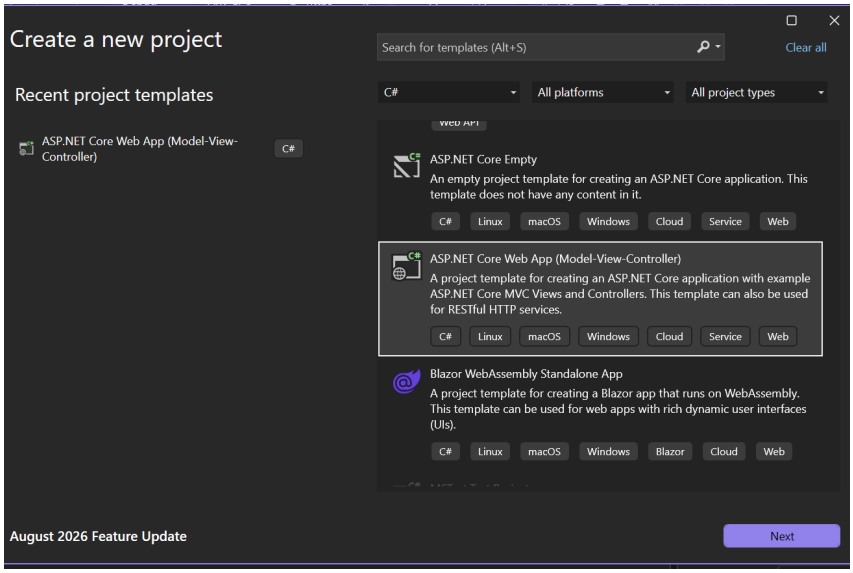
Software Required:

Visual Studio 2026

Procedure:

STEP 1: CREATE NEW PROJECT

1. Open Visual Studio 2026.
2. Click: Create a New Project
3. Open Visual Studio 2026.
4. Select Create a new project.
5. Search for ASP.NET Core Web App (Model-View-Controller).
6. Click Next.
7. Set the Project Name to Practical No. 9A.
8. Select a suitable location and click Next.
9. Select the installed/latest .NET version and keep the default options.
10. Click Create.



STEP 2: CREATE HOME PAGE

Open Views → Home → Index.cshtml and replace its contents with:

```
@{
```

```
    ViewData["Title"] = "E-Commerce PWA";
}



<h1>E-Commerce Store</h1>

<hr>

<h2>Products</h2>

<div>

    <h3>Laptop</h3>
    <p>Price: ₹50000</p>

    <button onclick="addToCart('Laptop', 50000)">
        Add to Cart
    </button>

</div>

<hr>

<div>

    <h3>Mobile Phone</h3>
    <p>Price: ₹20000</p>

    <button onclick="addToCart('Mobile Phone', 20000)">
        Add to Cart
    </button>

</div>

<hr>

<div>

    <h3>Headphones</h3>
    <p>Price: ₹2000</p>

    <button onclick="addToCart('Headphones', 2000)">
        Add to Cart
```

```
    </button>

</div>

<hr>

<h2>Shopping Cart</h2>

<div id="cart">
</div>

<h3 id="total">
</h3>


<script src="/js/site.js"></script>
```

Step 3: Create site.js file
Open:

wwwroot → js → site.js

Add:

```
let cart = JSON.parse(localStorage.getItem("cart")) || [];

function addToCart(name, price) {

    cart.push({
        name: name,
        price: price
    });

    localStorage.setItem("cart", JSON.stringify(cart));

    displayCart();
}

function displayCart() {

    let cartDiv = document.getElementById("cart");
    let total = 0;

    cartDiv.innerHTML = "";
```

```
    cart.forEach(function (product) {

        cartDiv.innerHTML +=
            product.name + " - ₹" + product.price + "<br>";

        total = total + product.price;
    });

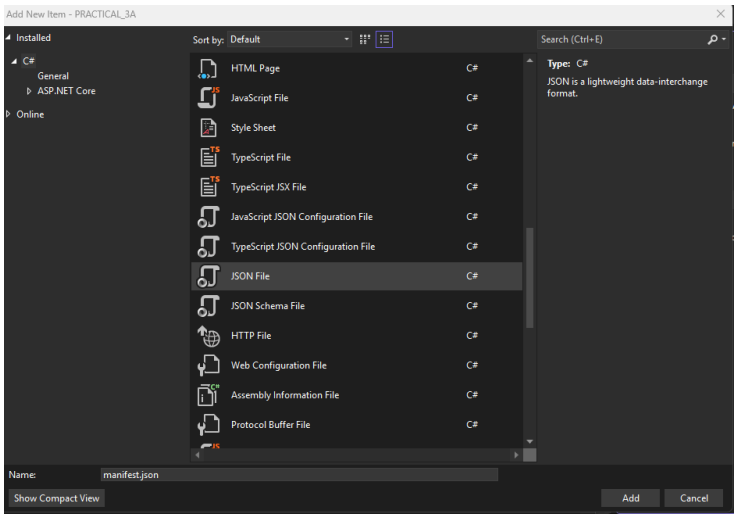
    document.getElementById("total").innerHTML =
        "Total: ₹" + total;
}

displayCart();

if ("serviceWorker" in navigator) {
    navigator.serviceWorker.register("/service-worker.js");
}
```

Step 4: Create Manifest File

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select JSON file.
4. Enter File Name: **manifest.json**
5. Click Add.



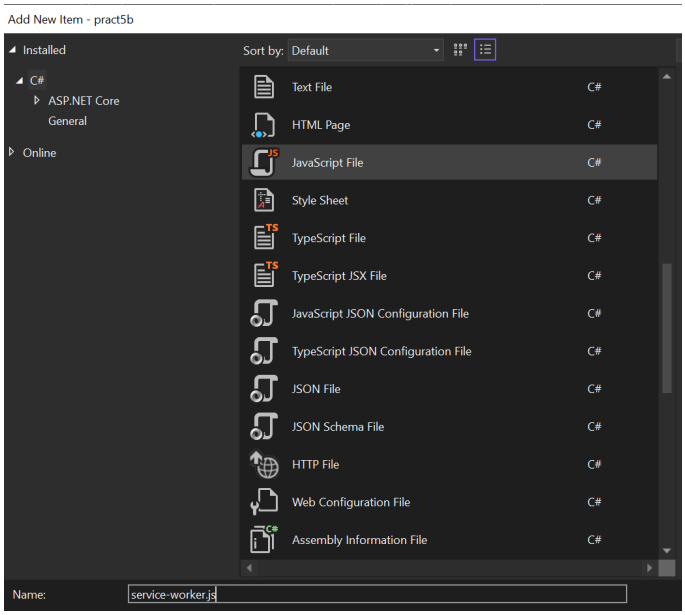
Write the following code in the manifest.json file.

manifest.json

```
{
 "name": "E-Commerce Store",
 "short_name": "E-Store",
 "start_url": "/",
 "display": "standalone"
}
```

Step 5: Create Service Worker

1. Right Click → **wwwroot** folder
2. Click Add → New item
3. Select JavaScript file.
4. Enter File Name: **service-worker.js**
5. Click Add.



Write the following code in the service-worker.js file.

service-worker.js

```
let cacheName = "shop-cache";

self.addEventListener("install", function (event) {

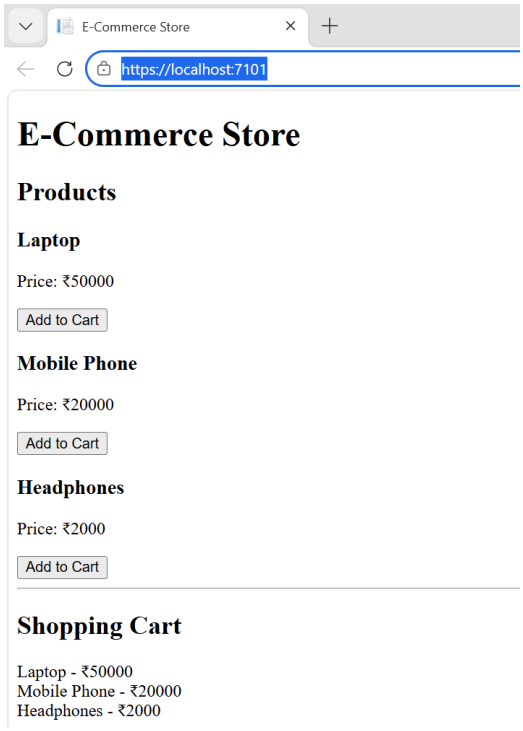
    event.waitUntil(

        caches.open(cacheName).then(function (cache) {

            return cache.add("/");
```

```
        })
    );
});
self.addEventListener("fetch", function (event) {
    event.respondWith(
        caches.match(event.request).then(function (response) {
            return response || fetch(event.request);
        })
    );
});
```

Step 6: Run the application
Press Ctrl + F5



Step 7: Test Offline Functionality
Open F12 → Network. → Select the Offline option and refresh the page.

